



中国科学院大学

University of Chinese Academy of Sciences

硕士学位论文

面向 GPU 上异步访存与计算的算子流水线自动调度研究

作者姓名：_____陈茜_____

指导教师：_____肖俊敏 副研究员 中国科学院计算技术研究所_____

学位类别：_____工学硕士_____

学科专业：_____计算机科学与技术_____

培养单位：_____中国科学院计算技术研究所_____

2026 年 6 月

Research on Automatic Scheduling of Operator Pipelines for
Asynchronous Memory Access and Computation on GPUs

A thesis submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Master of Engineering
in Computer Science and Technology

By

CHEN Xi

Supervisor: Associate Professor XIAO Junmin

Institute of Computing Technology, Chinese Academy of Sciences

June, 2026

中国科学院大学 学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。承诺除文中已经注明引用的内容外，本论文不包含任何其他个人或集体享有著作权的研究成果，未在以往任何学位申请中全部或部分提交。对本论文所涉及的研究工作做出贡献的其他个人或集体，均已在文中以明确方式标明或致谢。本人完全意识到本声明的法律结果由本人承担。

作者签名：

日 期：

中国科学院大学 学位论文授权使用声明

本人完全了解并同意遵守中国科学院大学有关收集、保存和使用学位论文的规定，即中国科学院大学有权按照学术研究公开原则和保护知识产权的原则，保留并向国家指定或中国科学院指定机构送交学位论文的电子版和印刷版文件，且电子版与印刷版内容应完全相同，允许该论文被检索、查阅和借阅，公布本学位论文的全部或部分内容，可以采用扫描、影印、缩印等复制手段以及其他法律许可的方式保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：

日 期：

导师签名：

日 期：

摘要

GPU 架构正在向异步化演进，专用数据搬运单元与计算单元独立工作，依赖软件流水线重叠执行来隐藏访存延迟。NVIDIA Hopper 架构引入张量内存加速器（Tensor Memory Accelerator, TMA）和线程束组矩阵乘累加（Warpgroup Matrix Multiply-Accumulate, WGMMMA）等异步硬件单元，显著提高了峰值吞吐，也把数据搬运、张量计算和同步屏障的排布暴露给软件。现有深度学习编译器已经提供 Tile 级编程抽象，但在 Hopper 上自动生成接近手工优化算子库的异步流水线内核仍然困难。

上述困难主要来自三个方面。首先，异步流水线的配置空间很大，可行配置占比很低。以 H100 上 FlashAttention 前向为例，在固定语句分组后，仅组间顺序与流水线阶段就产生 2328 个候选，其中约 10% 能成功编译运行；若进一步同时枚举分组、排序和阶段映射，候选数可超过 10^6 。其次，硬件指令模式、操作数布局、分块形状和算子分区共同决定寄存器与共享内存占用，单独调优某一维度容易错过更优配置。最后，现有线程束特化机制的同步粒度较粗，消费者顺序通常固定，软件流水与线程束特化分开处理时容易在复杂算子中引发寄存器溢出。

针对上述问题，本文依托 TileLang 代码生成流程，提出 AutoMixed、AutoPiped 和 Heddle 三个编译模块。AutoMixed 采用资源感知的图级优化策略，估计寄存器和共享内存占用，选择 WGMMMA 操作数布局、算子分区和候选分块形状，在编译早期排除明显超过硬件资源上限的方案。AutoPiped 在循环级分析访存指令、共享内存读写关系和生产者-消费者依赖，生成分组、执行顺序和流水线阶段等配置参数，减少手写流水线注解。Heddle 在线程束特化执行层引入细粒度线程束特化（Fine-Grained Warp Specialization, FGWS），重排消费者语句，优化屏障位置，并检查调度是否超过寄存器、共享内存和功能单元上限；对于同时涉及软件流水和线程束特化的调度问题，Heddle 使用基于约束规划的两阶段求解器联合处理启动时刻、WarpGroup 分工和资源约束。Heddle 还针对 TMA 规约加模式生成三角色线程束特化（Three-Role Warp Specialization, TRWS）配置，减少手动配置。三个模块按编译流水线衔接，AutoMixed 在前端决定图级内核结构，AutoPiped 在给定内核结构上生成流水线配置，Heddle 在最后阶段完成消费者重排、屏障优化和资源检查。

本文在 NVIDIA H100 上评估六类算子和 5 个注意力变体。实验结果表明，完整系统生成的内核在所测配置上高于开源编译器基线，并在多个场景中接近或超过厂商库与专用手写内核：GEMM 最高达到 cuBLAS 吞吐量的 96%，Dequant GEMM 达到 Marlin 的 78%–82%，Conv2D 在部分配置上超过 cuDNN。FlashAttention 前向达到 FlashAttention-3 吞吐量的 76%–93%；FlashAttention 反向在所测配置上相比 TileLang 平均加速 1.38 倍，相比 FlashAttention-3^[1] 平均加速 1.69 倍。端到端评测中，5 个注意力变体相比逐操作 PyTorch 基线实现 11.8–15.6 倍

加速，平均加速 13.2 倍；在 LLaMA-2 7B 模型级推理中，序列长度 4096 时实现 3.41–3.96 倍端到端加速，平均加速 3.62 倍；在 Qwen-2.5 7B 真实模型形状下，标准注意力模型级推理在序列长度 4096 时相比 PyTorch eager 和 torch.compile 分别加速 2.82 倍和 1.35 倍。

关键词：深度学习编译器，异步流水线，线程束特化，GPU 算子优化，约束求解

Abstract

GPU architectures are moving toward asynchronous execution, where dedicated data-movement units and compute units operate independently and rely on software pipelines to overlap memory access with computation. The NVIDIA Hopper architecture introduces the Tensor Memory Accelerator (TMA) and Warpgroup Matrix Multiply-Accumulate (WGMMMA), significantly increasing peak throughput while exposing the placement of data movement, tensor computation, and synchronization barriers to software. Existing deep learning compilers provide tile-level programming abstractions, but automatically generating asynchronous pipeline kernels that approach hand-tuned operator libraries remains difficult on Hopper.

The difficulty comes from three sources. First, the asynchronous pipeline configuration space is large, and only a small fraction of candidates are viable. For FlashAttention forward on H100, fixing statement groups yields 2,328 ordering and pipeline stage candidates, only about 10% of which compile and run correctly; enumerating grouping, ordering, and stage mapping together can exceed 10^6 candidates. Second, hardware instruction modes, operand layouts, tile shapes, and operator partitioning jointly determine register and shared-memory usage, so tuning one dimension in isolation can miss better configurations. Third, existing warp-specialization mechanisms use coarse synchronization and usually keep a fixed consumer order; handling software pipelining and warp specialization separately can trigger register spills in complex operators.

To address these problems, this work builds on the TileLang code generation flow and proposes three compilation modules, AutoMixed, AutoPiped, and Heddle. AutoMixed adopts a resource-aware graph-level optimization strategy to estimate register and shared-memory usage, select WGMMMA operand layouts, operator partitions, and candidate tile shapes, and prune candidates that clearly exceed hardware limits early in compilation. AutoPiped analyzes memory access instructions, shared-memory read/write relations, and producer-consumer dependencies at the loop level, then generates grouping, execution order, and pipeline stage parameters to reduce manual pipeline annotations. Heddle introduces Fine-Grained Warp Specialization (FGWS) at the warp-specialized execution level, reorders consumer statements, optimizes barrier placement, and checks register, shared-memory, and functional-unit limits; for schedules that involve both software pipelining and warp specialization, it uses a two-phase constraint-programming solver to jointly handle issue times, WarpGroup assignment, and resource constraints. Heddle also generates Three-Role Warp Specialization (TRWS) configurations for TMA reduce-add patterns, reducing manual configuration. The three modules are connected along the compilation pipeline. AutoMixed determines graph-level ker-

nel structure at the frontend, AutoPiped generates pipeline configurations for the given kernel structure, and Heddle performs consumer reordering, barrier optimization, and resource checking in the final stage.

This work evaluates six operator families and 5 attention variants on NVIDIA H100. The results show that kernels generated by the complete system outperform the evaluated open-source compiler baselines on the measured configurations, and approach or surpass vendor libraries and specialized hand-tuned kernels in several scenarios: GEMM reaches up to 96% of cuBLAS throughput, Dequant GEMM reaches 78%–82% of Marlin throughput, and Conv2D exceeds cuDNN on selected configurations. On FlashAttention forward, it reaches 76%–93% of FlashAttention-3 throughput; on the measured FlashAttention backward configurations, it averages 1.38× over TileLang and 1.69× over FlashAttention-3^[1]. In end-to-end evaluation, 5 attention variants achieve 11.8–15.6× speedup over an operator-wise PyTorch baseline, averaging 13.2×; on LLaMA-2 7B model-level inference, they achieve 3.41–3.96× end-to-end speedup at sequence length 4096, averaging 3.62×. On the real model shape of Qwen-2.5 7B, standard-attention model-level inference achieves 2.82× over PyTorch eager and 1.35× over torch.compile at sequence length 4096.

Key Words: Deep Learning Compiler, Asynchronous Pipeline, Warp Specialization, GPU Operator Optimization, Constraint Solving

目 录

第 1 章 绪论	1
1.1 研究背景	1
1.2 研究意义	2
1.3 研究内容	5
1.4 全文组织	6
第 2 章 背景知识与相关工作	9
2.1 GPU 架构及编程模型的演进	10
2.1.1 Ampere 架构下的显式同步批处理模型	11
2.1.2 Hopper 架构下的异步编程模型	12
2.2 算子优化的主要途径	15
2.2.1 手写高性能算子	15
2.2.2 高层优化框架与系统	17
2.2.3 深度学习编译器	18
第 3 章 AutoMixed: 资源感知的图级优化策略	25
3.1 从 PyTorch 到 TIR 的端到端编译流程	26
3.2 基于指令级实现变体的搜索空间扩展	27
3.3 基于连通子图的候选融合枚举	28
3.4 资源约束驱动的剪枝与决策	28
3.5 布局约束的推导与传播	30
3.6 候选配置评估与分区选择	31
3.7 FlashAttention 反向传播中的图级分区选择	31
3.7.1 融合策略分析	31
3.7.2 子图枚举与候选评估	33
3.7.3 代价分析	33
3.8 面向内存受限操作的融合决策	34
3.9 本章小结	35
第 4 章 AutoPiped: 语义驱动的流水线配置参数生成方法	37
4.1 循环体分析与配置推断	38
4.1.1 语句角色识别	38
4.1.2 分组模板生成与选择	38
4.1.3 阶段推断	40

4.2 执行顺序分配与 ASAP 顺序优化	42
4.3 FlashAttention 前向计算案例分析	45
4.4 本章小结	47
第 5 章 Heddle: 面向细粒度线程束特化的自动流水线调度框架	49
5.1 原有线程束特化机制的结构约束	50
5.2 细粒度线程束特化机制	51
5.3 时间维调度与消费者重排	53
5.3.1 启发式快速路径	54
5.3.2 异步 WGMMA 流水线优化	54
5.3.3 底层求解器 HeddleScheduler	57
5.3.4 求解器能力验证	60
5.4 多角色协同生成	62
5.4.1 双消费者模式的 WGMMA 流水线拆分	64
5.4.2 三角角色模式的阻塞写回剥离	65
5.5 与 Twill 的技术路线对比	66
5.6 本章小结	66
第 6 章 实验评估	69
6.1 实验设置	69
6.1.1 硬件与软件环境	69
6.1.2 测量方法与评价指标	69
6.2 性能对比	71
6.2.1 通用矩阵乘法	71
6.2.2 分组矩阵乘法	72
6.2.3 隐式卷积	73
6.2.4 低比特反量化矩阵乘	73
6.2.5 FlashAttention	74
6.2.6 注意力变体通用性与优化步骤分解	77
6.2.7 模型级推理性能验证	78
6.3 消融实验与机制验证	80
6.3.1 全栈功能消融	81
6.3.2 AutoPiped 的流水线级数验证	82
6.3.3 AutoMixed 的资源感知决策验证	82
6.3.4 联合调度的配置通过率、求解效率与硬件指标	84
6.3.5 FlashAttention 反向的多角色优化	87
6.3.6 双消费者调度消融与代价模型验证	87
6.4 本章小结	89

第 7 章 总结与展望	91
7.1 工作总结	91
7.2 未来工作	92
参考文献	93
致谢	97
作者简历及攻读学位期间发表的学术论文与其他相关学术成果 ..	99

图目录

图 1-1 FlashAttention 在 H100 上的性能与硬件利用率对比	4
图 1-2 研究动机、研究方案与研究目标总览	7
图 2-1 NVIDIA Tensor Core 在 Volta、Ampere 和 Hopper 架构下的变化 ..	11
图 2-2 GPU 上典型的重叠与流水线技术	12
图 2-3 Ampere 显式同步模型与 Hopper 异步流水线模型的对比	13
图 2-4 FlashAttention-3 中利用 WS 实现的异步流水线	17
图 3-1 算子在“生产力-性能”维度的帕累托前沿	25
图 3-2 AutoMixed 技术路线总览	26
图 3-3 FlashAttention 前向计算图	27
图 3-4 FlashAttention 前向中 WGMMMA 节点的操作数布局扩展示例	28
图 3-5 通过 DFS 枚举连通子图生成的候选分区方案	28
图 3-6 分区候选空间的寄存器与共享内存资源剪枝分析	30
图 3-7 FlashAttention 反向传播的三种图级分区策略	32
图 4-1 AutoPiped 技术路线总览	38
图 4-2 AutoPiped 依赖图构建示例	42
图 4-3 基于 min_T 的消费者顺序重排示例	43
图 4-4 FlashAttention 前向主循环的伪代码与硬件映射	45
图 4-5 FlashAttention 前向三类流水线配置生命周期对比	47
图 5-1 Heddle 模块总览	49
图 5-2 全局屏障与缓冲区级屏障的时序对比	52
图 5-3 异步 WGMMMA 调度的时序对比	56
图 5-4 HeddleScheduler 约束建模示例	58
图 5-5 FlashAttention 前向中的快速路径调度示例	61
图 5-6 FlashAttention 反向 dK/dV 中的约束路径调度示例	62
图 5-7 双消费者乒乓协议时序	65
图 6-1 FlashAttention 前向性能对比	75
图 6-2 FlashAttention 反向性能对比	76
图 6-3 LLaMA-2 7B 模型级推理加速比	79
图 6-4 双消费者性能与代价模型决策验证	88

表目录

表 1-1 FlashAttention 前向两类操作数布局的寄存器压力实验	3
--	---

表 1-2 FlashAttention 前向 SWP 与 WS 调度策略消融	5
表 2-1 TileLang 流水线配置参数	22
表 3-1 FlashAttention 反向各分区策略的资源占用分析	32
表 3-2 不同寄存器预算下的分区选择	33
表 3-3 Llama2-7B 逐元素融合的带宽代价分析	35
表 4-1 AutoPiped 的角色判定规则	39
表 4-2 AutoPiped 候选分组模板	39
表 4-3 稳态主循环执行流对比	46
表 5-1 原有 WS 与 FGWS 的编译通过率对比	50
表 5-2 FGWS 屏障编号空间的逻辑分段	52
表 5-3 典型算子的消费者语句数量统计	53
表 5-4 HeddleScheduler 生成的调度配置与编译决策输出示例	61
表 5-5 CUTLASS 乒乓调度与本文方案的设计维度对比	63
表 5-6 双消费者乒乓协议的命名屏障	65
表 6-1 实验主机硬件配置	69
表 6-2 软件栈与对比基准	70
表 6-3 主流模型代表性算子形状	71
表 6-4 算子级性能总结	71
表 6-5 GEMM 端到端性能对比	72
表 6-6 Grouped GEMM 不同子矩阵规模下的性能对比	72
表 6-7 Grouped GEMM 在不同 K 与 N 维度和分组数量下的性能	73
表 6-8 Conv2D 端到端性能对比	73
表 6-9 W4A16 Dequant GEMM 端到端性能对比	74
表 6-10 注意力变体端到端性能与库支持情况	77
表 6-11 注意力变体逐步优化加速比	78
表 6-12 AutoMixed 逐元素融合的带宽节省随序列长度的变化	79
表 6-13 LLaMA-2 7B 模型级推理延迟与加速比	79
表 6-14 Qwen-2.5 7B 模型级推理延迟	80
表 6-15 GEMM 全栈优化叠加式消融	81
表 6-16 FlashAttention 前向全栈机制移除式消融	81
表 6-17 不同模式随流水线级数的扩展性	82
表 6-18 两类操作数布局对比	83
表 6-19 线程数对寄存器溢出与性能的影响	83
表 6-20 FlashAttention 前向配置层叠消融	84
表 6-21 FlashAttention 前向不同调度策略的可行性与最佳延迟	85
表 6-22 约束求解时间统计	85

表 6-23 约束模型预测结果	85
表 6-24 单内核端到端编译耗时	86
表 6-25 NCU 硬件指标对比	87
表 6-26 FlashAttention 反向累进优化效果	87
表 6-27 双消费者逐步优化消融	88
表 6-28 双消费者代价模型验证	88

符号列表

调度与求解

符号

 II

的最短时间间距

 L T_v d_e δ_e $\text{warp}[v, w]$ $\text{live}[x, \tau]$ $\text{incoming_live}[x, \tau]$
代) T_v^{ASAP} T_v^{ALAP} slack_v

含义

启动间隔 (Initiation Interval), 相邻两次流水线迭代的

约束求解的时间窗口长度

操作 v 的启动时刻依赖边 e 的延迟 (周期数)依赖边 e 的迭代距离可选布尔变量: 操作 v 是否分配到 WarpGroup w 布尔变量: 输出值 x 在时刻 τ 是否存活 (本迭代)布尔变量: 循环携带值 x 在时刻 τ 是否存活 (跨迭操作 v 的尽早调度时间操作 v 的尽迟调度时间操作 v 的松弛量, $T_v^{\text{ALAP}} - T_v^{\text{ASAP}}$

资源与硬件

符号

 $\mathcal{G} = (V, E)$ $R_v[c, r]$ C_v $\text{cap}[r]$ RegLimit Cap_{SMEM}

含义

消费者依赖图, V 为操作节点, E 为依赖边操作 v 启动后第 c 个周期对资源 r 的占用操作 v 的延迟周期数 (资源预留表长度)功能单元 r 的每周期容量

单个 WarpGroup 的寄存器容量上限 (字节)

共享内存容量上限 (字节)

流水线与分块

符号

 S s_i B_i $\mathcal{M}$

含义

语句序列; 在代价模型中表示消费者语句集合

循环体中的第 i 条语句为语句 s_i 构造的临时计算块

全局缓冲区映射

$\mathcal{R}_i, \mathcal{W}_i$	语句 s_i 的读区域集合和写区域集合
S	流水线级数 (num_stages)
Δ	AutoPiped 阶段推断中的额外预取距离
$\text{ver}(s_i)$	语句 s_i 关联的共享缓冲区版本键
$v_{\max}$	当前分组下最大的缓冲区版本键
G_j	第 j 个语句组
π	AutoPiped 使用的分组模板或消费者排列
bM, bN, bK	分块形状参数 (tile 的 M、N、K 维度大小)
Δ_{BW}	逐元素融合预计减少的 GMEM 访问字节数
B_{mid}	逐元素融合前中间张量的字节数

执行机制代价模型

符号	含义
cfg	执行机制配置, 包含异步 WGMMA、提前释放、双消费者和 TRWS 的启用状态
T_{iter}	代价模型估计的稳态单次迭代时间
T_{producer}	生产者侧每迭代时间
T_{consumer}	消费者侧每迭代时间
T_{single}	单消费者模式的迭代时间
T_{dual}	双消费者模式的稳态迭代时间
$S_{\text{QK}}, S_{\text{PV}}$	双消费者模式下 QK 与 Softmax 侧和 PV 侧的消费者语句集合
$T_{\text{QK}}, T_{\text{PV}}$	QK 消费者、PV 消费者各自的语句延迟之和
t_i	消费者语句 i 的估计延迟
buf	生产者写入的共享缓冲区字节数
$t_{\text{bp_wait}}, t_{\text{signal}}$	反压等待和到达通知的估计开销
t_{barrier}	消费者侧屏障同步开销
t_{accum}	WGMMA 累加结果可被消费前的等待时间
$t_{\text{xfer}}^{\text{wr}}, t_{\text{xfer}}^{\text{rd}}$	跨 WarpGroup 中转的写入和读取开销
Δ_{early}	提前反压释放带来的生产者侧重叠窗口
Δ_{async}	异步 WGMMA 可隐藏的等待时间
Δ_{offload}	TRWS 将写回从消费者侧剥离后节省的延迟
$\mathcal{I}_{\text{gap}}$	相邻 WGMMA 之间的非 WGMMA 语句集合
BW_{TMA}	TMA 搬运带宽的标定值

缩写

缩写

全称

ALU	算术逻辑单元 (Arithmetic Logic Unit)
ASAP	尽早调度 (As Soon As Possible)
ALAP	尽迟调度 (As Late As Possible)
CP-SAT SAT Solver)	约束规划可满足性求解器 (Constraint Programming)
FA	FlashAttention
FGWS tion)	细粒度线程束特化 (Fine-Grained Warp Specializa-)
FU	功能单元 (Functional Unit)
GMEM	全局内存 (Global Memory)
IR	中间表示 (Intermediate Representation)
NCU	NVIDIA Nsight Compute 性能分析工具
PTX	并行线程执行汇编 (Parallel Thread Execution)
RMEM	寄存器内存 (Register Memory)
RS ory)	寄存器-共享内存操作数布局 (Register-Shared Mem-)
SASS	流式汇编 (Streaming Assembly), GPU 原生机器码
SDPA	缩放点积注意力 (Scaled Dot-Product Attention)
SFU	特殊功能单元 (Special Function Unit)
SIMT	单指令多线程 (Single Instruction, Multiple Threads)
SMEM	共享内存 (Shared Memory)
SS ory)	共享内存-共享内存操作数布局 (Shared-Shared Mem-)
SWP	软件流水线 (Software Pipelining)
Tile	数据块, TileLang 的核心编程抽象单元
TC	张量核心 (Tensor Core)
TIR	张量中间表示 (Tensor Intermediate Representation)
TMA	张量内存加速器 (Tensor Memory Accelerator)
TRWS	三角色线程束特化 (Three-Role Warp Specialization)
WGMMA Accumulate)	线程束组矩阵乘累加 (Warpgroup Matrix Multiply-)
WS	线程束特化 (Warp Specialization)

第 1 章 绪论

1.1 研究背景

GPU 算子的执行效率直接影响深度学习模型的训练与推理吞吐。随着模型规模从亿级参数增长至千亿级^[2]，单个算子的计算密度和访存模式对整体性能的影响进一步增大。矩阵乘法、卷积、注意力机制等算子通常占据端到端推理时间的主要部分，其吞吐能否接近硬件峰值，是系统性能的重要因素。以 Transformer^[3] 架构为例，其自注意力模块 (Self-Attention) 的中间张量大小与序列长度呈 $O(N^2)$ 关系，在长上下文场景下同时受到计算量和访存量的限制，是本文关注的典型优化对象。

随着摩尔定律的失效，为满足持续增长的算力需求，硬件加速单元正加速向异构化与异步化演进。NVIDIA 的 Hopper 架构引入了张量内存加速器 (Tensor Memory Accelerator, TMA)^[4] 和第四代张量核心 (Tensor Core)，理论算力大幅提升，但编程范式也随之发生了变化。与以往基于寄存器的同步计算模式不同，Hopper 架构采用全异步流水线的计算模式，通过 128 线程组成的线程束组 (Warpgroup) 协同发起线程束组矩阵乘累加 (Warpgroup Matrix Multiply-Accumulate, WGMMMA) 计算，并利用 TMA 异步将数据预取至共享内存 (Shared Memory, SMEM)。TMA 和 WGMMMA 之间的同步依靠硬件屏障 mbarrier 完成。生产者通过 mbarrier 通知消费者数据已就绪，消费者等待 mbarrier 后才开始计算。全异步流水线能够更好地利用硬件算力，但代价是开发者必须手动安排 mbarrier 的初始化、到达与等待时机，出错即导致死锁或数据竞争。

为了充分利用这些先进的硬件加速单元，硬件制造商通常会提供两种主要的编程模式。一种是类似于 cuBLAS^[5]、cuDNN^[6] 和 FlashAttention^[1,7,8] 等高性能专用算子库，提供粗粒度算子应用程序编程接口 (Application Programming Interface, API)，用户可以通过调用这些 API 快速且高效地使用硬件加速单元。这些库针对固定算子签名提供高度优化实现，但面向算子融合和自定义计算模式的扩展空间有限。另一种是在指令集架构 (Instruction Set Architecture, ISA) 级别的支持，开发者可以通过编写汇编指令或使用特殊汇编 API 调用这些张量加速硬件。这种方式要求开发者同时具备对硬件架构的深入理解与对汇编指令集的熟练掌握，工程成本极高。算子库本身的开发成本较高，且与硬件架构紧密耦合。例如，从 FlashAttention-2 到 FlashAttention-3 耗费了接近一年的工程投入，而从 FlashAttention-3 到 FlashAttention-4 的正式发版间隔约 18 个月¹。这种迭代周期限制了新硬件特性和新算子变体的快速适配。

由于用户自行编写的代码往往达不到厂商计算库的性能水平，近年来深度学习编译器迅速发展以弥合这一差距。深度学习编译器通过提供领域特定语言

¹FlashAttention-3 于 2024 年 7 月发布，FlashAttention-4 于 2026 年 1 月发布。

(Domain Specific Language, DSL) 前端与编译优化手段, 将模型自动转换为硬件加速单元的指令, 从而在保证推理性能的同时降低开发成本。早期的深度学习编译器如 Apache TVM^[9]、AutoTVM^[10] 和 Ansor^[11] 通过张量表达和启发式搜索, 在常规数据类型上效果良好。面向 Tensor Core 这类更低位数、更粗粒度的张量化硬件加速单元时, 编译器还需要更贴近指令形态和片上资源的调度表达。

更近的工作, 如 UNIT^[12]、AMOS^[13] 和 TensorIR^[14] 等, 填补了自动张量化代码生成的空白; Roller^[15]、Welder^[16] 和 FlashTensor^[17] 则通过分块图抽象与张量属性驱动的内核划分策略缓解了搜索空间膨胀与内存带宽压力。这些工作在同步执行模型下提升了硬件利用率。进入 Hopper 异步架构后, 高效算子还需要 WarpGroup 级的异步流水线排布。Triton^[18] 引入了自动线程束特化 (Automatic Warp Specialization) 支持, 其启发式分区算法主要面向生产者-消费者模式; Tawa^[19] 显式建模了跨 Warp 数据传递, 并以规则划分任务角色; PipeThreader^[20] 通过贪心策略分派就绪任务; Magellan^[21] 则尝试用进化算法搜索大量内核变体。上述路线体现了自动化异步调度的不同探索方向, 也暴露出复杂流水线中分组、阶段、顺序和资源约束需要协同处理的问题。

因此, Hopper 等异步硬件上的高性能算子需要同时具备手工优化库的细粒度调度能力和编译器的生成能力。本文关注的问题是如何在编译流程中通过循环体读写分析、资源模型和调度模型推断流水线配置、资源可行性和线程束分工, 从而缩小编译器生成内核与手工优化库之间的性能差距。

1.2 研究意义

尽管现有编译框架已提供底层数据块 (Tile) 抽象, 但在 Hopper 架构上构建高效算子仍面临三方面困难。

其一, 指令模式、分块形状与算子融合在硬件资源上深度耦合。Hopper 架构为 WGMMMA 指令提供了寄存器-共享内存布局 (简称 RS) 与共享内存-共享内存布局 (简称 SS) 两种源操作数布局, 二者在寄存器压力与共享内存占用上此消彼长, 且与流水线深度、分块大小和算子融合决策深度耦合。表 1-1 的压力测试表明, 前者在流水线深度 ≥ 4 时因寄存器超限而编译失败, 后者在大维度下因共享内存超限而失败。以 FlashAttention 反向为例, 强制融合 $dQ/dK/dV$ 到单一内核导致共享内存占用高达 104.5 KB, 而拆分后在序列长度 $T=4096$ 时获得 2.77 倍加速。

分块形状同样与上述决策深度耦合。以 FlashAttention 前向为例, bM 表示查询侧分块的序列位置数, bN 表示键值侧分块的序列位置数。将 bM 从 128 减至 64 后, 每个 WGMMMA 仅需 1 个 WarpGroup 即可完成, 省去了跨组同步, SM 占用率随之提高, 性能提升 1.62 倍; 进一步将 bN 从 64 增至 128, 单次矩阵乘的计算量翻倍, 摊薄了 Softmax 和屏障同步开销, 再加速 1.18 倍。

但更大的分块也会增加共享内存占用, 可能触发资源超限。因此, 两类操作数布局、分块形状、融合策略和流水线深度需要在同一资源约束下联合考虑; 只

调优单一维度通常无法覆盖最优配置。

表 1-1 FlashAttention 前向两类操作数布局的寄存器压力实验

Table 1-1 Register Pressure of Two Operand Layouts in FlashAttention Forward

流水线级数	操作数布局	寄存器数	溢出字节	延迟 (ms)	状态
2	RS	240	872	2.60	严重溢出，性能退化
2	SS	168	0	0.93	正常
3	RS	255	1456	3.23	溢出加剧
3	SS	180	0	1.12	正常
4	RS	—	—	—	编译失败
4	SS	192	0	1.35	正常

实验配置：FlashAttention 前向，头维度 $D=128$ ，批大小 $B=4$ ，头数 $H=32$ ，序列长度 $T=2048$ 。

其二，手动配置异步流水线的复杂性与有效配置的稀疏性。

如图 1-1a 和图 1-1b 所示，实验对多种 FlashAttention 实现进行了性能对比。该组实验的批大小为 $B=4$ ，头数为 $H=32$ ，头维度为 $D=128$ ，序列长度为 $T=4096$ 。前向传播中，FlashAttention-2 由于未使用 TMA、WGMMMA 等 Hopper 新特性，仅达到 331 TFLOPS；SASS 级手写的 FlashAttention-3 达到 608 TFLOPS，二者差距接近一倍，表明异步硬件单元的利用程度对性能影响显著。

编译器方面也呈现类似趋势。Triton 的自动调优获得 331 TFLOPS 量级，与 FlashAttention-2 接近；在本文测试配置上，它没有体现出 Hopper 异步硬件单元带来的额外收益。TileLang 借动手动线程束特化注解达到 544.0 TFLOPS，接近厂商闭源 cuDNN 的 570.6 TFLOPS²，但与 FlashAttention-3 仍有约 10% 的差距。由于 Heddle 与 TileLang 共享后端代码生成，这一差距提示调度策略本身仍有优化空间；同时，相关配置组合较多，完全依靠手动配置难以覆盖。

使用 TileLang 实现 Hopper 上的高性能内核需要同时决定语句分组、组间顺序与流水线阶段，并配合 mbarrier 同步与共享内存缓冲区管理。以 Attention 前向为例，循环体约 15 条语句，在固定官方 6 组分组后，仅对 4 个消费者组的组间顺序全排列 ($4! = 24$) 与流水线阶段分配 ($\text{num_stages} \in \{2, 3\}$, $2^4 + 3^4 = 97$) 进行穷举，共 2328 种配置，实验表明约 10% 能通过编译、数值校验和运行验证；其余配置主要因资源超限、非法同步或正确性问题被排除。

此外，较优配置并不总是符合默认经验。自动化搜索发现的最优配置延迟为 0.4325 ms，相比官方推荐配置的 0.5439 ms 加速 1.26 倍³。高效配置普遍采用更深的流水线阶段分配 $\text{Stage}=[-1, 1, \dots]$ ，不同于官方的 $\text{Stage}=[-1, 0, \dots]$ 模式。Stage 数组的每个元素对应一组语句的流水线阶段编号。-1 表示异步预取的生产者，非负整数表示消费者所处的流水线阶段。阶段编号越大，编译器为对应缓冲区分配的版本数越多，流水线越深。官方配置将消费者置于阶段 0，对应双缓冲的 2 级流水线；搜索发现的最优配置将消费者提升至阶段 1，对应三缓冲的 3 级流水

²该数据引用自 FlashAttention-3^[1]。

³实验配置：批大小 $B=4$ ，头数 $H=32$ ，头维度 $D=128$ ，序列长度 $T=4096$ 。

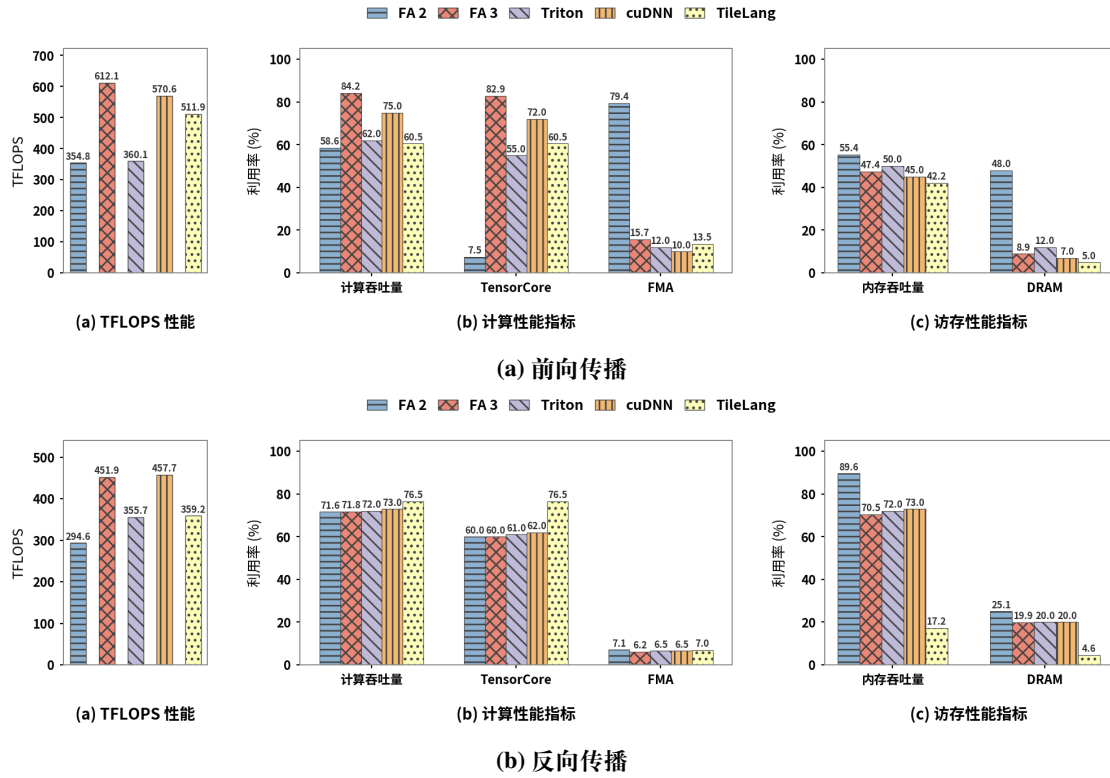


图 1-1 FlashAttention 在 H100 上的性能与硬件利用率对比

Figure 1-1 FlashAttention Performance and Hardware Utilization on H100

实验配置：批大小 $B=4$ ，头数 $H=32$ ，头维度 $D=128$ ，序列长度 $T=4096$ ，FP16。

线，以额外共享内存换取更深的跨迭代重叠。即使只在上述受限空间内穷举，也需要数小时。

其三，现有线程束特化基础设施与分离式调度的局限。在 Hopper 上实现高性能内核必须同时考虑时间维度的软件流水线（Software Pipelining, SWP）和空间维度的线程束特化（Warp Specialization, WS），但现有框架尚未完全解决这两类调度的组合问题。

在功能层面，现有 TileLang 实现的 WS 存在结构性缺陷。原有实现在所有生产者与第一个消费者之间设置全局同步边界，即使某个消费者仅依赖部分缓冲区的数，也必须等待全部生产者完成；这种过度同步不必要地延长了中间值的活跃周期，导致寄存器与共享内存的峰值占用被人为放大，大量配置因此编译失败。同时消费者顺序被固定为源码顺序，没有暴露任何优化自由度。

在调度层面，现有框架通常将软件流水线与线程束特化作为两个独立步骤处理。软件流水线为追求更小的启动间隔 II 会延长变量活跃周期，线程束特化又会把消费者操作集中到少数 WarpGroup 上，进一步增加局部寄存器压力，两者组合后容易触发溢出。如表 1-2 所示，即使在 FlashAttention 前向这一仅含 2 个 WGMMMA 的算子上，对 48 种分块与流水线配置组合进行实测，TileLang 手动线程束特化已有 6 个配置因寄存器溢出而编译失败，最低延迟为 15.0ms。若追求更深的流水线重叠，因溢出失败的配置进一步增加到 12 个，虽然通过编译配置

的最低延迟改善至 13.0 ms，但理论最优 12.5 ms 的配置仍因溢出无法通过编译。上述 48 种配置仅覆盖了固定分组下的阶段与线程参数组合；若进一步纳入语句分组与组间顺序的排列，实际搜索空间逾两千项，失效比例还会进一步上升。对于 FlashAttention 反向等含 4 个 WGMMMA 与原子归约操作的算子，单独的流水线或线程束特化启发式难以预判组合调度下的寄存器峰值。本文后续的联合调度模型正是围绕这一问题展开，在搜索阶段同时刻画时间维调度以及寄存器、共享内存等资源占用。

即使时间维调度达到最优，空间维的 WarpGroup 组织方式仍制约着性能上界。以 FlashAttention 前向为例，QK WGMMMA 与 PV WGMMMA 在单消费者 WarpGroup 上串行执行，Softmax 阶段张量核完全空闲；FlashAttention-3 通过手写的双消费者 WarpGroup，使第 k 次迭代的 QK 计算与第 $k-1$ 次迭代的 PV 计算交错执行，从而减少这段流水线气泡。这类双消费者交错执行目前主要出现在手写专用内核中，尚未成为开源编译器基线稳定支持的调度模式。类似地，FlashAttention 反向中的 `atomic_add(dQ)` 会阻塞消费者主循环。本文采用三角色线程束特化 (Three-Role Warp Specialization, TRWS)，将这部分写回从消费者路径中独立出来，使其与主计算并行执行。该设计把部分写回操作建模为独立角色，扩展了生产者-消费者二角色划分能够表达的并行结构。

这些空间维优化有明确收益，但涉及跨 WarpGroup 的数据传递与同步协议，手工实现和维护成本较高。

表 1-2 FlashAttention 前向 SWP 与 WS 调度策略消融

Table 1-2 FlashAttention Forward: SWP and WS Scheduling Strategy Ablation

策略	扫描配置数	溢出失败配置	最低延迟 (ms)
无流水线基线	16	0	25.0
TileLang 手动线程束特化	48	6	15.0
更深流水线调度	48	12	13.0

实验配置：FlashAttention 前向，批大小 $B=4$ ，头数 $H=32$ ，注意力头维度为 128，分块大小 128×128 。扫描配置包括分块大小、流水线级数取 1、2、3 以及线程数取 128 或 256 的组合；未通过编译的配置均由寄存器溢出导致。

上述三个问题分别来自资源组合、流水线配置和线程束分工。它们共同指向同一类核心需求：编译器需要在搜索阶段同时判断异步流水线内核在 Hopper 上的可编译性与执行效率。本文以此为出发点，在流水线构建、资源搜索、调度可行性判定和空间维分工等层面减少人工配置。

1.3 研究内容

针对上述三个问题，本文以 TileLang 作为后端代码生成框架，提出三个递进式编译模块——AutoMixed、AutoPiped 和 Heddle，在保留 TileLang Tile 抽象的基础上改造调度与代码生成流程。

主要研究内容与创新点如下：

1. AutoMixed：资源感知的图级优化策略（第3章）。针对指令模式、分块形状与算子融合在硬件资源上深度耦合的问题，AutoMixed 以 PyTorch 计算图为输入，在向 Tile IR 降级的过程中基于资源模型和编译反馈决策 WGMMA 操作数布局、算子融合边界和候选分块形状，在编译早期平衡寄存器压力、共享内存占用、GMEM 带宽收益与 SM 并行度。AutoMixed 的输出是结构确定的 Tile IR 内核，决定下游循环体中的 WGMMA 数目、缓冲区大小和可用并行结构。

2. AutoPiped：基于循环体分析的流水线配置参数生成（第4章）。在 AutoMixed 确定内核结构后，AutoPiped 分析循环体中的访存指令、共享内存读写关系和生产者-消费者依赖，生成分组、执行顺序和流水线阶段等异步流水线配置参数。如第 1.2 节所述，FlashAttention 前向在固定分组下已有逾两千种候选，若同时枚举分组则超过 10^6 。AutoPiped 的输出是 TileLang 后端可消费的 group、order 和 stage 注解，把循环级流水线配置从手工参数转化为编译期分析结果。

3. Heddle：面向细粒度线程束特化的自动流水线调度框架（第5章）。针对现有线程束特化机制中同步粒度粗、消费者调度空间有限、多 WarpGroup 协同依赖人工设计等问题，Heddle 在线程束特化执行层优化同步边界、消费者语句顺序和多角色协同。在执行模型层面，Heddle 提出细粒度线程束特化（Fine-Grained Warp Specialization, FGWS）机制，将传统 WS 中以流水线阶段为单位的全局同步细化为基于缓冲区版本的屏障协议，并将消费者侧语句顺序显式纳入调度空间。在调度算法层面，HeddleConsumerSchedule 基于消费者依赖图生成语句重排与屏障放置方案，并通过 CP-SAT 路径检查 Tensor Core 发射冲突、寄存器峰值和共享内存占用等硬件资源约束。在空间维优化层面，Heddle 识别 WGMMA 计算链和 TMA 写回路径中的可拆分结构，生成双消费者和 TRWS 配置。双消费者模式沿 QK-Softmax 链与 PV 计算链之间的边界拆分消费者并插入乒乓同步协议；TRWS 将 dQ 写回从消费者关键路径中剥离，降低阻塞写回对主计算流水线的影响。

图 1-2 概括了三个研究问题、对应编译模块和整体研究目标之间的关系。本文系统由 AutoMixed、AutoPiped 和 Heddle 三个模块组成，三者按照编译流水线的数据流顺序依次衔接。AutoMixed 在图级决定融合边界、WGMMA 操作数布局和候选分块形状，生成资源受限下的 Tile IR 内核；AutoPiped 在循环层根据语义依赖推断 group、order 和 stage 等流水线配置参数；Heddle 在指令层和 WarpGroup 层对流水线化循环体进行消费者重排、缓冲区级同步、异步 WGMMA 调度、多角色分工和资源约束检查。

1.4 全文组织

本文共分为七章，各章内容安排如下：

第 1 章为绪论，介绍研究背景、研究意义、研究内容与全文组织。

第 2 章为背景知识与相关工作，介绍 GPU 异构计算架构的演进、深度学习软件栈的发展，并梳理手写高性能算子库、高层优化框架与深度学习编译器等方面的代表性工作。

第 3 章介绍 AutoMixed，阐述资源感知的图级优化策略与决策机制。

第 4 章介绍 AutoPiped，阐述面向异步流水线的语义分析与配置推断方法。

第 5 章介绍 Heddle，阐述面向细粒度线程束特化的自动流水线调度框架。

第 6 章为实验评估，通过端到端性能对比、算法有效性验证和消融实验，评估各模块的有效性与系统整体性能。

第 7 章为总结与展望，概括本文的主要贡献，分析研究局限，并展望后续工作方向。

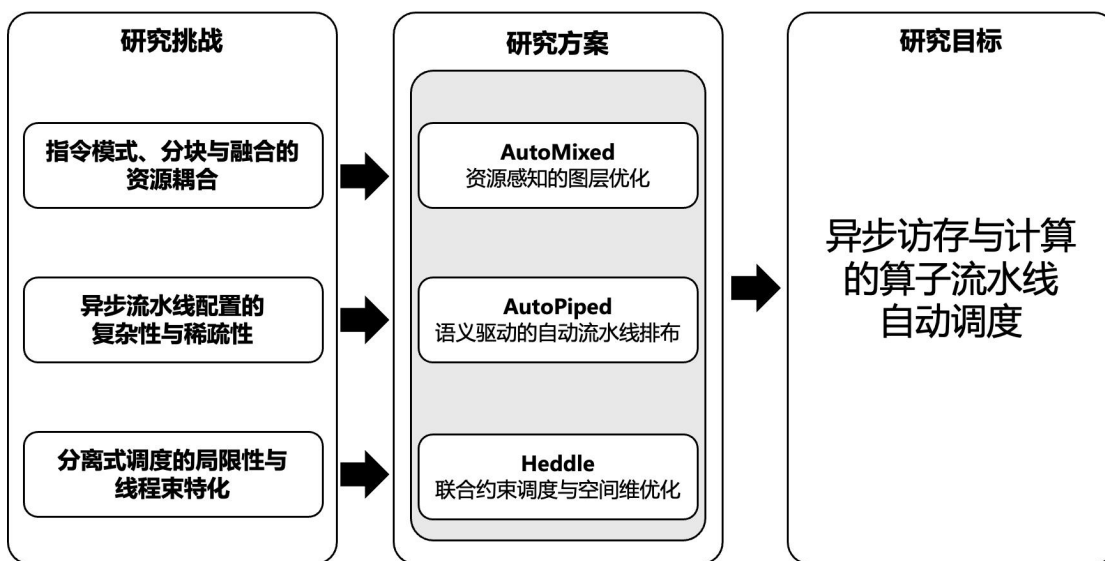


图 1-2 研究动机、研究方案与研究目标总览

Figure 1-2 Overview of Research Motivation, Approach, and Objective

第 2 章 背景知识与相关工作

GPU 的编程长期以单指令多线程 (Single Instruction Multiple Threads, SIMT) 模型为基础。SIMT 通过硬件调度器在每个流处理多处理器上同时维护大量活跃的线程束 (Warp) 实现对指令执行延迟的隐藏。当某个线程束由于访存操作或长延迟指令而发生阻塞时, 调度器能够在极低开销下快速切换到其他就绪线程束继续执行, 从而保持计算单元的高占用率与高吞吐。这种以吞吐量为导向、依赖硬件隐式调度来掩蔽延迟的设计理念, 在传统图形渲染及早期通用 GPU 计算场景中取得了良好效果。

然而, 随着大语言模型的兴起, 单纯依靠通用 SIMT 标量或向量算术逻辑单元和传统 Warp 调度来隐藏延迟, 已经难以在功耗与面积约束下持续提升性能。因此, 现代 GPU 逐步在片上集成粗粒度的异构专用加速单元, 例如面向矩阵运算的张量计算核心、特定算子优化单元以及面向压缩、解压和数据搬运的专用硬件模块等。这类专用单元以块、矩阵或算子为粒度执行运算, 相较于完全依赖通用 ALU 的 SIMT 执行路径, 能够在相同功耗和面积约束下提供更高的矩阵计算吞吐与数据搬运效率。

以 NVIDIA 为例, Hopper 架构引入了张量内存加速器 (Tensor Memory Accelerator, TMA) 和线程束组矩阵乘累加 (Warpgroup Matrix Multiply-Accumulate, WGMMA) 等专用异步硬件单元, 构建起更为复杂的片上计算与数据流管线。TMA 作为异步的硬件数据通路, 可以在不占用标量/向量计算核心的前提下完成从全局内存 (Global Memory, GMEM) 到共享内存 (Shared Memory, SMEM) 的数据搬运; WGMMA 允许以线程束组 (128 线程) 为单位, 异步地向张量核心流水线提交大粒度矩阵运算任务。

TMA、WGMMA 等专用硬件与指令协同设计, 拓展了 GPU 对张量计算和数据流组织的表达能力, 同时也提高了编程和调度难度。在现代架构中, 主导性能的关键操作已由单个线程或 Warp 级别的标量指令, 转向以数据块 (Tile) 为单位提交到专用单元的粗粒度任务。

一方面, 真正参与数值计算的逻辑集中在少量粗粒度张量操作上, 许多线程更多扮演控制和搬运角色, 可供硬件调度器调度以掩盖延迟的独立工作集实际上变少。

另一方面, 硬件停滞 (Stall) 的来源也从单纯的访存延迟, 转向跨异构单元的数据依赖与吞吐不匹配。例如, 在 WGMMA 指令执行期间, 如果加载指令未能及时将数据搬入寄存器或共享内存, 张量核心就会因数据未就绪而空转; 反之, 如果计算单元未能及时消费由 TMA 搬运来的数据, TMA 流水线也可能因缓冲区填满而停滞。硬件调度器只能根据局部就绪状态发射指令, 难以自动推理跨多个专用单元、跨多个循环迭代的全局数据流与时序关系。因此, 在 Hopper 等架构上, 开发者需要显式排布异步任务与同步屏障, 手动维护 TMA 搬运与

WGMMA 计算之间的依赖约束。这显著增加了内核开发和调试成本；一旦细粒度同步出错，问题往往难以复现，并会直接导致性能退化。

为了简化在新兴异步硬件上的编程负担，同时尽可能利用现有硬件潜力，深度学习编译器正在从依赖硬件隐式调度，转向由软件显式编排异步数据搬运、计算和同步。编译器需要承担起自动化分析和编排异步任务流的责任，以替代昂贵且易错的手工优化工作。本节将围绕这一趋势展开，系统梳理近年来 GPU 架构及编程模型的演进轨迹，并分析深度学习编译器在面向 NVIDIA Hopper 等现代异构架构时所面临的共性挑战。

2.1 GPU 架构及编程模型的演进

NVIDIA GPU 采用层次化存储架构，主要包括全局内存、L2 缓存和多个流处理多处理器 (Streaming Multiprocessor, SM)。全局内存基于 HBM (High Bandwidth Memory) 或 GDDR (Graphics Double Data Rate) 技术，是 GPU 的主要存储空间。每个 SM 包含多个 CUDA 核心、共享内存、寄存器文件、加载/存储单元以及线程束调度器等组件。

2017 年，Volta 架构引入第一代 Tensor Core，支持 $4 \times 4 \times 4$ 的 FP16 MMA 运算。每个 SM 包含 8 个 Tensor Core，采用 Warp-scoped 编程模式，由 8 个线程组成 Quad Pair 协作执行，支持 FP16 输入与 FP32 累加的混合精度计算^[22]。

2020 年，Ampere 架构中第三代 Tensor Core 新增对 TF32、BF16、INT8、INT4 等多种数据类型的支持，并引入 2:4 结构化稀疏计算支持。每个 SM 配备 4 个 Tensor Core。

2022 年，Hopper 架构推出第四代 Tensor Core，首次支持 FP8 精度；引入 warpgroup 概念，需要 4 个 warps 协同执行大规模 MMA 运算；同时配备 TMA 实现高效的多维张量异步搬运。其演变如图 2-1 所示。

随着 Tensor Core 等异构硬件单元的演进，GPU 的编程模型也从块同步并行 (Bulk-Synchronous Parallel, BSP) 模型转向异步流水线 (Asynchronous Pipeline)。Ampere 架构引入了 `cuda::memcpy_async`、`barrier`、`pipeline` 等异步编程接口，支持数据搬运与计算的重叠。Hopper 架构则通过 TMA 与线程束特化等技术，构建了更深层次的异步流水线架构。

在此之前，一种直接的重叠技术是在编译期或运行时显式设置不同波 (Wave) 或 Warp 的优先级，让调度器优先执行内存或计算操作，从而实现两者的并行。例如，AMD GPU 的 LLVM 编译器便支持通过 `intrinsic` 接口来调优波前 (Wavefront) 优先级，控制矩阵操作的调度与流水线化^[23]。

软件流水线 (Software Pipelining, SWP) 是另一种经典的调度技术，通过对顺序循环中的内存加载、计算等独立循环操作划分为若干流水线阶段，进行重新排序，利用指令级并行性 (Instruction-Level Parallelism, ILP) 来最大化功能单元的利用率。如图 2-2(b) 所示，典型的 SWP 对循环迭代进行展开，通过多级存储

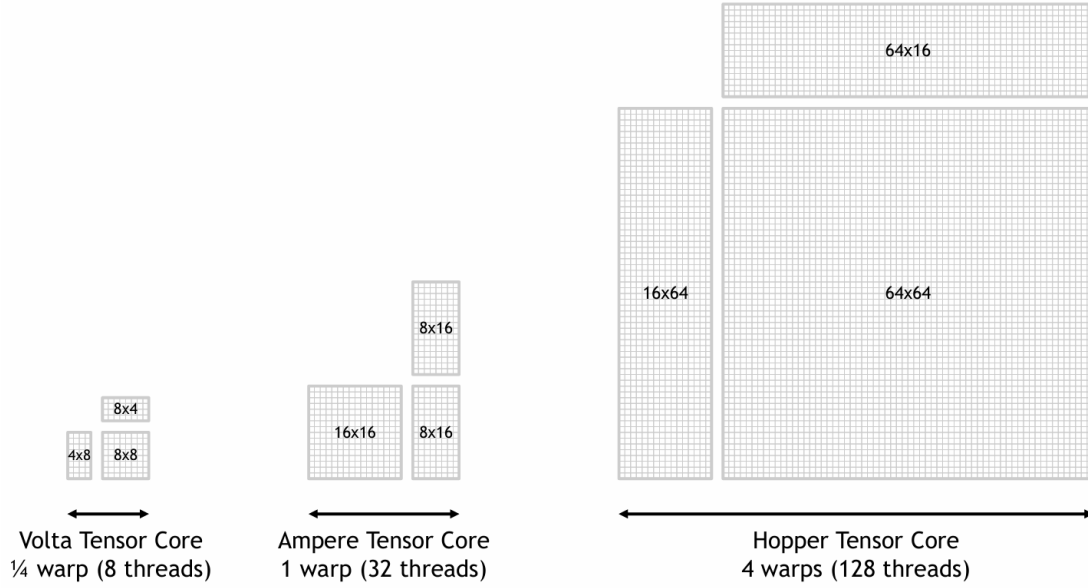


图 2-1 NVIDIA Tensor Core 在 Volta、Ampere 和 Hopper 架构下的变化

Figure 2-1 Evolution of NVIDIA Tensor Core across Volta, Ampere, and Hopper

缓冲区，将不同迭代的数据加载与计算操作交错重叠起来执行。SWP 能有效提升指令流水线的利用率，因而被广泛应用于高性能算子库及编译器的自动化算子生成中，但其代价是需要为每一级流水线额外分配寄存器等片上资源。

线程束特化 (Warp Specialization, WS)^[24,25] 是一项更进一步的 GPU 编程范式，打破了传统 SIMT 模型中所有线程束执行同构任务的限制。WS 基于“生产者-消费者”模型，将线程块内部的不同线程束绑定到不同的专用硬件单元，其中生产者线程束组专责数据预取与传输，消费者线程束组专注执行张量计算。如图 2-2(c) 所示，WS 区别于 SWP 的循环间重叠，它将不同类型的计算任务物理划分到不同线程束上，通过低开销的屏障同步（例如 `__syncwarp()`）协调任务流。每个 WarpGroup 拥有独立的指令流，依托多路线程束调度器 (Multi-way Warp Scheduler) 实现无中断连续执行，并通过共享内存完成 WarpGroup 间的高速数据交换。与所有线程束执行统一任务的传统方案相比，WS 能提升多任务调度效率和异构硬件资源利用率。

下面以 GEMM 为例展示，示例伪代码改编自 CUTLASS^[26] GEMM。

2.1.1 Ampere 架构下的显式同步批处理模型

算法 1 给出了 Ampere 架构上 GEMM 的典型调度方式。它以分块尺寸 (T_M, T_N, T_K) 将输出矩阵 C 划分为若干 $T_M \times T_N$ 的子块，并由每个线程块负责其中一块的计算。算法以 K 维分块为主循环，在共享内存与寄存器两级分别采用双缓冲，在计算当前分块的同时预取下一分块所需数据，并通过缓冲区交换将“搬运—计算”流水化。最终，寄存器累加结果写回得到 $C = A \times B$ ；由于跨线程的数据复用发生在共享内存阶段，算法需要显式同步以保证读写一致性。

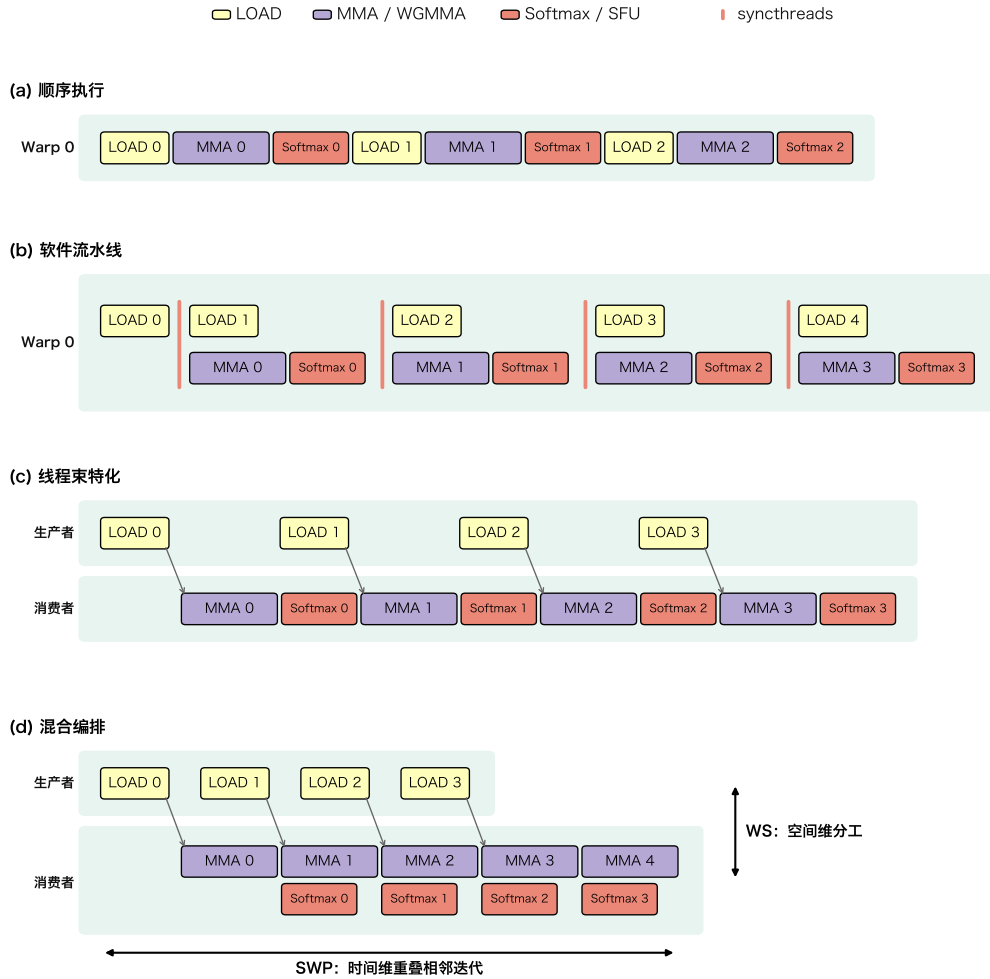


图 2-2 GPU 上典型的重叠与流水线技术

Figure 2-2 Typical Overlap and Pipelining Techniques on GPU

2.1.2 Hopper 架构下的异步编程模型

相比之下, NVIDIA Hopper 弱化了传统以 `syncthread()` 为核心的批处理同步方式。如算法 2 所示, Hopper 上的 GEMM 采用 WS, 将数据搬运与计算解耦, 由生产者 WarpGroup (DMA) 通过 TMA 按流水线深度 *PIPE* 将分块数据异步推送到共享内存, 而消费者 WarpGroup (MMA) 在数据就绪后以 `wgmma` 驱动 Tensor Core 完成计算, 并通过硬件屏障在生产者-消费者之间进行轻量同步。

图 2-3 以时间线对比了两种模型。Ampere 模型中所有 Warp 共用同构执行流, 借助双缓冲和 `cp.async` 可以让下一轮 LOAD 与当前轮 MMA 在稳态中重叠, 但阶段推进仍依赖线程块级显式同步。Hopper 模型进一步把 TMA 和 WGMMMA 放到两条独立的 Warp 流水线上, 生产者 Warp 用一条指令发起批量异步加载, 消费者 Warp 在 `mbarrier` 提示数据就绪后立即发起 WGMMMA, 两条流水线通过生产者-消费者关系持续重叠。mbarrier 按字节数追踪数据到达, 实现细粒度的生产者-消费者同步。

WS 通过轻量级的硬件屏障实现高效同步, 避免了传统锁机制带来的高昂开

算法 1 Ampere 风格 GEMM 调度伪代码**Algorithm 1** Ampere-Style GEMM Scheduling Pseudocode

输入：全局矩阵 gA, gB ；输出缓冲 gC ；分块尺寸 (T_M, T_N, T_K) ；流水线深度 $PIPE$

输出： $gC \leftarrow gA \times gB$

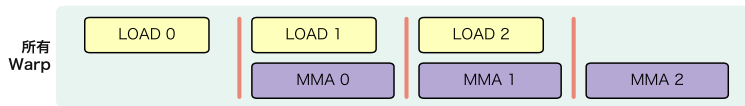
```

1: procedure AMPERESTYLEGEMM( $gC, gA, gB$ )
2:    $sA[T_M, T_K], sA\_next[T_M, T_K]$  ▷ 共享内存双缓冲
3:    $sB[T_K, T_N], sB\_next[T_K, T_N]$ 
4:   for  $k \leftarrow 0$  to  $gA.shape[1]/T_K - 1$  do ▷ 外层  $K$  维分块
5:      $copy(tile(gA, (blk\_x(), k+1)), sA\_next)$  ▷ 预取下一全局内存分块
6:      $copy(tile(gB, (k+1, blk\_y())), sB\_next)$ 
7:      $syncthreads()$ 
8:      $rA, rA\_next \leftarrow allocFragA(), allocFragA()$  ▷ 分配寄存器双缓冲
9:      $rB, rB\_next \leftarrow allocFragB(), allocFragB()$ 
10:     $copy(tile(sA, 0), rA)$  ▷ 加载当前  $A$  分块
11:     $copy(tile(sB, 0), rB)$  ▷ 加载当前  $B$  分块
12:    for  $kk \leftarrow 0$  to  $T_K/BLK_K - 1$  do ▷ 内层  $K$  维分片
13:       $copy(tile(sA, kk+1), rA\_next)$  ▷ 预取下一共享内存片段到寄存器
14:       $copy(tile(sB, kk+1), rB\_next)$ 
15:       $mma\_gemm(accum, rA, rB)$  ▷ 发起 Tensor Core 矩阵乘
16:       $swap(rA, rA\_next)$  ▷ 交换  $A$  寄存器缓冲
17:       $swap(rB, rB\_next)$  ▷ 交换  $B$  寄存器缓冲
18:    end for
19:     $swap(sA, sA\_next)$  ▷ 交换共享内存缓冲区
20:     $swap(sB, sB\_next)$ 
21:  end for
22:   $copy(accum, tile(gC, blk\_x(), blk\_y()))$  ▷ 写回累加器结果
23: end procedure

```

LOAD 同步拷贝 MMA/WGMMMA TMA 异步加载 mbarrier wait syncthreads

(a) Ampere: 显式同步的批处理模型



(b) Hopper: 异步流水线模型

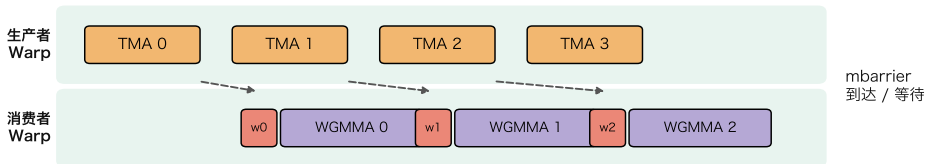


图 2-3 Ampere 显式同步模型与 Hopper 异步流水线模型的时间线对比

Figure 2-3 Ampere Synchronous Model vs Hopper Asynchronous Pipeline

销。同时，借助 WS，Hopper 可以把数据搬运和矩阵计算放到功能独立的硬件单元上执行，使流水线从传统双缓冲的 2 级结构扩展为由参数 PIPE 控制的深度异

算法 2 Hopper 风格 GEMM 调度伪代码**Algorithm 2** Hopper-Style GEMM Scheduling Pseudocode

输入：全局矩阵 gA, gB ；输出缓冲 gC ；分块尺寸 (T_M, T_N, T_K) ；流水线深度 $PIPE$

输出： $gC \leftarrow gA \times gB$

```

1: procedure HOPPERSTYLEGEMM( $gC, gA, gB$ )
2:    $sA[T_M, T_K, PIPE], sB[T_K, T_N, PIPE]$  ▷ 按流水线阶段展开的共享内存分块
3:    $sC[T_M, T_N]$  ▷ 输出暂存缓冲区
4:   barrier prod[PIPE], cons[PIPE], copyout ▷ 硬件屏障初始化
5:   if tid()  $\geq 128$  then ▷ 生产者 Warp
6:     for  $k \leftarrow 0$  to  $gA.shape[1]/T_K - 1$  do
7:       if  $k \geq PIPE$  then
8:         wait(cons[k%PIPE]) ▷ 等待消费者完成
9:       end if
10:      if tid() == 128 then ▷ 单线程发起 TMA
11:        TMA_load(prod[k%PIPE], tile(gA, (blk_x(), k)) -> sA[__, k%PIPE], tile(gB, (k, blk_y())) ->
        sB[__, k%PIPE])
12:      end if
13:      end for
14:      wait(copyout)
15:      if tid() == 128 then
16:        TMA_store(sC -> tile(gC, (blk_x(), blk_y())))
17:      end if
18:    else ▷ 消费者 Warp
19:      for  $k \leftarrow 0$  to  $gA.shape[1]/T_K - 1$  do
20:        wait(prod[k%PIPE]) ▷ 等待 TMA 完成
21:        warpgroup_sync() ▷ 128 线程协同启动 Tensor Core
22:        wgmma(accum, sA[__, k%PIPE], sB[__, k%PIPE])
23:        warpgroup_wait()
24:        arrive(cons[k%PIPE]) ▷ 通知生产者 Warp
25:      end for
26:      copy(accum, sC)
27:      syncthreads()
28:      arrive(copyout)
29:    end if
30: end procedure

```

步流水线。例如，当 Tensor Core 正在处理第 k 个循环迭代的数据时，TMA 已经开始异步预取第 $k + 1$ 个迭代所需的数据。这种计算与通信的重叠可以显著隐藏数据搬运延迟。此外，WS 将搬运和计算分配给不同 Warp，减少了不同类型指令在同一组 Warp 上交替执行时的资源争用，也让数据流更可预测。这样可以减少数据未就绪或资源冲突造成的流水线停顿，提高硬件实际吞吐效率。

然而，Hopper 等新架构引入的异步编程模型在带来性能潜力的同时，也提高了编译器在指令调度与依赖管理方面的难度。针对 wgmma 等张量指令，编译器需要显式插入 wgmma.fence、wgmma.commit_group 与 wgmma.wait_group 等同步指令，才能在深流水中正确维护数据依赖与完成状态；一旦同步点布局不当，就可能导致流水线利用率下降，严重时甚至引发死锁等问题。

2.2 算子优化的主要途径

GPU 算子优化的途径大致可以分为两条主线。一是由硬件厂商或领域专家针对特定算子手工编写高性能内核，形成专用算子库；二是通过深度学习编译器自动将高层算法描述转换为高效的底层代码。前者通常性能更高但开发维护成本高昂，后者追求通用性与自动化但在复杂异步架构上仍存在性能差距。本文的研究聚焦于 Transformer 模型中的核心算子优化，下面首先介绍 Transformer 的注意力机制作为本文算子优化的对象，然后分别沿这两条路径展开。

Transformer 架构^[3]已成为当代人工智能领域最具影响力的模型架构之一，其核心在于注意力机制（Attention Mechanism），通过动态计算输入序列中各个元素之间的相关性权重，使模型能够有效捕捉长程依赖与复杂上下文关系。

标准的缩放点积注意力（Scaled Dot-Product Attention, SDPA）可以写作

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2-1)$$

其中 Q（查询，Query）、K（键，Key）、V（值，Value）是输入序列经过线性变换得到的矩阵。参与点积的查询和键具有相同内维 d_k ，因此三者维度可写为 $N \times d_k$ 、 $N \times d_k$ 和 $N \times d_v$ ，其中 N 为序列长度。

与注意力机制强大的表达能力相伴的，是其复杂的计算依赖关系以及与输入序列长度 N 呈二次方关系的显存与算力开销。朴素的注意力实现需要显式计算并物化一个 $N \times N$ 的注意力得分矩阵 S 。为突破这一瓶颈，系统软件层大致形成了专用算子库与深度学习编译器两条主要技术路径。

专用算子库通过面向特定硬件平台开发高度优化、手工调优的计算内核（Kernel），为卷积、矩阵乘、注意力等关键算子提供接近硬件理论上限的性能。代表性工作包括 cuDNN^[6]、cuBLAS^[5]，以及在注意力算子方向上的 FlashAttention 系列^[1,7,8]等。

深度学习编译器则侧重通用性、自动化与可移植性，如 Apache TVM^[9]、Triton^[18] 以及同时具备图优化与内核生成能力的 TensorRT 等。它们的目标是将与硬件无关的高级模型描述，自动转换为可在多种硬件后端上高效执行的底层代码，通常依托多层次中间表示（Intermediate Representation, IR）以及一系列复杂的自动优化策略来生成高效且具有一定可移植性的实现。

这两条路径体现了性能专用化与开发可移植性之间的权衡。手写内核通过硬件耦合获取高性能，但每次架构升级都需要重写；编译器通过自动化屏蔽硬件差异，但在复杂异步架构上仍有性能差距。实际系统中二者互补，高性能算子库作为性能保障被框架调用，编译器负责图优化与调度。

2.2.1 手写高性能算子

高性能算子库通过针对特定平台手工实现关键算子的高效内核来提升性能，代表性工作包括通用矩阵乘法库 cuBLAS^[5]、深度学习原语库 cuDNN^[6]，以及

注意力算子领域的 FlashAttention 系列^[1,7,8]。由于注意力算子是本文优化的核心对象之一，下面以 FlashAttention 的演进为例，展示手写算子库如何通过硬件架构的持续深度协同实现性能的迭代提升，同时分析这种极致专用化所带来的高昂开发维护成本。

FlashAttention-1^[7] 指出当序列长度 N 很大而单头维度 d 相对较小时，注意力计算的瓶颈往往来自 $N \times N$ 中间矩阵在全局内存上的读写，直接实现需要将 S, P 物化到全局内存，带来巨大的内存 I/O。

为避免 S, P 的全量物化，FlashAttention-1 采用分块与在线 Softmax 递推，将 Q, K, V 沿序列维度切分为块 $\{Q_i\}, \{K_j\}, \{V_j\}$ ，块大小分别为 b_q, b_k, b_v （其中 $b_k = b_v$ ），并对每个 Q_i 沿 j 维逐块扫描 (K_j, V_j) 来累积输出 O_i ，该扫描循环在后文中称为 K 循环。对固定的 i 与第 j 个 (K_j, V_j) 块，有

$$S_i^{(j)} = \frac{Q_i K_j^\top}{\sqrt{d}}, \quad (2-2)$$

$$m_i^{(j)} = \max\{m_i^{(j-1)}, \text{rowmax}(S_i^{(j)})\}, \quad (2-3)$$

$$\tilde{P}_i^{(j)} = \exp(S_i^{(j)} - m_i^{(j)}), \quad (2-4)$$

$$\ell_i^{(j)} = \exp(m_i^{(j-1)} - m_i^{(j)}) \ell_i^{(j-1)} + \text{rowsum}(\tilde{P}_i^{(j)}), \quad (2-5)$$

$$O_i^{(j)} = \text{diag}(\exp(m_i^{(j-1)} - m_i^{(j)})) O_i^{(j-1)} + \tilde{P}_i^{(j)} V_j, \quad (2-6)$$

其中 $m_i^{(j)}$ 与 $\ell_i^{(j)}$ 为 $b_q \times 1$ 向量，初始化为 $m_i^{(0)} = -\infty$ 、 $\ell_i^{(0)} = 0$ ， $O_i^{(0)} = 0$ 。最终输出通过归一化得到

$$O_i = \text{diag}(\ell_i^{(J)})^{-1} O_i^{(J)}. \quad (2-7)$$

上述递推仅需维护行最大值与行求和等向量级状态，从而在计算 O 的同时避免将 S 与 P 写回全局内存，将放存量从 $O(N^2)$ 降低至 $O(Nd)$ ，减少了全局内存与共享内存之间的数据传输，在 BERT-large 和 GPT-2 等模型训练中实现了 1.15 到 3 倍的加速。

FlashAttention-1 主要在批次大小 (batch size) 和头数维度上并行，每个注意力头分配一个线程块。当序列很长而批次大小和头数较少时，这种策略会导致 GPU 上的 SM 利用率偏低。FlashAttention-2^[8] 针对硬件特性进行优化，通过序列长度维度上的额外并行、算子融合、Warp 级规约 (Reduce) 和洗牌 (Shuffle) 机制降低 Shared Memory 通信开销，并重构计算流以适配 Tensor Core 执行模式，在 NVIDIA A100 平台达到 70% 的峰值计算利用率，较 FlashAttention-1^[7] 提升 40%–75%。

但当 FlashAttention-2 被直接移植到 NVIDIA H100 GPU (Hopper 架构) 上时，其硬件利用率再次跌至 35% 左右。这反映出 Hopper 的 TMA 数据搬运通路和 WGMMMA 张量指令改变了注意力算子的执行模型，软件需要重新组织计算流才能利用异步硬件能力。FlashAttention-3^[1] 将注意力计算重构为生产者-消费者异步流水线。通过 WS，一部分 warps 作为生产者，使用 TMA 从全局内存异步预

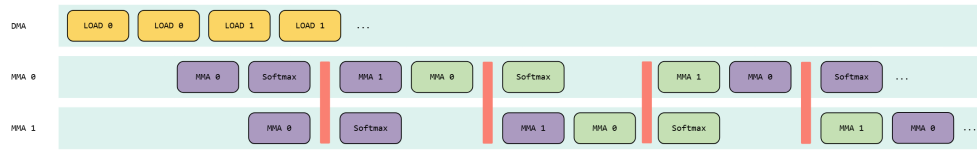


图 2-4 FlashAttention-3 中利用 WS 实现的异步流水线

Figure 2-4 Asynchronous Pipeline with WS in FlashAttention-3

取数据到共享内存；另一部分 warps 作为消费者，使用 WGMMMA 在 Tensor Core 上对已就绪的数据执行矩阵乘累加。该模式在时间维度上重叠数据搬运与计算，从而隐藏访存延迟。如图 2-4 所示，通过乒乓调度（ping-pong scheduling），一个 WarpGroup 在执行矩阵乘计算的同时，另一个 WarpGroup 并行执行 Softmax 等后续操作，从而减少流水线停顿。同色区域对应同一次循环迭代。

在此基础上，流水线还可以进一步细化，比如通过在 WarpGroup 内对不同指令流的穿插调度，可以在同一循环迭代中将这些慢速 CUDA Core 操作隐藏在异步 Tensor Core 操作的执行周期内，充分利用异构硬件的算力。此外，FlashAttention-3 还进一步适配了 Hopper 架构引入的 FP8 数据格式，在理论上使得 GEMM 吞吐量相较 FP16 再提升一倍。

FlashAttention 的演进表明，高性能注意力实现越来越依赖对特定硬件并行模型的专用适配，从 FlashAttention-2 到 FlashAttention-3 耗费了接近一年的工程投入。手写内核性能高但维护成本高，因此需要在自动化编译与硬件特化之间寻找折中。

2.2.2 高层优化框架与系统

FlashAttention 等手工内核的开发与维护成本高昂，且与特定硬件紧密耦合。为了缓解这一问题，出现了各类高层抽象框架。这些框架通常通过针对性的抽象，聚焦注意力优化中的少数关键问题，如内存管理、内核定制化或跨平台支持等。本节介绍的框架与系统主要用于阐明领域背景和设计思路，其中 FlexAttention 的 torch.compile 路径和 PyTorch SDPA 后端在第 6 章的端到端实验中作为对比基线出现。

FlexAttention^[27] 是 PyTorch 2.5.0 新增的 API，使研究者仅需少量 Python 代码即可实现多种复杂的注意力变体，如滑动窗口注意力（Sliding Window Attention）等。FlexAttention 结合了 torch.compile 及其 Triton 后端。用户只需提供 score_mod 的实现，即可在计算过程中动态修改注意力得分的计算方式。torch.compile 会捕捉该 Python 函数的计算图，并生成优化的 Triton 核函数，将这些修改操作直接融合到注意力计算的核心循环中，从而避免物化中间张量和额外内存读写。在此基础上，FlexAttention 能达到 FlashAttention-2 约 85% ~ 90% 的性能，降低了探索与部署新型注意力变体的门槛。

$$FlexAttention(Q, K, V) = \text{softmax}(\text{score_mod}(\frac{QK^T}{\sqrt{d_k}}))V \quad (2-8)$$

与侧重 PyTorch 生态和 NVIDIA 硬件的 FlexAttention 不同, AttentionEngine^[28] 的设计目标是为异构硬件提供统一的注意力优化框架。AttentionEngine 将注意力计算抽象为两个基础的、模块化的操作, 相关性评分 (计算 $Q \times K^T$) 和聚合 ($\times V$)。通过 DeviceConfig 等组件框架封装了分块尺寸、存储层次结构等硬件细节, 使上层逻辑保持相对平台无关。系统通过统一的降级 (Lowering) 机制, 将用户定义的计算图转换为硬件特定的内核模板, 支持 TileLang、CUTE、Triton 等多种后端以及 NVIDIA、AMD 等不同 GPU。AttentionEngine 不仅支持标准 Transformer 注意力, 还支持多种线性注意力变体, 统一的编程接口与自动化内核生成显著降低了跨平台迁移成本。

在推理场景中, KV 缓存的内存管理和调度同样影响注意力算子的端到端效率。PagedAttention^[29] 以分页方式管理 KV 缓存以消除内存碎片, vAttention^[30] 进一步利用 CUDA 虚拟内存 API 将分页逻辑从内核中解耦, FlashInfer^[31] 提供面向块稀疏 KV 缓存的预编译内核与 JIT 编译接口, KTransformers^[32] 通过运行时模块替换将优化内核注入 Hugging Face 模型。这些工作侧重推理系统层的内存管理与调度策略, 与本文关注的片上异步流水线编译优化正交。

2.2.3 深度学习编译器

与提供特定抽象的框架不同, 深度学习编译器的重要目标是提高从高级计算图到高性能底层代码的自动化程度, 即将以 PyTorch 等高级语言描述的计算图 (Computation Graph), 通过多层级 IR 和一系列针对硬件特性的优化, 转换成能在目标硬件上高效运行的底层代码。本节讨论编译器的优化策略, 并以 Triton^[18] 和 TileLang^[20] 为代表分析面向异步硬件的编译器现状与局限, 并梳理模调度、Tawa、Magellan、Twil 等近期在异步流水线调度方向上的代表性工作。

2.2.3.1 基于 DAG 的编译器

基于有向无环图 (Directed Acyclic Graph, DAG) 的中间表示是深度学习编译器最传统的前端形式之一, 图的节点表示算子, 例如卷积、归一化、矩阵乘等, 边表示张量数据流。编译器可在图上进行如常量折叠 (Constant Folding)、公共子表达式消除 (Common Subexpression Elimination, CSE)、布局优化 (Layout Optimization)、算子融合 (Kernel Fusion) 等各类图优化方法, 并进一步降级到后端进行代码生成。

TVM^[9] 是典型的端到端 DAG 编译器, 其内部使用张量表达 (Tensor Expression) 描述算子计算, 并通过一系列低级循环变换, 如循环分裂 (Split)、循环重排 (Reorder)、分块 (Tile)、向量化 (Vectorize)、循环展开 (Unroll) 等完成性能调优。

AutoTVM 与 Ansor^[11] 等工作将搜索/自动调优引入调度空间。AutoTVM 依赖模板定义搜索空间，而 Ansor^[11] 尝试自动构造更全面的候选并进行自动调优以获得较优实现。这类方法通常面临两个问题。其一，调度搜索空间容易膨胀到全局寻优成本很高的规模；其二，当张量表达迅速 lowering 为循环之后，高层语义基本丢失，进一步限制了对复杂依赖结构的分析与重写能力。

为增强硬件感知能力，部分系统尝试引入具备更多专家经验的算子与模板库辅助编译优化。例如 BOLT^[33] 通过使用 CUTLASS 等手写模板实现融合矩阵乘与卷积，在共享内存与寄存器层进行数据复用，减少算子间读写与内核启动开销，并扩大优化范围以探索更优指令调度。

2.2.3.2 拓展算子融合的编译器

在“内存墙”问题日益突出的当下，算子融合已成为图级优化的核心。它通过将多个独立的算子合并为一个融合算子，减少了中间结果在全局内存中的读写次数，并降低了 GPU 上频繁的内核启动（Kernel Launch）开销。

根据 Roofline 模型对算子计算强度（Arithmetic Intensity）的建模，可以把算子分为计算密集型和访存密集型算子。最常见的融合模式是利用生产者-消费者关系，将计算密集型算子（如 CONV、GEMM）与其后续的访存密集型算子（如 ReLU、BatchNorm）进行垂直融合（Vertical Fusion）^[9,34-38]，利用 GPU 的片上存储（共享内存或寄存器）直接传递数据，避免了昂贵的全局内存的中转。DNNFusion^[36] 系统地梳理了算子输入和输出张量之间的元素映射关系，将算子分为一对一（One-to-One）、一对多（One-to-Many）、多对多（Many-to-Many）、重组（Reorganize）和洗牌（Shuffle）五种类型，并定义了不同类型算子之间的组合规则。

AStitch^[39] 等工作则专注于不规则形状算子以及多个访存密集型算子之间的融合，将计算密集型算子之间的多个访存密集型算子按照预先定义的规则合并成一个较大的内核，进一步减少了内核启动和内存访问的开销。

多个计算密集型算子之间的融合会显著增加寄存器与共享内存占用，因此相关研究相对较少。Chimera^[40] 和 MCFuser^[41] 等工作开始探索特定负载下多计算密集型算子融合带来的调优空间和性能收益；基于多面体模型的方法^[42] 则提供了更细粒度的表达，但搜索空间也随之快速扩大。

为缓解细粒度调度的搜索复杂性，研究者们提出基于分块图（Tile-Graph）的抽象。Roller^[15] 倾向于通过构造性方法直接生成满足硬件约束（如片上存储容量、向量宽度、并行粒度）的合法 Tile 配置，以规避在巨大搜索空间中盲目搜索的问题。Welder^[16] 更关注跨算子的存储层次映射，通过统一图抽象管理中间张量在内存层次中的流动，以提升片上复用效率，实现了对算子间中间张量在存储层次结构中更细粒度的调度。Tile-Graph 类抽象主要作用于算子边界和跨算子数据流，对算子内部复杂依赖结构的表达粒度相对有限。

FlashTensor^[17] 是针对长序列 Transformer 推理提出的一种全新编译优化系

统，主要通过系统性地标注每条中间张量在每个维度上的 `reduce` 依赖等张量属性，构建张量属性驱动的非凸核映射机制，并基于此设计代数等价变换和轻量级搜索策略，减少中间张量尺寸与内存带宽压力。

2.2.3.3 面向异构硬件的编译器

前文介绍的软件流水线在不同硬件上有不同实现方式。Ampere 架构引入 `cp.async` 指令，支持全局内存到共享内存的异步拷贝，使同一线程束内不同迭代的计算与访存可以重叠。这种方式下线程块内所有线程束执行相同的指令流，通过多级缓冲实现迭代间重叠，无需跨线程束同步。Hopper 架构在三个方面改变了这一格局。TMA 将数据搬运从逐线程操作升级为单线程发射的批量传输，减少了地址计算的寄存器开销。`setmaxnreg` 指令允许同一线程块内不同线程束分配不同数量的寄存器，使生产者线程束仅保留少量寄存器，将更多寄存器留给计算密集的消费者。`mbarrier` 以精确字节数追踪数据到达，适合生产者与消费者之间的解耦同步。在 `mbarrier` 协议中，生产者在发起 TMA 或 `cp.async` 拷贝前通过 `expect_tx` 向 `mbarrier` 预告本次传输的字节数，硬件在传输完成后自动执行 `arrive` 更新到达计数；消费者通过 `try_wait` 轮询 `mbarrier`，直到所有预告字节到达后才开始读取共享内存。三者叠加使 WS 在 Hopper 上获得实质性的寄存器收益，并在 Hopper 高性能算子实现中成为常见的软件流水线组织方式。Ampere 时期 `mbarrier` 已经存在，但缺少 `setmaxnreg` 的支持，WS 的生产者和消费者线程束被分配相同数量的寄存器，寄存器收益无法兑现，同构方式在当时更为实用。

这种从同构指令流到异构多角色协同的转变使传统的编译器调度理论面临新的挑战。

经典的模调度 (Modulo Scheduling)^[43–45] 将循环体建模为依赖图 $G = (V, E)$ ，其中每条指令 $v \in V$ 附带一张资源预留表 (Resource Reservation Table, RRT)，描述其在每个时钟周期占用的功能单元；算法在满足数据依赖、并避免同类功能单元同时被多条指令占用的前提下，寻找最小启动间隔 I 和调度映射 M ，使流水线吞吐最大化。这一框架在传统 GPU 上效果不错。将其应用于 FlashAttention 的分块级依赖图，可以描述 FlashAttention-3 等手写实现中的迭代重叠关系，为自动搜索类似调度提供建模基础。

将模调度应用于 Hopper 架构时面临以下新挑战。其一，TMA 在全局内存与 SMEM 之间搬运数据的延迟受访存模式影响波动较大^[4]，而模调度对延迟取上界的假设在此失效，过估导致资源利用率低，欠估则引发运行时停顿。其二，WS 将单一指令流拆分为生产者和消费者两个并发流，二者通过 FIFO 队列耦合但执行速率不同^[1,19]，模调度需要从单线程视角扩展到耦合双循环调度。其三，WGMMMA 等张量指令需要多个线程束协作发起^[46]，而大多数 GPU 编译器基于顺序中间表示（如 LLVM/PTX），协作线程束代码生成需要额外的结构化表达。其四，固定功能单元的阻塞同步（如 `wgmma.wait`）会中断同一线程束上并发操作的指令发射，导致独立功能单元之间产生伪依赖^[47]。

寄存器分配同样面临瓶颈。模调度通过增大流水线深度来掩盖 TMA 数百周期的延迟，但这要求同时保持更多活跃的数据缓冲区和控制变量，加剧了寄存器压力。NVIDIA GPU（从 Fermi 到 Hopper 架构）对每个线程的寄存器上限均为 255 个 32 位寄存器^[22]，而溢出到片外的代价很高，溢出数据通常难以保留在缓存中，性能损失显著。基于压缩（Compression-based）的寄存器分配算法试图缩减变量的活跃区间（Live Range），但在 Hopper 上压缩过度会导致 TMA 流水线断流，使 Tensor Core 因数据未就绪而饥饿。

上述挑战可以归结为线程束特化之所以有价值的三个条件。第一是资源溢出，累加器等线程级状态接近每线程 255 寄存器的硬件上限时，特化通过为生产者和消费者分配不同寄存器预算，成为缓解溢出的有效手段之一。第二是延迟不确定性，前述 TMA 访存的延迟波动使静态调度失效，特化后的多线程束由硬件调度器动态交织，无需猜测每条指令的精确延迟。第三是阻塞同步干扰，前述 `wgmma.wait` 等等待指令会挂起整个线程束的发射流水线，编译器难以将后续独立指令安全地重排到等待之前，特化使其他线程束在挂起期间继续执行。后两个条件本质相关，线程束特化将顺序发射的 SM 转变为准乱序处理器，特化边界标记了静态难以利用的指令级并行。不同算子对这三个条件的敏感度不同，GEMM 的循环体结构规则，通过精心的循环剥离和双重流水线化可以在不特化的条件下达到接近 cuBLAS 的性能，但 FlashAttention 的循环体包含 softmax 归约与指数计算等大量非加速器工作，这些操作与异步 WGMMMA 和 TMA 密集交错，静态调度难以覆盖全部功能单元空闲周期，是线程束特化更有优势的典型场景。

Triton^[18] 将编写高性能并行内核的复杂性封装在数十行 Python 代码中，显著降低了开发门槛。开发者只需通过 `@triton.jit` 装饰器定义核函数，Triton 编译器即可自动完成针对异构 GPU 后端的循环分块、向量化、内存布局优化和指令选择。其 3.x 版本引入了对 Hopper 架构 WS 的支持，旨在通过预设的启发式规则，自动完成任务分区、流水线构建与同步原语插入，从而对用户隐藏异步编程的复杂性。在 FlashAttention 等复杂算子上，性能还取决于跨 WarpGroup 的任务切分、流水线阶段划分与同步点布局等细粒度调度决策；这些决策在 Triton 中主要由编译器内部启发式处理，用户只能通过 `num_warps`、`num_stages` 等高层次参数间接影响。对于 FlashAttention-3 所采用的多级复杂流水线等模式，现有规则仍有扩展空间。

为提供更可控的优化空间，Tawa^[19] 在 Triton 的基础上引入了 `aref` 这一中间表示，将跨 Warp 的数据传递显式建模为具有空/满状态屏障的有界环形缓冲。其中，生产者通过 `put(slot)` 将数据投递到指定缓冲位置，消费者通过 `get(slot)` 获取数据，并在使用完成后以 `consumed(slot)` 归还该缓冲位置，从而显式刻画生产、消费、归还的完整生命周期。借助 `aref`，数据依赖边被明确暴露给编译器与用户，使得缓冲深度、归还时机以及生产者/消费者并发度等关键参数都可以被精确控制，并在编译期进行一致性检查与验证，从而为复杂流水线提供更确定的表达能

力与更可控的优化空间。Tawa 通过识别“计算/内存密集型”子图来基于规则划分 Warp 角色，并利用模板快速生成代码；其性能由模板覆盖范围和自动调优参数共同决定。面向 FlashAttention-3 式乒乓调度等高级模式时，仍需要进一步扩展规则和模板。Tawa 面向 Hopper 的 WS 支持目前主要以独立分支形式发布，尚未进入 Triton 主线。

Magellan^[21] 等框架尝试利用进化算法在大量内核变体上进行搜索，但其搜索空间导致编译成本极高。

TileLang^[20,48] 代表了更接近底层异步硬件的编程与调度路线。它以 Tile 为核心抽象，向开发者暴露 TMA、WGMMA 和 mbarrier 等 Hopper 原语，并将 Tile 级操作组织为特化任务图（sTask-graph），其中任务节点映射到专用执行单元（Specialized Execution Unit, sEU），再由调度器安排就绪任务。该路线提供了比高层 DSL 更直接的调度控制，但流水线构建仍依赖用户为 T.Pipelined 循环指定 group、order 和 stage 三类配置参数，如表 2-1 所示。复杂算子中，这些参数与依赖关系、缓冲区版本和同步位置紧密耦合，仍需要进一步自动推断。

表 2-1 TileLang 流水线配置参数

Table 2-1 TileLang Pipeline Configuration Parameters

参数	含义
group	将循环体中的语句划分为若干组，每组内的语句作为一个整体参与调度。例如 [[0], [1, 2], [3..11], ...] 表示语句 0 单独一组，语句 1 和 2 合并为一组。
order	每组的发射顺序编号。-1 为生产者，在循环体开始前异步预取；0, 1, 2, ... 为消费者组，按此顺序依次执行。
stage	每组所处的流水线阶段编号。-1 为生产者阶段；非负整数为消费者阶段，编号越大缓冲区版本越多、流水线越深。

以 FlashAttention 前向为例，循环体约 15 条语句，固定分组后仅执行顺序与流水线阶段的组合已逾两千项，其中仅约 10% 能成功编译运行（详见第 1.2 节）。这三类参数的手动配置成本极高，是本文第 4 章 AutoPiped 模块要解决的核心问题。

如图 2-2(d) 所示，SWP 与 WS 构成两个正交的优化维度：SWP 沿时间维重叠相邻迭代的搬运与计算，WS 沿空间维将 WarpGroup 划分为生产者与消费者角色，二者通过 TMA 搬运、WGMMA 计算和 mbarrier 同步三种异步原语协同工作。上述工作分别从不同角度优化了 SWP 或 WS，但二者协同仍需要更系统的编译分析。在 Ampere 上，同构的 SWP 通过异步拷贝在迭代间重叠计算与访存，GEMM 等结构规则的算子无需 WS 即可达到接近峰值的性能。在 Hopper 上，WS 将不同功能分配到不同线程束并行执行，成为 SWP 的主流实现方式，同时也引入生产者与消费者之间的共享内存数据交换、mbarrier 同步以及角色划分带来的线程利用率变化。目前的优化方法主要依赖开发者经验^[1]或编译器启发

式^[18–20]，对 SWP 流水线深度、消费者排列顺序与 WS 资源分配的联合分析仍有扩展空间。

Twll^[49]是近期在这一方向上的重要工作。Twll 将经典模调度理论引入张量核心 GPU，通过 Yices2 SMT 求解器生成量化自由线性整数算术 (Quantifier-Free Linear Integer Arithmetic, QFLIA) 约束，自动求解 Hopper 与 Blackwell 架构上的 SWP 调度与 WS 映射。Twll 的核心思路是将每条 Tile 级指令建模为占用特定功能单元若干拍的操作节点，以最小启动间隔 II 为优化目标，同时处理功能单元占用与数据依赖时序，从而实现无启发式的精确调度。实验表明 Twll 能够自动生成逼近 FlashAttention-3 和 FlashAttention-4 手写调度的代码。

总体来看，要充分利用 Hopper 等现代 GPU 的性能，需要让高层的流水线调度与底层的线程束特化更紧密地配合，使复杂异步流水线模式能够从编译期分析中自动生成。这正是本文所要解决的核心问题。

第 3 章 AutoMixed: 资源感知的图级优化策略

从软件栈视角看，算子开发语言与系统正在沿着开发效率与执行性能两个目标轴形成一条帕累托前沿（Pareto Frontier），如图 3-1 所示。PyTorch、NumPy 等框架开发效率最高，但难以在新硬件上逼近性能上限。SASS、PTX 等底层实现能触达更高的性能，但要求开发者手工管理指令依赖和资源约束。Triton^[18]、TileLang^[48]、Helion^[50] 等 DSL 编译器试图在二者之间取得更优的权衡。

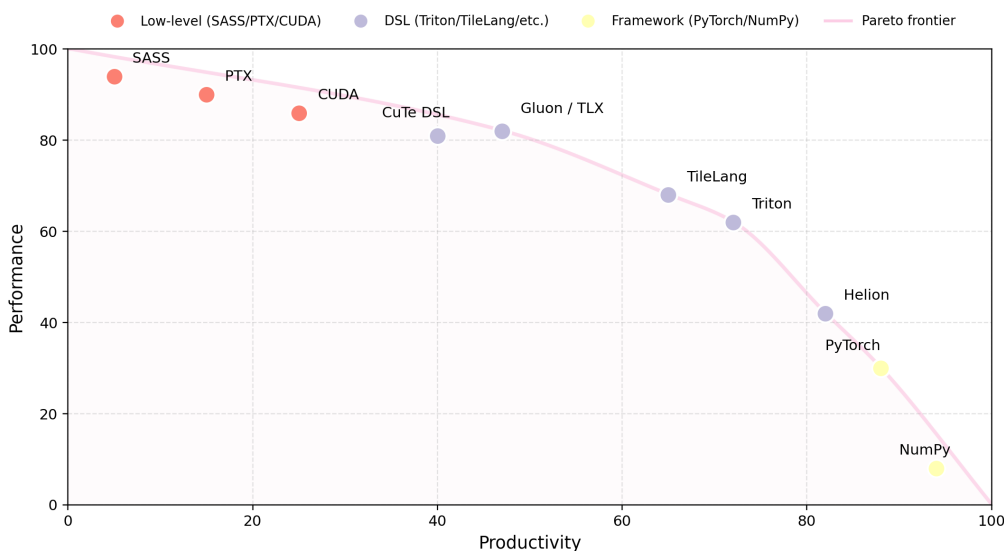


图 3-1 算子在“生产力-性能”维度的帕累托前沿^[51]

Figure 3-1 Pareto Frontier of Operator along Productivity-Performance Axes^[51]

如第 1.2 节所述，在 Hopper 架构上写出高性能算子，PTX 级的指令选择、流水线深度、分块大小以及算子融合与分离策略，共同构成了一个受寄存器容量、共享内存容量等硬件限制影响的复杂组合优化问题。传统的图编译器在进行算子融合决策时，往往缺乏对底层硬件微架构资源的精确感知，默认尽量多融合以减少内核启动开销。但在 H100 这类异步 GPU 上，融合过多会扩大活跃变量集合，寄存器溢出或共享内存不足，反而编译失败或性能下降。

性能不由单一层级控制。图级融合决策和底层流水线调度都会影响最终执行效率，二者必须协同。如果寄存器溢出或共享内存不足到后端才暴露，前端融合决策的调整成本会显著增加。

AutoMixed 以 PyTorch 计算图为输入，在向 Tile IR 降级的过程中引入资源感知机制。它通过指令级实现变体扩展、子图枚举与资源剪枝三个阶段，在编译早期排除必然导致溢出的编译路径，使最终生成的子图不超过寄存器和共享内存等硬件限制。资源感知的图划分还能分离被过度融合掩盖的并行结构，为下游的流水线调度提供更多选择。

3.1 从 PyTorch 到 TIR 的端到端编译流程

AutoMixed 的整体工作流分为以下三个阶段，其技术路线总览如图 3-2 所示。

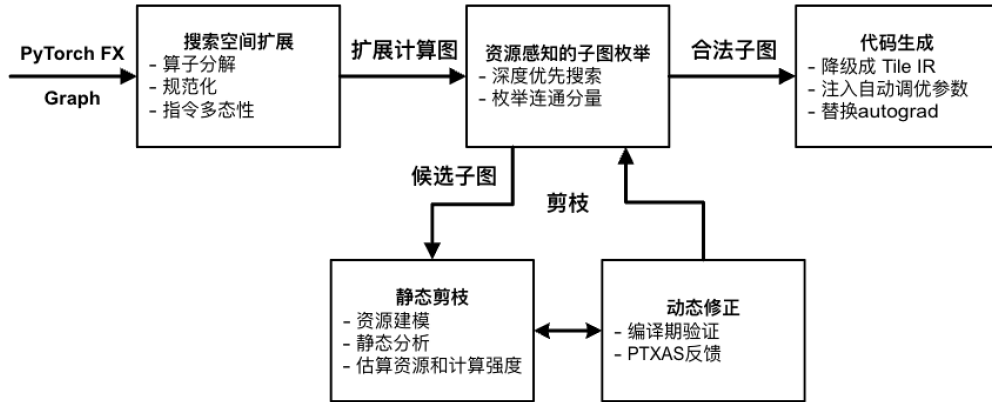


图 3-2 AutoMixed 技术路线总览

Figure 3-2 AutoMixed Technical Roadmap Overview

三个阶段都围绕寄存器与共享内存这两项片上资源展开。第一阶段为每个算子保留可选的指令模式，第二阶段用资源模型过滤明显不可行的候选，第三阶段再把剩余候选交给后端生成代码，并在需要时调用自动调优处理动态形状。

第一阶段是**计算图捕获与搜索空间扩展**。系统通过 `torch.compile` 捕获 PyTorch FX Graph 及其张量元数据，并执行算子分解与规范化，将 LayerNorm、Softmax 等复合算子拆解为元算子序列，以扩大可融合空间。随后，系统为同一算子保留多个语义等价但资源开销不同的实现。以 WGMMA 为例，寄存器-共享内存布局把左操作数放在寄存器中以提高带宽，共享内存-共享内存布局把两侧操作数都放在共享内存中以降低寄存器压力。原本藏在后端模板里的资源差异由此暴露到图级搜索中。

第二阶段是**资源感知的子图枚举与剪枝**。AutoMixed 首先用深度优先搜索枚举连通的候选子图，再用轻量级资源模型估算每个候选的寄存器峰值与共享内存占用，超出预算的候选直接丢弃。这个过程既减少搜索量，也会主动拆开过度融合的算子，把原本被融合掩盖的并行结构重新暴露出来。静态估算之外，系统还解析 `ptxas` 的编译反馈，用真实的寄存器数量、溢出字节数和动态共享内存占用修正判断。

第三阶段是**自动调优与代码生成**。通过资源校验的子图会被降级为 Tile IR，并在原计算图中替换为自定义的 `autograd.Function` 节点。当输入形状在编译期未知时，该节点保留必要的高层信息，并在运行时为当前形状选择合适的分块和线程参数。

AutoMixed 的决策发生在 TileLang 后端代码生成之前。它在图级选择候选融合边界和指令模式，并将通过资源检查的内核交给 TileLang 后端继续生成和调优；后端自动调优默认开启，用于在 AutoMixed 保留下来的候选配置中选择分块形状和线程参数。

3.2 基于指令级实现变体的搜索空间扩展

下面以 FlashAttention 的前向计算图为例，展示 AutoMixed 第一阶段的搜索空间扩展过程。这里选择前向图，是因为 QK 与 PV 两个 WGMMMA 节点已经足以说明操作数布局扩展；第 3.7 节再转向反向传播，用多输出梯度图说明图级分区选择。如图 3-3 所示，FlashAttention 的前向包含 QK、PV 两个 GEMM 节点和掩码、Softmax 等标量归约节点。在 Hopper 架构上，每个 GEMM 节点在降级时需要选择寄存器-共享内存布局或共享内存-共享内存布局，同时还需要决定哪些节点融合为同一内核、哪些拆开。

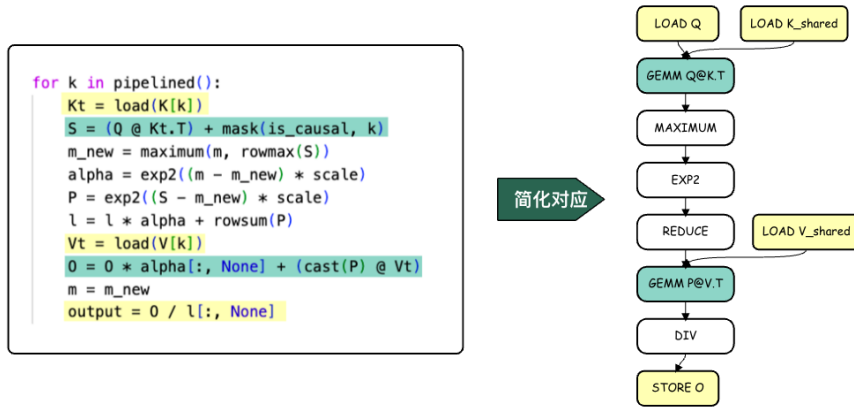


图 3-3 FlashAttention 前向计算图

Figure 3-3 FlashAttention Forward Computation Graph

为了在图级阶段考虑 Hopper 的硬件差异，AutoMixed 首先扩展计算图的搜索空间。传统图编译器通常只记录算子语义，而 AutoMixed 还记录同一算子在底层指令上的不同实现方式。

对于将被映射到 WGMMMA 的矩阵乘节点，系统构造两类候选实现。以 FlashAttention 前向为例，如图 3-4 所示，寄存器-共享内存变体（RS）将左操作数放在寄存器中、右操作数放在共享内存中，通常带宽利用率更高，但寄存器压力也更大。共享内存-共享内存变体（SS）将两侧操作数都放在共享内存中，寄存器压力更低，但会增加共享内存占用，也更容易受到共享内存带宽和存储体冲突的影响。提前保留这两类候选后，后续的资源剪枝和分区决策才能比较它们的真实代价。

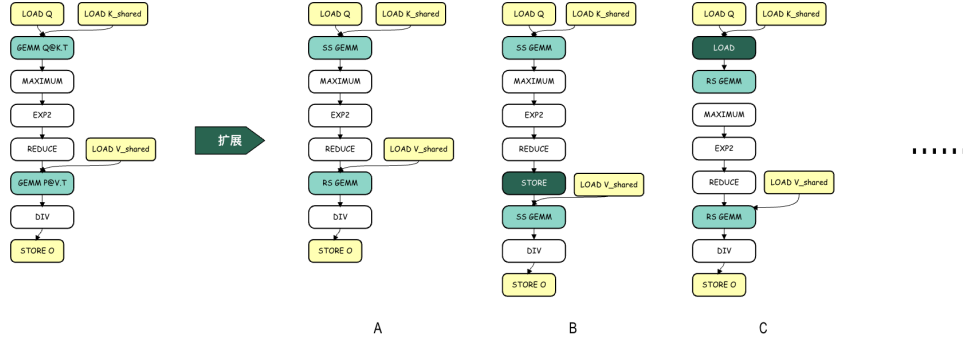


图 3-4 FlashAttention 前向中 WGMMMA 节点的操作数布局扩展示例

Figure 3-4 Operand Layout Expansion of WGMMMA Nodes in FlashAttention Forward

3.3 基于连通子图的候选融合枚举

指令级实现变体扩大了单节点的候选空间之后，还需要决定哪些节点融合为一个内核。AutoMixed 参考 FlashTensor^[17] 的分区方法，用深度优先搜索枚举计算图中所有合法的连通子图，每个子图对应一种候选融合方案。图 3-5 以 FlashAttention 前向为例展示了该枚举过程，从单算子粒度的不融合方案一直到整图融合方案都会被列入候选。

对每个候选子图，系统会继续分析其中张量的规约与广播属性，判断哪些中间结果可以在寄存器或共享内存中复用；随后再结合上一阶段的操作数布局，生成具体的编译配置。原本抽象的图级融合问题被转化为一批具体配置的资源检查问题。候选子图的数量随图节点数快速增长，但下一节的资源模型会提前剪枝，实际进入代码生成的候选通常只有个位数。

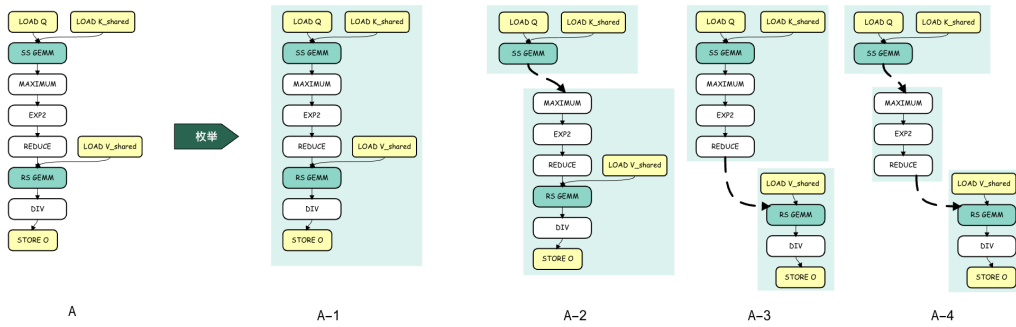


图 3-5 通过 DFS 枚举连通子图生成的候选分区方案

Figure 3-5 Candidate Partition Plans via DFS Enumeration of Connected Subgraphs

3.4 资源约束驱动的剪枝与决策

为降低候选评估成本，AutoMixed 在图级阶段先做一次轻量级资源估算。系统根据算子的读写关系和张量属性，估计候选子图在执行过程中需要同时保留哪些数据，例如 GEMM 累加器、未写回的中间结果、双缓冲片段和同步状态。

由此可以得到寄存器与共享内存的峰值占用，并在代码生成前剔除明显超出硬件预算的候选。

针对将被映射为 WGMMMA 的矩阵乘 $C = A \times B$ ($A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$)，设每个 WarpGroup 负责一个 $M \times N$ 输出分块，一次 WGMMMA 读取的 K 维片段大小为 K_{frag} ，累加器元素类型为 C_{acc} 。下面的寄存器估算均以每线程字节数计。输出累加器、寄存器驻留 A 片段、两类 WGMMMA 布局 and 深流水线总需求可统一写为

$$\begin{aligned} \text{Reg}_{\text{accum}} &\approx \frac{M \times N \times \text{sizeof}(C_{\text{acc}})}{\text{ThreadsPerWG}}, \\ \text{Reg}_A &\approx \frac{M \times K_{\text{frag}} \times \text{sizeof}(A)}{\text{ThreadsPerWG}}, \\ R_{\text{SS}} &\approx \text{Reg}_{\text{accum}} + \text{Reg}_{\text{misc}}^{(0)}, \\ R_{\text{RS}} &\approx \text{Reg}_{\text{accum}} + \text{Reg}_A + \text{Reg}_{\text{misc}}^{(0)} = R_{\text{SS}} + \text{Reg}_A, \\ \text{Reg}_{\text{total}} &\approx \text{Reg}_{\text{accum}} + n_A^{\text{live}} \text{Reg}_A + \text{Reg}_{\text{misc}}. \end{aligned} \quad (3-1)$$

其中 $\text{Reg}_{\text{misc}}^{(0)}$ 表示单次 WGMMMA 周围的固定寄存器开销， n_A^{live} 为同一时刻仍存活的寄存器驻留 A 片段数量， Reg_{misc} 汇总指针、同步状态以及其他中间值开销。对于共享内存-共享内存布局， $n_A^{\text{live}} = 0$ ；对于寄存器-共享内存布局， n_A^{live} 由流水线阶段数、预取位置和消费位置共同决定，通常不超过流水线深度 S 。因此，式 (3-1) 是上面单次 WGMMMA 估算在流水线场景下的推广，也是 AutoMixed 剪枝阶段使用的寄存器模型。

Hopper 架构单线程最多 255 个 32-bit 寄存器，每 SM 共 65536 个。为维持每 SM 同时驻留 2 个 WarpGroup (共 256 个线程)，每线程寄存器上限为 $65536/256 = 256$ 个。扣除编译器临时值和地址计算所需的余量后，**AutoMixed 将预算设为约 240 个 32-bit 寄存器/线程 (约 94%)**。该预算在功能上等价于 PTX 的 .maxnreg 240 指令，但作用于图级分区阶段而非 PTX 编译阶段：AutoMixed 用它做静态资源估算和候选剪枝，在代码生成前排除必然超出寄存器预算的融合方案。

系统遍历子图时会重点检查 GEMM 等会产生大中间张量的节点。当候选融合方案的资源需求逼近硬件上限时，AutoMixed 会倾向于把它拆成多个更小的内核。对于每个 GEMM 节点，系统还会比较两类操作数布局的代价；若寄存器-共享内存布局超过安全阈值，则切换为共享内存-共享内存布局，用少量理论性能换取更高的并发度和更稳定的编译结果。明显不可行的方案在图级阶段被排除，后端编译失败带来的试错成本也随之降低。

静态估算之后，AutoMixed 还会接入编译反馈。在 Hopper 这类资源约束很紧的硬件上，静态资源模型先快速排除必然溢出的候选；剩余候选再进行轻量级编译尝试，并根据 PTX 汇编器返回的寄存器数量、溢出字节数和动态共享内存占用做最终判定。这一两级流程在搜索成本和资源判断精度之间取得平衡。

图 3-6 展示了几类典型候选以及它们被保留或剪枝的原因。

第一列对应混合操作数布局的基线融合方案，即 GEMM 1 采用共享内存-共

享内存布局，GEMM 2 采用寄存器-共享内存布局。这种配置自然地利用了 TMA 和 CUDA Core 加载的位置，不引入额外的存取指令。它往往能在不过度放大活跃寄存器集合的前提下，利用寄存器驻留与带宽优势获得较高吞吐。然而，其寄存器压力的主要来源仍集中在寄存器-共享内存布局的 GEMM 上，当进一步增大分块或加深流水线时，很容易逼近寄存器容量阈值。

第二列展示了引入额外加载指令、将所有 GEMM 统一为寄存器-共享内存布局的方案。该策略能够发挥寄存器驻留操作数的高吞吐优势，但会显著扩大活跃寄存器集合。随着流水线加深，寄存器需求快速增长，更容易触发溢出甚至编译失败。因此，在资源受限的场景下，这类候选需要被优先识别并剔除。

第三列展示了引入额外的存储指令，将算子全部变成共享内存-共享内存布局的方案。该方案把更多操作数与中间状态放入共享内存，降低寄存器压力。代价是共享内存占用和访存指令增加；收益是可采用更深的流水线，使 TMA 与 WGMMA 有更长的重叠窗口。在寄存器压力主导的配置中，这类候选反而可能更优。

最后一列的两个例子表明，图切分位置不合适时，本应流式传递的中间结果会被迫写入 GMEM。这会产生大规模中间张量并降低算术强度，使内核从计算受限转为访存受限。因此，这类候选即使局部资源占用看起来安全，也会因为端到端代价过高而被提前剪枝。

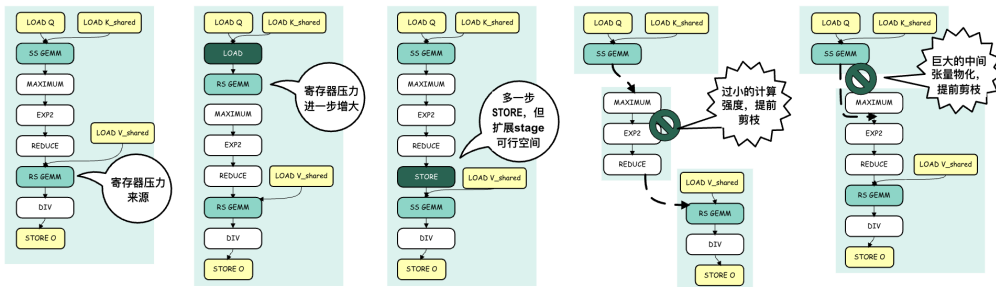


图 3-6 分区候选空间的寄存器与共享内存资源剪枝分析

Figure 3-6 Register and Shared Memory Resource Pruning of Partition Candidate Space

图中几类候选对应不同的剪枝原因。端到端代价明显过高的配置会被直接剪枝；涉及寄存器与共享内存权衡的候选，则继续结合运行时性能数据与 PTXAS 编译反馈选择。

3.5 布局约束的推导与传播

使用 Tensor Core 时，张量在共享内存中的布局必须与目标 WGMMA 指令匹配。AutoMixed 在 TileLang 后端布局推导能力的基础上^[20,48]，将相关约束纳入图级候选生成流程，从将要映射为 WGMMA 的计算节点出发，反向推导上游张量需要满足的对齐和混洗约束。例如，当某个节点将指定为 wgmma.mma_async.m64n128k16.f16 时，该指令的访存模式就成为布局约束的来源。随后，约束沿数据依赖边传播

到上游的全局内存加载和中间计算节点。生成 Tile IR 时，系统据此设置 TMA 存储和共享内存分配的布局属性，减少手写混洗异或掩码的工作，并降低目标 WGMMA 访问模式下的共享内存存储体冲突。

布局约束进入候选生成后，融合边界、操作数布局 and 分块形状会同时接受后端合法性检查。AutoMixed 在资源剪枝和分区评估阶段只保留满足布局要求的候选，后续代码生成面对的是可降级、可调优的内核形态。

3.6 候选配置评估与分区选择

静态资源模型可以快速排除明显不可行的候选，但剩余候选仍需要进一步排序。同一融合边界下仍可能存在多组分块形状、流水线级数、线程数和操作数布局；这些配置会改变共享内存占用、寄存器压力以及 TMA 与 WGMMA 的重叠空间。若全部交给真实编译和实测筛选，搜索成本会过高。

实现中，AutoMixed 将分块形状、流水线级数、线程数和操作数布局组合成候选配置。系统先用简化的内核代价模型为候选排序，估计每个候选的共享内存占用、寄存器风险、启动间隔 II 以及 TMA 与 WGMMA 的可重叠程度。其中 II 表示相邻两次循环迭代的启动间隔。该估计用于决定后续尝试顺序，最终代码生成和分块参数选择交由 TileLang 后端及其自动调优完成。

排序完成后，AutoMixed 先用共享内存估算排除超出硬件上限的候选，再对排序靠前的若干候选进行轻量编译尝试。系统解析 ptxas 输出的寄存器数、栈帧大小、溢出读写字节数和动态共享内存占用；若候选触发溢出或超过硬件预算，则按固定回退规则生成更保守的候选，例如将寄存器-共享内存布局改为共享内存-共享内存布局、降低流水线级数、缩小分块或增加线程数。最终被接受的配置必须同时通过静态资源检查和编译反馈检查。

图级分区的处理粒度更高。以 FlashAttention 反向为例，系统先从 dQ 、 dK 、 dV 等输出节点出发枚举候选分区，再对每个分区中的内核做资源估计和配置搜索。分区是否保留由资源可行性、编译反馈和候选配置的实测性能共同决定。

3.7 FlashAttention 反向传播中的图级分区选择

前向计算图适合说明单个 WGMMA 节点的实现变体；反向传播同时计算 dQ 、 dK 、 dV 三个梯度，包含更复杂的循环依赖和数据复用关系，更适合观察图级分区对资源占用和调度空间的影响。如图 3-7 所示，AutoMixed 在该算子上评估了三种图划分策略。

3.7.1 融合策略分析

传统的激进融合策略试图将三个梯度的计算合并至单一内核，例如 FlashAttention^[7] 中 Triton 的实现。然而，如表 3-1 所示，融合式内核必须同时维护 dO 、 V 、 S 、 P 等 9 个活跃张量，在头维度 $D=128$ 时共享内存占用高达 104.5 KB，严重挤压 L1

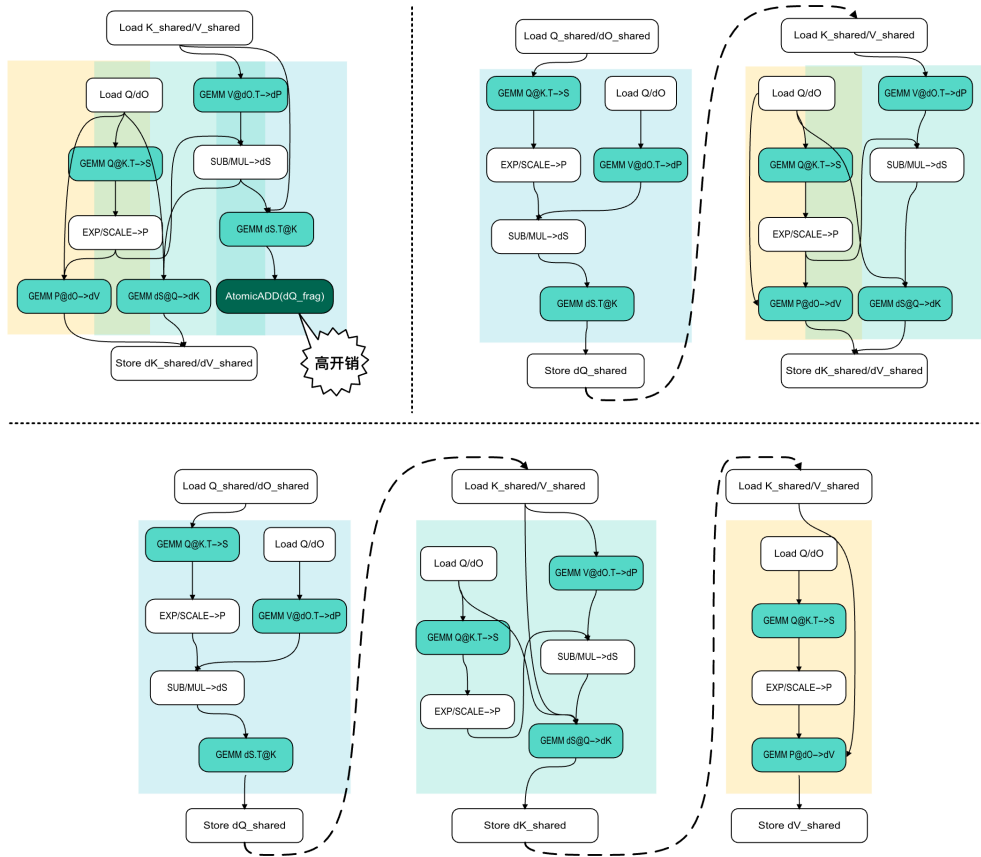


图 3-7 FlashAttention 反向传播的三种图级分区策略

Figure 3-7 Three Graph-Level Partitioning Strategies for FlashAttention Backward

缓存空间并限制 Warp 并行度。此外，融合策略下 dQ 的跨块累加需要依赖低效的原子操作（atomic_add）。

表 3-1 FlashAttention 反向各分区策略的资源占用分析

Table 3-1 Resource Footprint by Partition Strategy for FlashAttention Backward

划分策略	内核	头维度 D	估计 SMEM (KB)	活跃张量数
融合	dQ+dK+dV	64	56.5	9
2-分离	dKdV	64	48.5	8
	dQ	64	40.5	7
3-分离	dK	64	40.0	6
	dV	64	36.0	5
融合	dQ+dK+dV	128	104.5	9
2-分离	dKdV	128	96.5	8
	dQ	128	80.5	7
3-分离	dK	128	72.0	6
	dV	128	64.0	5

3.7.2 子图枚举与候选评估

面对 FlashAttention 反向传播的复杂计算图，AutoMixed 通过 DFS 枚举合法的连通子图划分。虽然理论上划分数会随节点数快速增长，但分离后的子图必须各自连通，再加上资源模型会提前剪枝，实际进入后续评估的候选数量并不大。下文选取三种代表性分区展示评估过程，分别是完全融合、将 dQ 剥离的 2-分离，以及将 dQ、dK、dV 全部分离的 3-分离。AutoMixed 先通过资源模型和轻量编译筛掉明显不可行的分区，再结合调度估计比较保留下来的候选。

分离后的 dKdV 内核不再维护与 dQ 相关的中间状态，活跃张量集合随之缩小。独立的 dQ 内核还可以利用确定的输出映射，消除原子竞争并缩短同步链路。分离的代价是 dKdV 内核需要在每个 Q 块中重新计算注意力概率矩阵 $P = \text{softmax}(QK^T)$ ；如果没有流水线重叠，这部分额外计算会直接增加延迟。

表 3-2 展示了不同寄存器预算下的分区选择。

表 3-2 不同寄存器预算下的分区选择

Table 3-2 Partition Selection under Different Register Budgets

$R_{\max}$ (B/线程)	选择的分区	估计调度长度	寄存器峰值
≥ 768	融合	84	768B
600	2-分离 (dKdV+dQ)	166	512B
512	2-分离 (dKdV+dQ)	166	512B
400	3-分离 (dK+dV+dQ)	245	256B

分区选择随寄存器预算呈现清晰的阶梯变化。当寄存器预算充裕时，融合方案可以保留更多数据局部性；当 2-分离方案中任一内核的寄存器峰值超过给定预算时，更细粒度的分离才会进入候选集合。这一过程由资源检查和候选评估共同决定，无需人工设定固定阈值。

3.7.3 代价分析

在默认的 $R_{\max}=960\text{B}$ 预算下，即 $240\text{ 寄存器} \times 4\text{ 字节}$ ，AutoMixed 不保留 3-分离作为最终方案，原因有三。

关键路径不可压缩。无论如何划分，每个内核都包含注意力分数计算 (score_kq, Tensor Core) $\rightarrow$ softmax (SFU) $\rightarrow$ 累加 WGMMA (Tensor Core) 这一串行依赖链，三者延迟之和构成关键路径下界。拆分不会缩短单个内核的关键路径，只会增加内核的数量。调度估计显示，3-分离的调度长度约为 2-分离的 1.5 倍。多出的 dV 独立内核净增一条完整的关键路径，却没有带来相应的调度收益。

重计算占用 SFU。3-分离方案中 dK 和 dV 各自独立重算 softmax，总共需要 3 次 softmax 计算 (2-分离仅需 2 次)，导致总算术量增加约 50%。流水线可以重叠一部分重计算延迟，但额外的 SFU 占用仍会进入稳态调度，并增大 II。

寄存器收益递减。2-分离已将最大内核的寄存器峰值降至 512B，远低于 960B 的阈值。3-分离虽然进一步降至 256B，但额外余量没有转化为更高吞吐。

评估结果显示，在当前寄存器上限下，继续拆分主要增加重计算和内核调度开销。

3.8 面向内存受限操作的融合决策

前述各节的代价模型聚焦于计算密集型算子，以寄存器与共享内存为核心约束指标。Transformer 层中还包含大量逐元素操作，如残差加、层归一化、激活函数与逐元素乘法，其计算强度很低，每字节仅 1 到 2 次浮点运算，性能瓶颈由全局内存带宽主导。若逐一独立执行，相邻操作之间的中间张量需写入 GMEM 再读回，带宽浪费随模型深度线性累积。

AutoMixed 将代价模型从寄存器与共享内存维度扩展到 GMEM 带宽维度。设某对相邻操作之间的中间张量大小为 B_{mid} 字节，融合后消除该张量的一次写入与一次读取，带宽节省为

$$\Delta_{\text{BW}} = 2B_{\text{mid}}. \quad (3-2)$$

对本文处理的逐元素模式，融合后的寄存器增量很小，且不引入新的跨线程同步；在这一前提下，决策主要由 GMEM 带宽节省主导。带宽模型用于前置筛选。候选融合需同时满足不引入额外同步、不破坏访存合并、寄存器增量不降低 SM 并发度三个条件，AutoMixed 再按 Δ_{BW} 估计收益并保留候选；否则转入寄存器、共享内存与编译反馈共同约束的判断。

以 Llama2-7B 为例，AutoMixed 在每个 Transformer 层中识别出三种可融合模式。

残差加与 RMSNorm 融合。未融合时，残差加的输出写入 GMEM，RMSNorm 再将其读回。融合后残差在寄存器中直接传递给归一化计算，消除一个 (B, T, H) 张量的往返读写。该模式在每层出现两次，分别对应注意力子层和 FFN 子层的残差连接。

Gate 与 Up 投影合并。gate 和 up 两个线性层各自独立读取相同的输入张量，合并为单次 GEMM 调用后输入仅需读取一次，消除一次冗余读取。

SiLU 激活与逐元素乘法融合。未融合时，SiLU 的输出写入 GMEM，再由逐元素乘法读回。融合后二者在同一内核中完成，消除中间维度为 I 的大张量的往返读写。

表 3-3 给出了各模式在 $T=4096$ 时的定量分析。三项融合每层合计消除约 332 MB 的 GMEM 流量，32 层累计消除约 10.4 GB。以 H100 的 3.35 TB/s HBM 带宽估算，带宽节省对应约 3.3 ms 的带宽时间下界。该估算未计入内核启动次数减少和内存控制器排队变化；这些因素将在第 6 章的模型级实验中体现。

这一扩展使 AutoMixed 从针对 WGMMMA 的寄存器与共享内存优化，延伸到内存受限操作的图级融合决策。两类操作共享同一套子图枚举与融合决策框架，区别在于代价函数的选择。WGMMMA 使用公式 (3-1) 的寄存器模型，逐元素操作使用公式 (3-2) 的带宽模型。

表 3-3 Llama2-7B 逐元素融合的带宽代价分析

Table 3-3 Bandwidth Cost Analysis of Elementwise Fusion for Llama2-7B

融合模式	中间张量	每次节省	每层频次	全模型节省
残差加 + RMSNorm	$(B, T, 4096)$	64 MB	2	4096 MB
Gate 与 Up 合并	$(B, T, 4096)$	32 MB	1	1024 MB
SiLU + 逐元素乘	$(B, T, 11008)$	172 MB	1	5504 MB
合计				10624 MB

$B=1$, $T=4096$, FP16, 32 层。残差加 + RMSNorm 与 SiLU + 逐元素乘消除一次写入与一次读取, 节省 $2S$; Gate 与 Up 合并仅消除一次输入读取, 节省 S 。

3.9 本章小结

本章针对 Hopper 架构上算子融合与硬件资源约束之间的矛盾, 提出了 AutoMixed 资源感知的图级优化策略。通过在从 PyTorch 到底层 Tile IR 的降级过程中引入指令级实现变体扩展、子图枚举与资源剪枝机制, AutoMixed 能够在图编译的早期阶段提前识别并剔除必然导致寄存器溢出或共享内存不足的无效配置。

在此基础上, AutoMixed 将分区选择与轻量级调度估计结合, 在同一评估流程中比较不同分区的寄存器占用、共享内存占用和预估调度长度。这里的调度估计服务于分区排序; 最终代码生成时的屏障位置和消费者排列由下一章的 Heddle 调度流程处理。

以 FlashAttention 反向传播为例, AutoMixed 按候选枚举、编译反馈和流水线后的实测结果选择分区。融合方案因寄存器压力导致溢出, 且加深流水线几乎没有收益; 3-分离方案虽然进一步降低寄存器占用, 但重复 softmax 和额外内核边界使吞吐下降; 2-分离方案在消除溢出的同时仍保留 dK 与 dV 的共享输入结构, 并能通过流水线把重计算开销部分隐藏。在该测试配置下, 2-分离成为最终方案, 也为下游调度保留了更多可用方案。

在计算密集型算子之外, AutoMixed 将代价模型扩展到 GMEM 带宽维度, 对 Transformer 层中的逐元素操作进行融合决策。以 Llama2-7B 为例, 残差加与 RMSNorm 融合、Gate 与 Up 投影合并、SiLU 与逐元素乘法融合三项优化可以消除多次中间张量往返 GMEM, 使 AutoMixed 的适用范围从计算密集型融合扩展到带宽受限逐元素融合。

AutoMixed 的融合决策影响代码生成的正确性, 也决定下游调度优化的可用空间。融合边界确定流水线循环体中的 WGMMA 数量, 以及相邻 WGMMA 之间是否包含 Softmax、归约等非张量核运算; 分块形状决定每条 WGMMA 的 K 维迭代次数和共享缓冲区大小。这些结构信息随后由 AutoPiped 转化为循环级流水线注解, 并构成 Heddle 模块的多角色候选评估 (§5.4) 的输入。WGMMA 数量影响异步等待策略, 非张量核间隔影响重叠窗口, 缓冲区字节数影响 TMA 延迟。由此, AutoMixed 决定图级融合边界, AutoPiped 决定循环级流水线注解, Heddle 决定融合后内核的线程束特化执行方式。三者通过 IR 结构衔接, 无需额外传参。

第4章 AutoPiped：语义驱动的流水线配置参数生成方法

在第3章中，AutoMixed 已确定算子融合边界与 WGMMA 指令布局，并可结合后端自动调优确定分块形状，产出了结构确定的 Tile IR 循环体。要把该循环体编译成 Hopper 异步流水线，还需生成指令分组 group、组间执行顺序 order 和流水线阶段 stage 三类注解，其含义见表 2-1。

穷举这三类参数的组合成本很高。以一个包含 n 条语句、 S 个流水线阶段的循环体为例，若不预设连续边界，将语句划分为若干非空指令组的方案数为第 n 个贝尔数 B_n ¹。若某种分组得到 m 个指令组，最多还需为这些组选择有标号的流水线阶段；由于阶段编号允许没有消费者使用，这一步有 $O(S^m)$ 种阶段标注。再考虑组间执行顺序，最多还有 $O(m!)$ 种排列。上述计数是搜索空间上界，尚不包含依赖关系、生产者位置和同步边界带来的剪枝；即便如此，实际算例中的候选总数仍可超过 10^6 。第 1.2 节给出的 FlashAttention 前向案例中，固定分组后仅枚举组间顺序与流水线阶段已有 2328 个候选，约 10% 能通过验证。可用配置占比低，组间执行顺序与流水线阶段还必须匹配 Hopper 的 mbarrier 同步协议，否则会产生数据竞争或流水线死锁。

AutoPiped 利用循环体自身的语义信息生成这些参数。某条语句是否为生产者，取决于它执行的指令和数据流向；消费者处在哪个阶段，取决于它读取哪个共享内存缓冲区；哪些语句应该归为一组，取决于算子的计算结构。这些信息已经存在于 Tile IR 中，可由编译期分析转化为流水线配置。AutoPiped 生成 TileLang 后端所需的 group、order 和 stage 三类配置参数；TileLang 后端据此完成环形缓冲区展开、mbarrier 注入和代码生成。

如图 4-1 所示，AutoPiped 的推断分为 IR 展开与读写区域分析、语句角色与分组推断、阶段推断、执行顺序分配与优化四个步骤。第一步提取后续推断需要的读写信息；第二步确定 group 作用到哪些语句；第三步为各组生成 stage；最后一步在依赖允许的范围内存成 order。

TileLang^[20,48] 已经提供环形缓冲区和 mbarrier 的底层代码生成能力，但上述三类注解仍需要用户手动填写。以 FlashAttention 前向为例，**开发者需要把 15 条循环体语句分成若干组，并为消费者组指定发射顺序和流水线阶段，任一项填错都可能导致死锁或数据竞争**。当分块形状或算子结构变化时，这些注解还要重新调整。AutoPiped 将这一过程变成编译器 Pass，通过读写区域分析识别生产者-消费者结构，生成合法的流水线配置。后续 Heddle 在这些注解基础上，进一步处理缓冲区级屏障、消费者重排，以及寄存器和共享内存等资源上限检查。

¹ 贝尔数 B_n 等于将 n 个元素划分为非空子集的总方案数，例如 $B_{10} = 115975$ ， $B_{15} \approx 1.38 \times 10^9$ 。

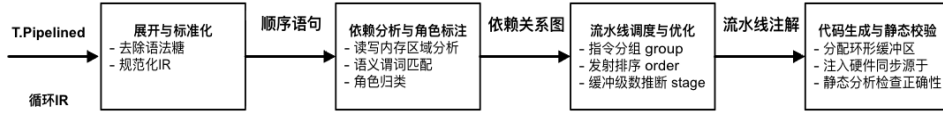


图 4-1 AutoPiped 技术路线总览

Figure 4-1 AutoPiped Technical Roadmap Overview

4.1 循环体分析与配置推断

AutoPiped 的输入是带 `num_stages` 注解的 T.Pipelined 循环。在 TVM TIR 中，循环体通常会被 BlockRealize、Block、LetStmt、AttrStmt 等结构层层包裹。分析过程先剥离这些包裹层，将循环体整理成顺序语句序列 $\langle s_0, s_1, \dots, s_{n-1} \rangle$ 。整理后的每条顶层语句 s_i 都作为独立分析单元，对应循环体中的一次计算或访存操作。

对序列中的每条语句 s_i ，AutoPiped 提取其读写缓冲区。系统将 s_i 封装成临时计算块 B_i ，再结合全局缓冲区映射 $\mathcal{M}$ 调用 TVM 的区域分析接口，得到读集合 $\mathcal{R}_i$ 和写集合 $\mathcal{W}_i$ 。映射 $\mathcal{M}$ 由函数参数缓冲区和函数体内的局部分配共同组成。返回的 $\mathcal{R}_i$ 与 $\mathcal{W}_i$ 是 BufferRegion 集合，记录语句访问的缓冲区、存储域 scope 和多维索引范围。TVM 区域分析器遍历块内的 BufferLoad 与 BufferStore，估计索引范围并按缓冲区聚合。AutoPiped 以这一分析结果为基础，再按 global、shared/shared.dyn 和 local 等存储域区分全局显存、共享内存与线程私有内存访问。后续角色识别根据读写集合判断语句是否执行全局显存到共享内存的搬运；阶段推断根据消费者的读集合判断它依赖哪一版共享缓冲区。

4.1.1 语句角色识别

获取每条语句的读写区域后，AutoPiped 先判定语句角色，再生成候选分组，随后把共享缓冲区版本关系聚合为组级 stage 注解。在这一流程中，group 先给出注解作用的语句范围，stage 再描述每个组读取的缓冲区版本。

AutoPiped 根据表 4-1 的规则为每条语句标注角色。两类生产者分别通过异步拷贝指令名和存储域的读写方向识别；同步语句通过硬件同步原语识别，并在后续依赖图中作为保序约束处理；其余语句标注为计算消费者。

4.1.2 分组模板生成与选择

分组决定 order 和 stage 注解作用到哪些语句，因此先于阶段和顺序生成。AutoPiped 遵循两个原则。生产者语句单独成组，以满足后端对异步加载语句的处理要求；消费者语句按计算结构合并或切分，以减少不必要的同步，同时保留必要的重排空间。同步语句不参与合并，其相对位置通过保序约束维护。基于这两个原则，AutoPiped 生成表 4-2 中的四类候选模板。

枚举 n 条消费者语句的全部分组方案会形成贝尔数 B_n 量级的搜索空间，在实际算子规模下评估成本很高。上述模板把任意分组搜索约束为少数几种结构

表 4-1 AutoPiped 的角色判定规则

Table 4-1 Role Determination Rules of AutoPiped

角色	判定依据	在流水线中的作用
异步拷贝生产者	语句包含 tma_load、tma_load_im2col 或 ptx_cp_async 等异步拷贝指令	作为生产者组, 触发环形缓冲区与 mbarrier 注入
SIMT 生产者	$\exists b \in \mathcal{W}_i, \text{IsShared}(b)$, 且 $\mathcal{R}_i \neq \emptyset$, $\forall b \in \mathcal{R}_i, \text{IsGlobal}(b)$	作为传统生产者, 定义共享缓冲区版本
同步切分点	语句含以下同步原语 mbarrier_wait_*, ptx_arrive_barrier、ptx_wait_barrier	保守保序点, 禁止破坏同步时序
计算消费者	其余计算语句	参与分组、阶段推断与顺序优化

表 4-2 AutoPiped 候选分组模板

Table 4-2 AutoPiped Candidate Grouping Templates

模板	消费者处理方式	典型算子
按缓冲区版本切分	相邻消费者依赖的共享缓冲区版本不同时切分	GEMM、简单双阶段算子
按缓冲区版本与计算类型切分	共享缓冲区版本变化, 或 WGMMA、SFU 与普通 CUDA Core 边界处切分	FlashAttention 前向、含标量归约的算子
按生产者边界合并	位于相邻生产者之间的连续消费者合并为一个逻辑组	两段式结构
全消费者粗合并	保留生产者单独成组, 消费者按更粗粒度合并以减少同步切分	FlashAttention 反向等交错结构

化切分。按缓冲区版本切分保留共享缓冲区版本边界; 按缓冲区版本与计算类型切分进一步区分 WGMMA 与标量计算; 按生产者边界合并降低细粒度屏障数量; 全消费者粗合并用于消费者高度交错、细粒度阶段划分收益较低的场景。分组生成由此从 B_n 级枚举降为对四类模板的线性扫描。

这四类模板形成由细到粗的分组粒度。细粒度模板保留更多组间重排空间, 也能更精确地表达不同消费者对共享缓冲区的依赖; 粗粒度模板减少屏障切分和搜索规模, 适合消费者高度交错或版本差异影响较小的算子。按缓冲区版本切分模板需要知道相邻消费者是否读取同一版生产者数据, 因此会先从读写关系中提取缓冲区版本键, 用于判断是否切分; 最终组级 stage 在分组选定后再统一生成。AutoPiped 以按缓冲区版本切分作为默认模板; 当生产者-消费者图包含多条 WGMMA 链、标量归约链或明显的生产者边界时, 会补充更细或更粗的模板候选, 并交由顺序优化和后端验证筛选。

选定模板 π 后, AutoPiped 得到逻辑分组 $\mathcal{G} = \{G_0, G_1, \dots\}$ 。模板决定消费者的合并或切分粒度, 依赖合法性由读写关系和保序约束保证。后续阶段推断以这些组为对象生成 stage, 顺序优化再为消费者组生成 order。

4.1.3 阶段推断

流水线阶段表示消费者与环形缓冲区版本之间的对应关系, 语句发射先后由后续的 order 决定。上一节用于分组的缓冲区版本键, 在这里被聚合为组级 stage。AutoPiped 在给定 $S = \text{num_stages}$ 的候选上推断各组的 stage。S 来自 T.Pipelined 的 num_stages 注解, 也可以由外层配置搜索给出; AutoPiped 在每个给定的 S 上生成对应的阶段与顺序注解。S = 2 对应双缓冲, 环形缓冲区保留两个版本; S = 3 对应三缓冲, 环形缓冲区保留三个版本。更大的 S 使生产者更晚遇到同一缓冲区版本的复用等待, 也带来更多共享内存占用和更长的前序/后序。

stage 和 order 负责不同问题。stage 与 S 共同决定环形缓冲区的版本映射和生产者可提前准备的分块数; order 决定同一次主循环重复片段中各组的发射先后, 例如是否把 PV 提前到 Softmax 和 Cast 之前。两组配置即使 order 相同, 也可能因 stage 和 S 不同而形成不同的跨迭代距离。

阶段推断分两步。第一步计算语句级缓冲区版本, 确定每条消费者语句依赖哪一个生产者写入版本。设 B^{prod} 为所有生产者写入的共享缓冲区集合。AutoPiped 线性扫描语句序列, 每遇到一个生产者, 就把它写入的共享缓冲区记录为当前生产者序号 $\text{prod_ver}(b)$ 。后续消费者如果读取该缓冲区, 就继承这个生产者序号。对消费者语句 s_i , 记 R_i^{shared} 为它读取的共享缓冲区集合, $P_i = R_i^{shared} \cap B^{prod}$ 。扫描到 s_i 时, 记 p_i 为此前最近生产者的版本, c_i 为此前最近消费者的版本; 若此前没有生产者或消费者, 则对应值取 0。

$$\text{ver}(s_i) = \begin{cases} \max_{b \in P_i} \text{prod_ver}(b), & P_i \neq \emptyset, \\ p_i, & P_i = \emptyset \text{ 且此前已有生产者}, \\ c_i, & P_i = \emptyset \text{ 且此前没有生产者}. \end{cases} \quad (4-1)$$

消费者优先继承其读取的共享缓冲区版本。若消费者不读取任何生产者写入的共享缓冲区, 则沿用最近生产者版本; 在尚未出现生产者时, 沿用前一条消费者版本。该规则把 Softmax、Cast 这类处理寄存器中间结果的语句保留在相邻计算链上, 避免被分配到无关的共享缓冲区版本。

第二步把语句级版本聚合到已选定的分组。给定分组模板 π 、流水线级数 S 和阶段偏移 Δ 后, 组 G_j 的阶段为

$$\text{stage}(G_j) = \min \left(S - 1, \max_{s_i \in G_j} \text{ver}(s_i) + \Delta \right). \quad (4-2)$$

其中 Δ 表示额外阶段偏移。 $\Delta = 0$ 采用数据依赖自然给出的最小阶段; $\Delta = 1$ 表示把消费者对应的缓冲版本整体后移一个阶段; 在 S 足够大时, 这会为生产者侧保留更多可提前写入的缓冲区版本。设 $v_{\max} = \max_j \max_{s_i \in G_j} \text{ver}(s_i)$, 为避免阶段饱和后的重复配置, Δ 的有效枚举范围为 $\Delta \in \{0, \dots, S - 1 - v_{\max}\}$ 。若 $S - 1 - v_{\max} < 0$, 则只保留 $\Delta = 0$ 并通过上式截断到 $[0, S-1]$ 。按缓冲区版本切

分时, 同组消费者通常具有相同版本键; 按缓冲区版本与计算类型切分、按生产者边界合并和全消费者粗合并时, 同组消费者可能来自多个版本, 因此组级阶段取组内最大值。

以 FlashAttention 前向为例, Load K 写入 K 缓冲区, Load V 写入 V 缓冲区。QK 读取 K , 因此版本为 0; Softmax 和 Cast 不读取新的共享缓冲区, 沿用 QK 的版本 0; PV 读取 V , 因此版本为 1。语句级结果为 $[-1, 0, 0, 0, -1, 1]$, 其中 -1 表示生产者。对按缓冲区版本切分模板, $S = 2, \Delta = 0$ 给出的组级阶段与版本键一致, 即 $[-1, 0, 0, 0, -1, 1]$ 。若将缓冲级数增至 $S = 3$, 但仍保持 $\Delta = 0$, 组级阶段向量仍为 $[-1, 0, 0, 0, -1, 1]$, 但后端已经会按三缓冲展开环形缓冲区。进一步取 $\Delta = 1$ 时得到 $[-1, 1, 1, 1, -1, 2]$, 对应本案例中的三缓冲优化配置。该候选增加一份共享内存缓冲区占用, 并延长前序和后序; 对应生成代码中, 生产者可以连续写入三组 K 和 V 后才复用旧版本, 而双缓冲配置在第三组 K 和 V 起就需要等待旧版本释放。阶段候选确定后, order 再决定这些组在同一次主循环重复片段中的发射先后。

算法 3 汇总了从角色识别、分组模板选择、阶段推断到执行顺序生成的流程。

算法中的符号单位如下: S 是循环体顶层语句序列, $|S|$ 为语句条数; $\mathcal{G}$ 是逻辑分组集合; S 为流水线级数, 单位是共享内存环形缓冲区版本数; Δ 为非负整数阶段偏移, 用于枚举更深的消费者阶段; order 中的 -1 表示生产者组, 非负整数表示消费者组在同一次主循环重复片段中的发射顺序。

算法 3 AutoPiped 组级流水线注解生成流程

Algorithm 3 AutoPiped Group-Level Pipeline Annotation Generation

输入: 语句序列 $S = \langle s_0, \dots, s_{n-1} \rangle$; 角色标记 $\{\text{role}_i\}$; 读写区域 $\{(\mathcal{R}_i, \mathcal{W}_i)\}$; 阶段数 S ; 阶段偏移 Δ ; 可选分组配置 π_{cfg}

输出: 选定分组 $\mathcal{G}^*$ 及对应的 order^* 、 stage^*

阶段一: 分组模板生成与选择

- 1: $\text{ver} \leftarrow \text{InferBufferVersionKey}(S, \text{role}, \{(\mathcal{R}_i, \mathcal{W}_i)\})$ ▷ 用于按缓冲区版本切分模板判断边界
- 2: $\Pi \leftarrow \{\text{按缓冲区版本切分, 按缓冲区版本与计算类型切分, 按生产者边界合并, 全消费者粗合并}\}$
- 3: $\pi^* \leftarrow \text{SelectTemplate}(S, \text{role}, \text{ver}, \Pi, \pi_{\text{cfg}})$
- 4: $\mathcal{G}^* \leftarrow \text{BuildGroups}(S, \text{role}, \text{ver}, \pi^*)$ ▷ 生产者单独成组, 消费者按 π^* 切分或合并

阶段二: 组级阶段推断

- 5: $\text{stage}^* \leftarrow \text{AggregateStage}(\mathcal{G}^*, \text{ver}, S, \Delta)$

阶段三: 执行顺序生成

- 6: $\text{order}^* \leftarrow \text{ASAPSearch}(S, \mathcal{G}^*, \{(\mathcal{R}_i, \mathcal{W}_i)\})$
- 7: **if** order^* 不合法 **then**
- 8: $(\mathcal{G}^*, \text{stage}^*, \text{order}^*) \leftarrow \text{按缓冲区版本切分得到的保守配置}$
- 9: **end if**
- 10: **return** $(\mathcal{G}^*, \text{order}^*, \text{stage}^*)$

4.2 执行顺序分配与 ASAP 顺序优化

确定分组与组级阶段后，还需为每个消费者组分配 order。生产者组固定为 $\text{order} = -1$ ，消费者组初始按源码顺序赋 $\text{order} = 0, 1, 2, \dots$ 。源码顺序保证依赖安全，流水线重叠效率还取决于消费者组之间的交错方式。在稳态流水线中，不同消费者组的顺序会影响流水线气泡分布；两个消费者组之间没有数据依赖时，交换顺序仍保持语义正确。AutoPiped 在依赖允许的范围内重排消费者组，以提高计算与预取的重叠比例。

AutoPiped 先为语句建立依赖图，如图 4-2 所示。图中每个节点对应一条语句，边分为程序顺序边和数据依赖边。程序顺序边保留同步语句、具有副作用的语句、别名关系无法判断的语句，以及同一组内部相邻语句之间的相对顺序。数据依赖边记录共享内存上的写后读关系。若消费者语句 s_j 读取了生产者语句 s_i 写入的共享缓冲区，则从 s_i 连边到 s_j ，表示 s_j 必须在 s_i 之后启动。依赖无关的消费者组可脱离源码中的相邻顺序，参与后续重排。

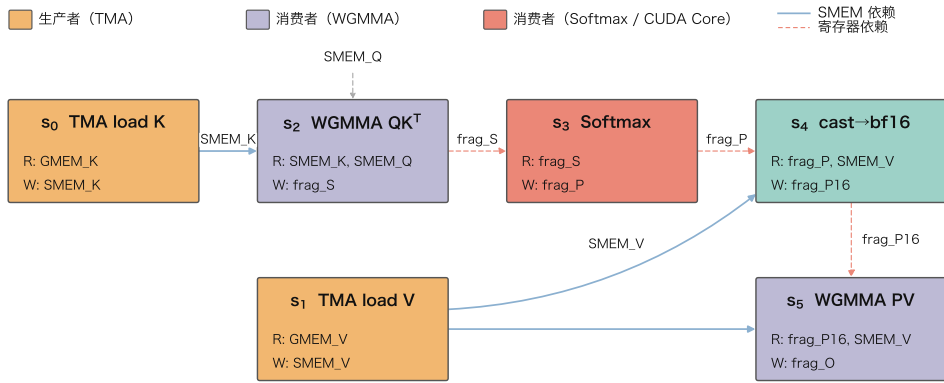


图 4-2 AutoPiped 依赖图构建示例

Figure 4-2 Example Dependency Graph Construction in AutoPiped

在该依赖图上，AutoPiped 为每条语句 s_i 计算拓扑层级 $T[i]$ 。 $T[i]$ 是用于排序的相对层级，表示在语义依赖约束下，语句 s_i 进入消费者序列的最早位置。无前驱语句的层级为 0；有前驱语句的层级由最长前驱链决定。AutoPiped 从无入边的源节点出发，通过最长路径松弛迭代更新。

$$T[i] = \begin{cases} 0, & \text{pred}(s_i) = \emptyset, \\ \max_{s_j \in \text{pred}(s_i)} (T[j] + 1), & \text{pred}(s_i) \neq \emptyset. \end{cases}$$

初始时令 $T[i] = 0$ 。该式等价于在依赖图上计算到达每条语句的最长路径长度，用于度量语句的依赖深度。

得到各语句的拓扑层级后，AutoPiped 为每个消费者组 G_j 计算组级排序键，记为 min_T 。

$$\min_T(G_j) = \min\{T[i] \mid i \in G_j\}$$

$\min_T(G_j)$ 表示组内入口语句的最早依赖层级，用作组级排序键。随后，消费者组按 $\min_T$ 升序排列；相同 $\min_T$ 的组沿用源码位置稳定排序。组内语句的实际先后仍由组内顺序和依赖边约束。

图 4-3 给出了 FlashAttention 前向中的一个具体例子。为突出消费者顺序，图中省略了 Load K 和 Load V 两个生产者组，它们的 order 固定为 -1。在语义推断起点中，消费者按程序顺序排列为 $QK_{k+1} \rightarrow \text{Softmax}_{k+1} \rightarrow \text{Cast}_{k+1} \rightarrow PV_k$ 。依赖图显示， Softmax_{k+1} 依赖 QK_{k+1} ， Cast_{k+1} 依赖 Softmax_{k+1} ，因此三者的拓补层级分别为 0, 1, 2。 PV_k 消费上一重复片段已经产生的概率分块，与当前 QK_{k+1} 链无语义依赖，因此 $T = 0$ 。

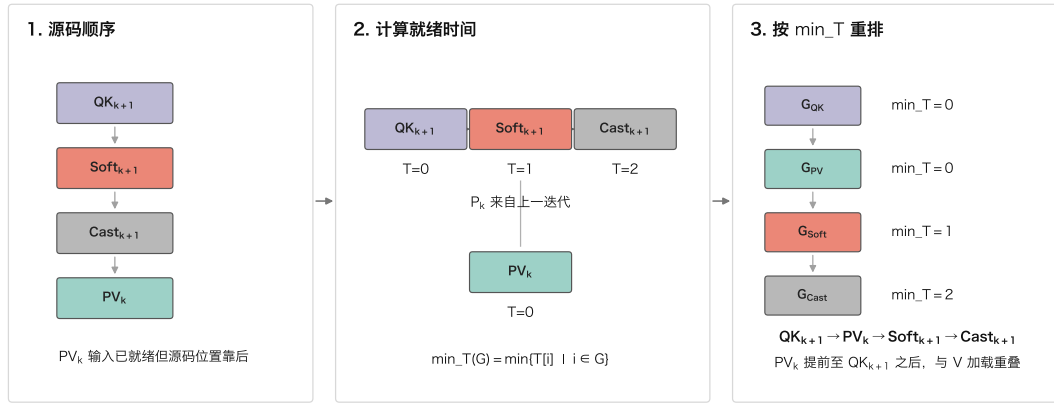


图 4-3 基于 $\min_T$ 的消费者顺序重排示例

Figure 4-3 Example of Consumer Order Refinement Based on $\min_T$

因此，各消费者组的排序键为

$$\min_T(G_{QK}) = 0, \quad \min_T(G_{PV}) = 0, \quad \min_T(G_{Soft}) = 1, \quad \min_T(G_{Cast}) = 2.$$

按 $\min_T$ 升序排列，并对相同 $\min_T$ 的组沿用源码位置作为稳定排序规则，可得到新的消费者顺序 $QK_{k+1} \rightarrow PV_k \rightarrow \text{Softmax}_{k+1} \rightarrow \text{Cast}_{k+1}$ ，对应组级 order 为 $[-1, 0, 2, 3, -1, 1]$ 。该顺序与第 4.1.3 节的阶段候选共同决定最终流水线配置。

算法 4 给出了从依赖图构建到 order 生成的完整流程。

AutoPiped 的 ASAP 顺序优化以依赖图的最早就绪层级为排序依据。第 5 章的 Heddle 在此基础上加入 ALAP 松弛量和资源峰值检查，用于寄存器压力更高的消费者重排场景。

完成注解推断后，AutoPiped 将 `tl_pipeline_group`、`tl_pipeline_order`、`tl_pipeline_stage` 写入循环的 IR 注解，交由 TileLang 后端完成底层实现。WarpSpecializedRewriter 根据注解将循环体拆分为生产者和消费者两类线程束组，分别负责异步数据搬

算法 4 ASAP 执行顺序优化流程**Algorithm 4** ASAP-Based Execution Order Optimization**输入：** 语句序列 S ，分组列表 $\mathcal{G}$ ，读写区域 $\{(\mathcal{R}_i, \mathcal{W}_i)\}$ **输出：** 优化后的 order 列表

阶段一：按程序顺序初始化执行顺序

```

1:  $order\_counter \leftarrow 0$ 
2: for all  $G_j \in \mathcal{G}$  do
3:   if  $G_j$  为生产者组 then
4:      $order[j] \leftarrow -1$ 
5:   else
6:      $order[j] \leftarrow order\_counter$ 
7:      $order\_counter \leftarrow order\_counter + 1$ 
8:   end if
9: end for

```

阶段二：构建依赖图

10: $E_1 \leftarrow \{(s_{i-1} \rightarrow s_i) \mid \text{MustPreserveOrder}(s_{i-1}, s_i)\}$

▷ 必须保留的程序顺序边

11: $E_2 \leftarrow \{(s_i \rightarrow s_j) \mid i < j, \exists b \in \mathcal{R}_j \cap \mathcal{W}_i, \text{IsShared}(b)\}$

▷ 共享内存写后读依赖边

阶段三：计算拓扑层级

```

12: for all  $s_i \in S$  do
13:    $T[i] \leftarrow 0$ 
14: end for
15: repeat
16:   for all  $(s_u \rightarrow s_v) \in E_1 \cup E_2$  do
17:      $T[v] \leftarrow \max(T[v], T[u] + 1)$ 
18:   end for
19: until  $T$  不再变化

```

阶段四：按组级排序键重排消费者组

```

20: 消费者组集合  $C \leftarrow \{G_j \mid order[j] \geq 0\}$ 
21: for all  $G_j \in C$  do
22:    $\min\_T(G_j) \leftarrow \min\{T[i] \mid i \in G_j\}$ 
23: end for
24: 将  $C$  中各组按  $\min\_T$  升序排列，依次赋  $order = 0, 1, 2, \dots$ 
25: if 新 order 与程序顺序一致 then
26:   return 程序顺序
27: end if
28: return 优化后的  $order$ 

```

运与矩阵计算。InjectSoftwarePipeline 执行环形缓冲区扩展，将每个共享内存缓冲区的访问索引改写为按阶段轮转的地址；阶段索引为 $\text{iter} \bmod \text{num_stages}$ ，最终地址由缓冲区起始地址、阶段索引乘以单阶段缓冲区大小、以及原始偏移三部分相加得到。随后，后端在相应位置注入 `mbarrier_expect_tx`、`mbarrier_arrive`、`mbarrier_wait` 等 Hopper 硬件同步原语，为软件流水的环形缓冲建立逐缓冲区的同步关系。PipelinePlanning 在编译期静态验证 arrive/wait 的配对合法性，在运行前捕获可能导致死锁的非法配置。线程束特化场景中原有全局生产者-消费者边界的进一步细化由第 5 章的 FGWS 机制完成。

4.3 FlashAttention 前向计算案例分析

本节以非因果 FlashAttention 前向算子为例，分析指令分组、执行顺序和流水线阶段对 Hopper 异步流水线结构的影响。对比对象为 TileLang 默认双缓冲配置与 AutoPiped 生成的三缓冲配置，重点观察二者在稳态重复片段中的调度差异。

FlashAttention 循环体可抽象为六个任务组，映射到 Hopper 的三类执行单元。图 4-4 给出了 k 循环的精简伪代码及其硬件映射。Load K 和 Load V 属于 TMA 侧生产者；QK 与 PV 两个 GEMM 属于 Tensor Core；掩码、Softmax 递推和类型转换等操作由 CUDA Core 及 SFU 执行。加载、张量计算和标量归约分别占用不同硬件通道，为跨迭代重叠提供调度空间。

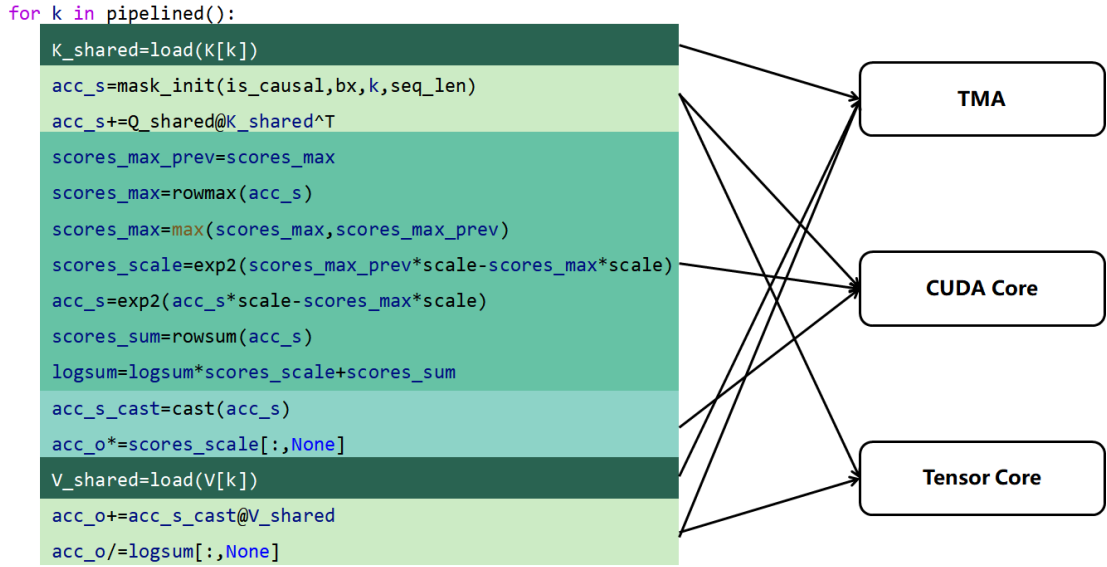


图 4-4 FlashAttention 前向主循环的伪代码与硬件映射

Figure 4-4 Pseudocode and Hardware Mapping of FlashAttention Forward Main Loop

流水线阶段注解与缓冲级数决定环形缓冲区的版本映射和生产者侧复用距离，执行顺序注解决定同一次主循环重复片段中各组的发射先后，两者共同决定稳态下的交错结构。表 4-3 对比了 TileLang 默认配置与三缓冲优化配置在稳态主循环中的执行流。

图 4-5 给出三个配置的生命周期对比。图中包含 TMA、Tensor Core 和 CUDA Core 与 SFU 三条通道；横轴表示逻辑时间，同一通道内的色块按从左到右顺序发射。下标表示被处理的 K 和 V 分块编号。红色虚线框标为反压等待，对应生产者侧的 `mbarrier[bp].wait(phase)`，表示 TMA 覆盖某个环形缓冲区版本前需要等待该版本被消费者释放。

图中三种配置分别用于分离不同因素的影响。图 4-5(a) 是搜索得到的三缓冲优化配置，同时采用 `order=[-1,0,2,3,-1,1]` 和 `stage=[-1,1,1,1,-1,2]`。图 4-5(b) 保持相同 `order`，但改回双缓冲阶段 `stage=[-1,0,0,0,-1,1]`，用于单独观察阶段和

表 4-3 稳态主循环执行流对比

Table 4-3 Steady-State Main Loop Execution Flow Comparison

TileLang 默认配置, 双缓冲						三缓冲优化配置					
order=[-1,0,3,1,-1,2], stage=[-1,0,0,1,-1,1], num_stages=2						order=[-1,0,2,3,-1,1], stage=[-1,1,1,1,-1,2], num_stages=3					
Order	Group	单元	操作内容	数据	调度影响	Order	Group	单元	操作内容	数据	调度作用
-1	G0/G4	TMA	Load K, V	$k+2$	双缓冲下缓冲区版本更早复用, TMA 需等待旧版本完成读取	-1	G0/G4	TMA	Load K, V	$k+2$	三缓冲提供额外缓冲区版本, 推迟反压等待出现的位置
0	G1	TC	QK^T GEMM	$k+1$	启动下一块 QK, 后续 PV 仍需使用同一 TC 通道	0	G1	TC	QK^T GEMM	$k+1$	启动下一块 QK; 为 CC 侧 Softmax 提供输入
1	G3	CC	Cast	k	占用 CC 通道, 使下一块 Softmax 后移	1	G5	TC	PV GEMM	k	紧跟 QK 发射, 提高 TC 通道的连续性
2	G5	TC	PV GEMM	k	与 QK 共用 TC 通道, 双缓冲下等待链更长	2	G2	CC	Softmax 与 Rescale	$k+1$	与 $PV(k)$ 分属不同功能单元, 可并行推进
3	G2	CC	Softmax 与 Rescale	$k+1$	排在序列末端, QK 结果就绪后仍需等待前序组完成	3	G3	CC	Cast	$k+1$	生成 $P(k+1)$ 的低精度版本, 为下一轮 $PV(k+1)$ 准备输入

缓冲级数的影响。图 4-5(c) 是 TileLang 默认配置, 采用 $\text{order}=[-1,0,3,1,-1,2]$ 和 $\text{stage}=[-1,0,0,1,-1,1]$, 用于展示默认顺序下 CUDA Core/SFU 通道的错位。

生成代码进一步说明了三缓冲收益的来源。三缓冲配置与双缓冲同顺序配置的消费者主体顺序相同, 差别在于环形缓冲区的复用距离。双缓冲只有两个 K/V 槽位, $k=0$ 和 $k=1$ 可以连续加载, 到 $k=2$ 就需要复用槽位 0; 三缓冲有三个槽位, $k=0,1,2$ 可以连续加载, 直到 $k=3$ 才第一次复用槽位 0。生产者加载 K 和 V 前会分别等待对应槽位释放, 其中 K 槽位在 QK 之后即可释放, V 槽位需要等 PV 完成后才能释放, 因此 V 更容易形成反压。三缓冲多出的一组槽位把这条等待链向后推迟一轮, 使 TMA 更容易持续预取后续分块。

TileLang 默认配置同样采用双缓冲, 但消费者顺序不同, Cast 被排在下一块 Softmax 之前, 使 CUDA Core/SFU 通道的 Softmax 启动后移。三种配置的对比说明, stage 与 num_stages 共同影响生产者的预取距离和环形缓冲区复用压力, order 决定 Tensor Core 与 CUDA Core/SFU 上的发射交错。

这里的跨迭代重叠仍发生在单个消费者 WarpGroup 内。AutoPiped 通过 group、stage 和 order 注解形成 QK_{k+1} 、 Softmax_{k+1} 与 PV_k 的错位关系; 第 5 章的双消费者模式沿用这一错位关系, 把 QK 与 Softmax 链和 PV 链映射到两个消费者 WarpGroup, 并用共享内存中转概率块。AutoPiped 确定循环体可表达的跨迭代交错关系, Heddle 进一步决定该关系在 WarpGroup 空间维上的执行方式。

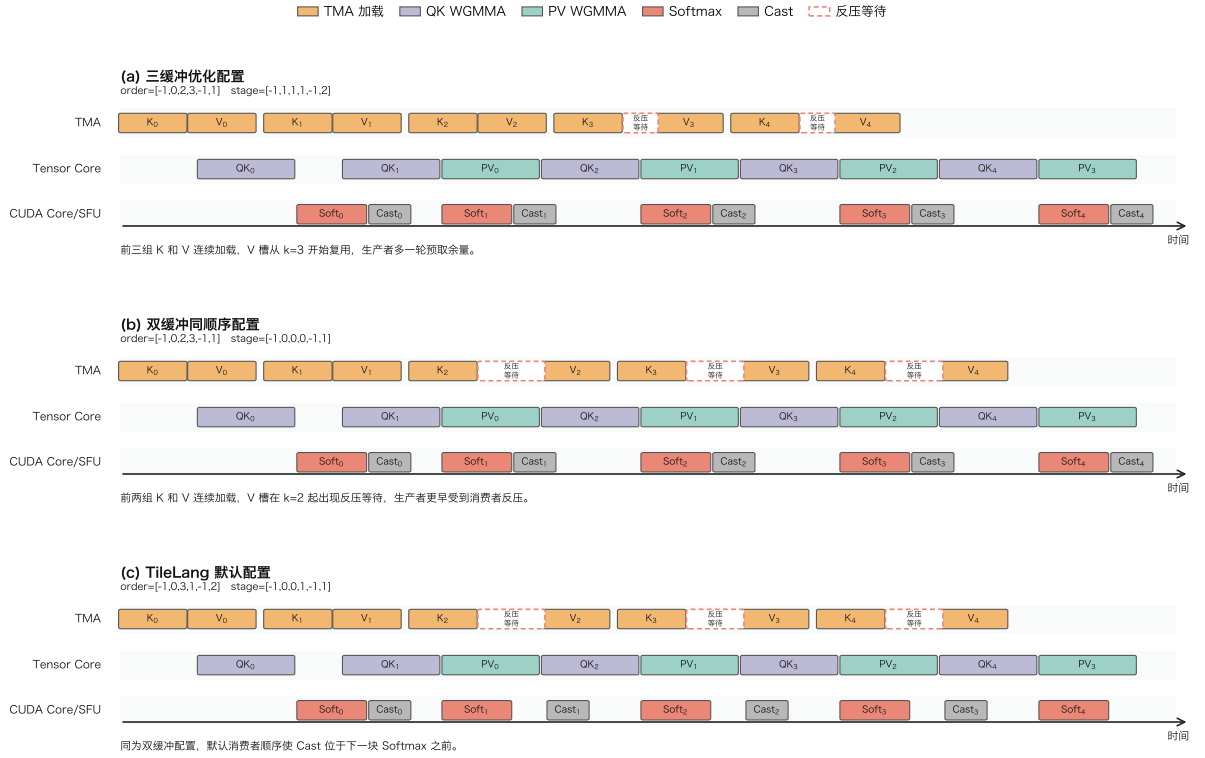


图 4-5 FlashAttention 前向三类流水线配置生命周期对比

Figure 4-5 Lifecycle Comparison of Three Pipeline Configurations for FlashAttention Forward

4.4 本章小结

本章将 Hopper 异步流水线三类配置参数的生成, 从手动填写改为编译期分析。读写区域分析提取每条语句的访存依赖, 角色判定识别生产者与消费者, 共享缓冲区的读写关系确定各消费者的流水线阶段, 结构规则选择分组策略, ASAP 顺序优化在依赖允许的范围内寻找改善稳态重叠的执行顺序。FlashAttention 前向案例表明, group、order 和 stage 注解能够表达三缓冲调度及其跨迭代重叠结构。AutoPiped 的输出是循环级流水线注解, 后端据此完成环形缓冲区展开、mbarrier 注入和代码生成; 后续 Heddle 在这些注解基础上继续处理缓冲区级屏障、消费者调度和资源上限检查。

第5章 Heddle：面向细粒度线程束特化的自动流水线调度框架

如第1.2节所述，Hopper架构上的高性能实现同时依赖时间维的软件流水线和空间维的线程束特化。软件流水线（Software Pipelining, SWP）通过跨迭代重叠访存与计算提升吞吐，线程束特化（Warp Specialization, WS）通过生产者-消费者角色分离组织执行流。二者在资源层面紧密耦合，减小启动间隔 II^1 会拉长中间值活跃区间，将计算集中到少数消费者 WarpGroup 又会放大局部寄存器压力。因此，流水线阶段、消费者顺序、同步位置和资源峰值需要在同一内核上下文协同处理。

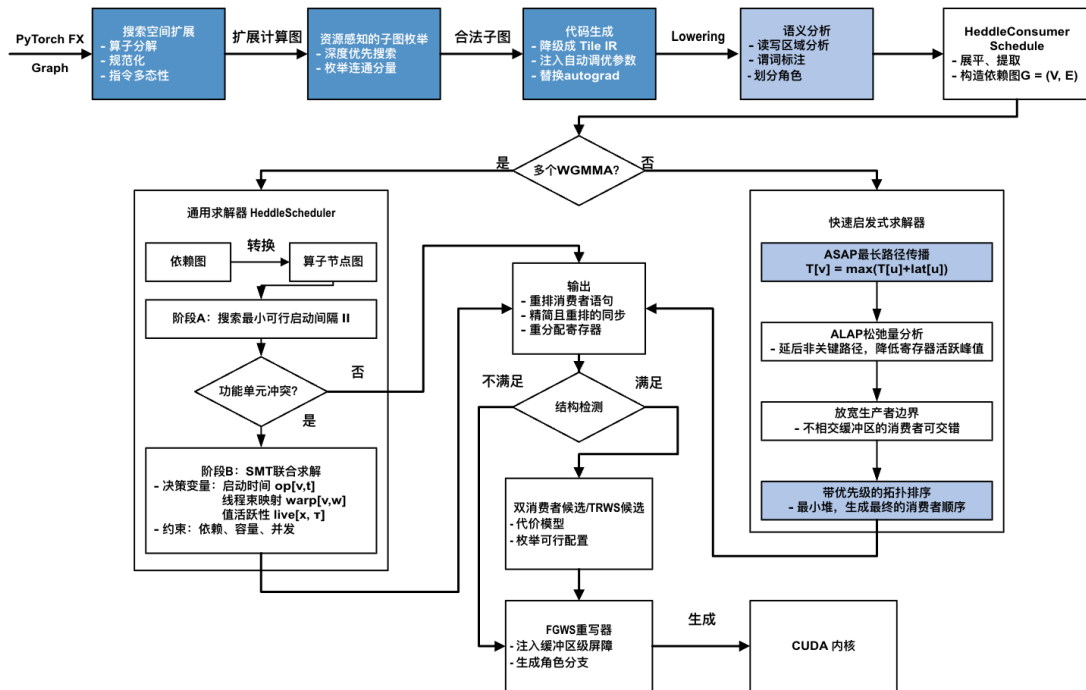


图 5-1 Heddle 模块总览

Figure 5-1 Overview of the Heddle Module

在 AutoMixed 确定内核结构、AutoPiped 生成基础流水线注解之后，Heddle 进一步处理内核内部的细粒度线程束特化调度。其核心流程分为三层。执行层的细粒度线程束特化（Fine-Grained Warp Specialization, FGWS）将生产者-消费者同步细化到共享缓冲区版本；HeddleConsumerSchedule 在消费者依赖图上生成语句顺序和屏障位置提示，并由 HeddleScheduler 检查 Tensor Core 发射、寄存器峰值和共享内存占用；多角色扩展把同一套同步与调度信息用于双消费者 WarpGroup 和三角角色线程束特化（Three-Role Warp Specialization, TRWS）。图 5-1 给出了 Heddle 的整体流程。上方蓝色流程为从 PyTorch 计算图到 Tile IR 的前端

¹ II 表示相邻两次循环迭代的启动间隔。

处理，左侧为 CP-SAT 约束调度路径，右侧为启发式快速路径，菱形节点表示超时回退判定。

Heddle 以 TileLang 的 Tile IR、TMA/WGMMA 原语、环形缓冲区和代码生成流程为承载层，从循环体中提取异步生产者块、消费者读写位置和缓冲区生命周期，并把这些信息转化为缓冲区级屏障、消费者重排和多 WarpGroup 协同执行的后端注解。

5.1 原有线程束特化机制的结构约束

TileLang 的线程束特化后端已经提供生产者-消费者分支生成、必要语句复制和寄存器重分配等基础能力。Heddle 在此基础上面向复杂异步流水线补充三类内核内调度能力。

约束一，全局生产者-消费者边界。阶段级同步会把同一阶段内的多个生产者合并到一个等待点。以 FlashAttention 前向为例，QK GEMM 只需要 K 分块即可启动，V 分块只在后续 PV GEMM 中使用；若 K 与 V 的 TMA 加载共享同一个等待点，QK 需要等待 V 加载完成，V 加载与 QK 计算的重叠窗口被压缩。

约束二，消费者执行顺序固定。源码顺序会同时固定消费者排列和屏障插入位置。每个共享缓冲区的首次读取位置决定前向等待点，末次访问位置决定反压释放点；消费者排列改变后，跨迭代重叠窗口也随之改变。如第 4 章案例分析所示，同一 FlashAttention 前向内核在不同消费者排列下的性能差距可达 20%。

约束三，资源峰值集中。阶段级边界与固定消费者顺序会把部分寄存器和共享内存活跃区间集中到同一段循环体。在 FlashAttention 反向与 Grouped GEMM 等资源密集型算子上，这会压缩可编译配置空间。表 5-1 给出了阶段级 WS 与 FGWS 的编译通过率对比。

后续小节按这三类问题展开。FGWS 建立缓冲区级同步协议，HeddleConsumerSchedule 调整消费者顺序与屏障位置，HeddleScheduler 检查功能单元发射、寄存器和共享内存等资源上限。

表 5-1 原有 WS 与 FGWS 的编译通过率对比

Table 5-1 Compilation Success Rate: Legacy WS vs FGWS

算子	配置范围	原 WS 通过率	FGWS 通过率
FlashAttention 前向	键值分块为 128; 流水线级数取 1、2、3	50.0%	100%
FlashAttention 反向	键值分块取 32 或 64; 流水线级数取 1 或 2	50.0%	100%
Grouped GEMM	流水线级数取 1、2、3	33.3%	100%

5.2 细粒度线程束特化机制

FGWS 是 Heddle 接入 TileLang WS 后端的执行层。它沿用 TileLang 的分支生成流程，并额外提取异步拷贝模式、消费者读写位置和缓冲区生命周期。FGWS 的设计包括异步拷贝模式提取、缓冲区级屏障布局、分支结构信息传递和条件执行下的相位计数器四个部分。

生产者-消费者分支生成需要知道语句归属，缓冲区级同步还需要知道每个共享缓冲区版本何时就绪、何时首次被读取、何时可以被覆盖。异步搬运通常由一组语句共同完成。一个 TMA 生产者包含反压等待、事务字节数预告、TMA 加载和数据到达通知；一个 cp.async 生产者包含拷贝、提交和等待组操作。消费者侧也可能写共享内存临时结果。FGWS 因此在语句角色之外继续分析读写关系，提取缓冲区版本级的生命周期。

FGWS 在分支生成前增加异步拷贝模式提取。编译器先将流水化循环体展平成语句序列，再做一次线性扫描，识别 TMA 生产者块和 cp.async 生产者块。TMA 生产者块由反压等待、事务字节数预告、tma_load 以及数据到达通知构成；cp.async 生产者块由 ptx_cp_async、commit_group 和 wait_group(0) 等语句构成。每个生产者块记录写入的共享缓冲区、字节数、流水线阶段表达式以及对应的屏障组。其余语句作为消费者保留，并继续记录读写区域。经过这一步，循环体被表示为确定的生产者块列表和消费者语句列表，后续的屏障放置和消费者重排都基于这两个列表进行。

在屏障协议上，FGWS 为每个生产者写入的共享缓冲区分配独立的前向等待屏障和反压释放屏障。前向等待屏障表示该缓冲区版本已经填充完毕，插在首次消费者读取之前；反压释放屏障表示该缓冲区版本已经不再被当前消费者使用，插在末次访问之后。消费者等待自身读取的缓冲区，生产者等待即将覆盖的缓冲区版本释放。因此，同一阶段内的 K 与 V 加载可以拥有不同的就绪边界，QK 在 K 就绪后即可启动，不必等待 V 生产者完成。

前向等待屏障又分为主循环 fwd 和预循环 preloop_fwd 两类。二者语义相同，都是数据就绪通知；区别在于数据由谁产生。主循环 fwd 对应稳态循环中的生产者加载，阶段和相位随循环计数轮转。预循环 preloop_fwd 对应循环开始前的预取数据，这些数据会被前几个消费者迭代读取，但其到达通知发生在主循环计数开始之前。拆分两类屏障后，预循环到达信号和主循环同阶段到达信号各自维护相位状态，前序、稳态和后序的 wait/arrive 配对更加直接。FGWS 在编译期对 mbarrier 编号空间做逻辑分段，按缓冲区 $\times$ 流水线级数展开，使每个共享缓冲区版本拥有独立的屏障。表 5-2 列出三类屏障段。

生产者在写入某个共享缓冲区版本前先等待对应 bp，确认该版本已被消费者释放；随后通过 expect_tx 预告传输字节数，发起 tma_load，并由 TMA 完成时的 arrive 更新对应 fwd。消费者在首次读取该版本前等待前向屏障，在末次访问后到达反压屏障。当同一循环中同时存在 TMA 和 cp.async 生产者时，FGWS 为 cp.async 独有的就绪信号分配额外前向屏障，并在首次读取和末次访问位置相同

表 5-2 FGWS 屏障编号空间的逻辑分段

Table 5-2 Logical Segments of the FGWS Barrier Numbering Space

屏障段	含义
fwd	前向就绪屏障。生产者完成 TMA 或 cp.async 填充后通知消费者数据已就绪，消费者在首次读取对应缓冲区版本前等待该屏障
bp	反压释放屏障。消费者在末次访问对应缓冲区版本后到达该屏障，通知生产者该版本可被覆盖写入
preloop_fwd	预循环就绪屏障。循环启动前的预取阶段使用，通知前几个消费者迭代所需的数据已经就绪

的生产者块之间合并屏障组，以减少 mbarrier 数量。

在 block-sparse 注意力等场景中，循环体被运行时条件包裹，被跳过的迭代不触发屏障到达，但标准相位标记仍随循环变量递增。FGWS 为此引入相位计数器，仅在循环体实际执行时递增，阶段索引与相位标记由下式导出，使等待和到达始终按实际执行次数配对。

$$\text{stage} = \text{counter} \bmod S, \quad \text{parity} = \lfloor \text{counter}/S \rfloor \bmod 2. \quad (5-1)$$

图 5-2 以 FlashAttention 前向为例对比了两种同步方式。全局屏障要求所有生产者完成后消费者才能启动，导致空闲等待；缓冲区级屏障允许 QK WGMMMA 在 K 就绪后立即启动，并与 V 的 TMA 加载并行执行。

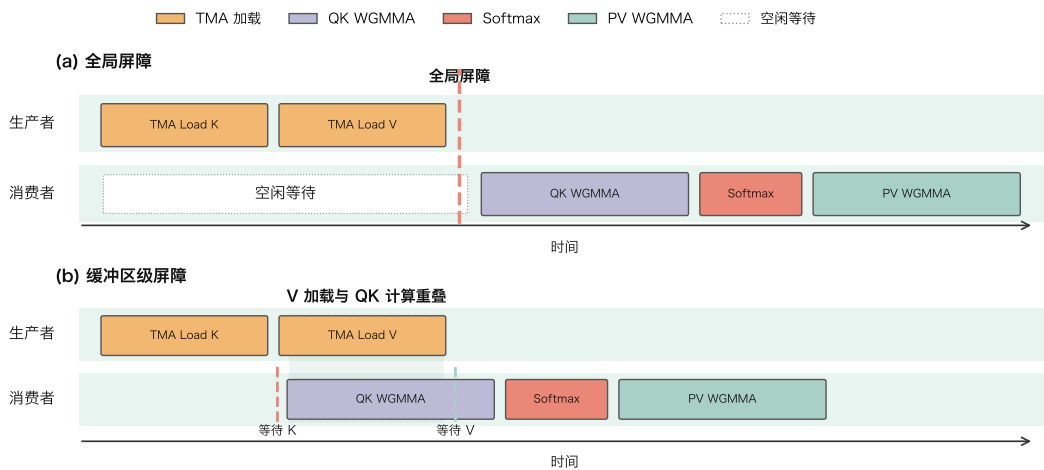


图 5-2 全局屏障与缓冲区级屏障的时序对比

Figure 5-2 Timing: Global Barriers vs Buffer-Level Barriers

除屏障编号外，FGWS 还在循环体分析阶段确定分支结构和缓冲区级同步位置。分支结构包括生产者语句、消费者语句，以及需要在两侧使用的地址计算或初始化语句；缓冲区级同步位置记录每个共享缓冲区版本的数据就绪等待点和可覆盖释放点。Heddlle 向后端传递生产者块写入的缓冲区、搬运字节数、首次读取位置、末次访问位置、重排后的消费者序列，以及异步 WGMMMA 等待深

度、双消费者切分点等扩展标记。TileLang 分支生成流程根据这些信息完成语句过滤、必要语句复制、寄存器重分配、前向等待和反压释放等具体 IR 重写。

5.3 时间维调度与消费者重排

FGWS 确定生产者侧协议后，消费者语句顺序直接决定等待屏障和反压释放屏障的位置。等待屏障插在首次读取之前，释放屏障插在末次访问之后；因此，消费者排列改变后，生产者与消费者之间的重叠窗口也会改变。

以 FlashAttention 前向为例，QK GEMM 只依赖 K，PV GEMM 只依赖 V。按 QK→PV 排列时，QK 可以在 K 就绪后启动，V 的 TMA 加载继续与 QK 计算重叠；若排列为 PV→QK，V 的等待点被提前到 QK 之前，Tensor Core 流水线会出现额外等待。第4章的流水线案例正是这一现象的具体表现。

HeddleConsumerSchedule 将消费者顺序纳入调度空间，同时保持生产者发起、屏障初始化和寄存器分配由 FGWS 的确定规则生成。表 5-3 列出了本文测试算子的消费者语句规模；这些循环体通常只有数条到二十余条消费者语句，适合使用启发式排序和 CP-SAT 约束求解结合的方式处理。

表 5-3 典型算子的消费者语句数量统计

Table 5-3 Consumer Statement Counts for Representative Operators

算子	消费者语句数	主要消费者语句
GEMM	3	一次 WGMMA，加上结果写回相关的 2 条拷贝语句
Grouped GEMM	5	WGMMA 加上批次索引计算、结果写回的辅助语句
Dequant GEMM	6	WGMMA、位解包、反量化拷贝及结果写回
Conv2D	8	WGMMA、im2col 索引重排、多条边界掩码与拷贝
FlashAttention 前向	12	2 次 WGMMA (QK、PV)，加 Softmax 的最大值、指数、求和、重缩放、掩码、类型转换等
FlashAttention 反向（分离式）	16	4 次 WGMMA (QK、dP、dV 累加、dK 累加)，加 Softmax 反向与 dS 的归一化
FlashAttention 反向（融合式）	22	分离式之外还包含 dQ 的原子累加路径

HeddleConsumerSchedule 的输入是带 num_stages 注解的流水化循环和硬件预算。它展平循环体，提取语句级读写区域、同步信息、生产者块和消费者语句，并在消费者子序列上构建依赖图。图中节点是消费者语句，边表示必须保持的先后关系；硬件预算包括 Tensor Core、TMA、CUDA Core/SFU 等功能单元上限，以及寄存器和共享内存活跃字节上限。调度结果包括消费者排列 π 、启动时刻估计 T_o ，以及每个共享缓冲区的等待/释放位置提示。

本章算法中的时间量均为调度器内部的离散相对时隙，而非精确硬件周期。 L 表示被流水化的循环体， C 表示消费者语句索引集合， $\mathcal{W}$ 表示可分配的 WarpGroup 集合；硬件预算中的寄存器以字节或寄存器/线程计，共享内存以字节计，功能单元容量表示同一相对时隙内允许的发射数量。

调度过程分为启发式快速路径和 CP-SAT 约束路径。启发式路径先在依赖图上计算 ASAP/ALAP 层级，再结合缓冲区末次读取位置和拓扑排序生成消费者顺序。对于独立 WGMMA 较多或资源峰值较高的候选，CP-SAT 路径在有限窗口内重新分配启动时刻；求解成功后按启动时刻排序，并把时间表转换为屏障提示。若求解超时或不可行，编译流程使用启发式结果继续生成代码。

底层 HeddleScheduler 提供 II 搜索和 CP-SAT 两类接口。前者通过 Bellman-Ford 最长路径松弛快速得到最早启动时间，并检查模相位上的功能单元占用；后者同时检查依赖下界、功能单元互斥和资源峰值。第 5.3.3 节给出建模细节。

5.3.1 启发式快速路径

启发式快速路径面向依赖结构清晰的消费者图。该路径在依赖传播和拓扑排序中加入资源压力相关的优先级，直接生成消费者顺序和屏障位置提示。

具体流程保持轻量。首先，调度器沿用第 4 章的 ASAP 传播，在语句粒度计算每个消费者的最早就绪层级；随后从汇点反向计算 ALAP 层级，用二者差值判断哪些语句可以后移。关键路径和同步语句保持 ASAP 优先级，非关键路径语句可适当后移，以减少中间值过早产生带来的寄存器压力。

其次，调度器根据共享缓冲区的末次读取位置修正优先级。能够释放缓冲区版本的消费者会被适当提前，使反压释放屏障更早出现。若相邻生产者写入的共享缓冲区集合不相交，对应的全局保序边可以省略，读取不同缓冲区的消费者因此可以交错排列。最后，调度器在修正后的依赖图上执行最小堆拓扑排序，得到消费者顺序，并据此提取每个缓冲区的首次读取和末次访问位置。

启发式路径在依赖链主导的场景下通常可在毫秒内完成。当多个操作竞争同一功能单元时，依赖合法的顺序仍可能超过 Tensor Core 发射上限，后续 CP-SAT 路径会重新安排启动时刻。

算法 5 综合展示了 HeddleConsumerSchedule 的完整流程，包括双消费者模式检测、启发式与 CP-SAT 两条路径的选择、以及屏障提示的提取。

5.3.2 异步 WGMMA 流水线优化

消费者重排通过调整语句顺序扩大屏障窗口，但即使在最优排列下，每次 WGMMA 的 `warpgroup_wait<0>` 仍会阻塞消费者直到累加器写入完成。WGMMA 的执行分为发射和累加两个阶段。发射阶段在数个周期内将共享内存操作数提交到张量核心内部缓冲区，累加阶段将结果写入寄存器累加器。累加阶段的延迟为几十个时钟周期，但期间不再访问共享内存。若将最后一个 WGMMA 的等待深度从 0 放宽到 1，其累加过程即可与下一迭代的屏障等待和初始化工作在时间

算法 5 HeddleConsumerSchedule 主流程**Algorithm 5** HeddleConsumerSchedule Main Flow**输入：** 带 num_stages 注解的流水化循环 L ；硬件预算与调度选项**输出：** 重排后的循环体及屏障提示注解

```

1:  $S \leftarrow$  展平  $L$  的循环体得到语句序列, 构造 info 数组标注角色、读写集与 WGMMA 标志
2:  $C \leftarrow \{i : \neg \text{info}_i.\text{producer}\}$  ▷ 消费者索引集合
3: if  $|C| \leq 1$  then
4:   return  $L$ 
5: end if
6:  $G_C = (C, E_C) \leftarrow$  基于读写集、同步语句和生产者边界构建消费者依赖图
7: 对相邻生产者写入缓冲区不相交的边界, 省略对应保序边
   步骤一: 双消费者模式检测
8:  $\mathcal{W} \leftarrow$  消费者中 WGMMA 语句的索引
9: if  $|\mathcal{W}| \geq 2 \wedge \mathcal{W}$  首尾之间存在非 WGMMA 消费者 then
10:   在每个 WGMMA 边界之后枚举候选分割点  $p$ , 按式 (5-4) 估算  $T_{\text{dual}}$  与  $T_{\text{single}}$ 
11:    $p^* \leftarrow \arg \min_p T_{\text{dual}}(p)$ 
12:   if  $T_{\text{dual}}(p^*) < T_{\text{single}}$  then 标注双消费者模式并记录分割点  $p^*$ 
13:   end if
14: end if
   步骤二: 调度路径选择
15:  $n_{\text{tc}} \leftarrow |\mathcal{W}|$ ;  $\text{run\_stage2} \leftarrow \text{use\_stage2} \vee (n_{\text{tc}} \geq 3 \wedge |C| \geq 6)$ 
16: solved  $\leftarrow$  False
17: if run_stage2 then
18:    $T_{\text{max}} \leftarrow \min(500 + 100|C|, 15000)$  ms ▷ 自适应超时
19:    $(\pi, \{T_v\}) \leftarrow \text{STAGE2JOINTSOLVE}(G_C, \text{硬件预算}, T_{\text{max}})$ 
20:   if 求解成功 then
21:     hints  $\leftarrow$  从  $\{T_v\}$  提取缓冲区等待/释放位置
22:     solved  $\leftarrow$  True
23:   end if
24: end if
25: if  $\neg$ solved then
   步骤三: 启发式消费者重排
26:    $\{T_v^{\text{ASAP}}\} \leftarrow \text{BELLMANFORDASAP}(G_C)$ 
27:    $\{T_v^{\text{ALAP}}\} \leftarrow \text{BELLMANFORDALAP}(G_C)$ 
28:   为每条消费者  $v$  计算  $\text{slack}_v = T_v^{\text{ALAP}} - T_v^{\text{ASAP}}$ 
29:   for all  $v \in C$  do
30:     if  $v$  为同步语句或  $\text{slack}_v = 0$  then  $\text{priority}[v] \leftarrow T_v^{\text{ASAP}}$  ▷ 关键路径与同步语句保持 ASAP
31:     else  $\text{priority}[v] \leftarrow T_v^{\text{ALAP}}$  ▷ 非关键路径延后
32:   end if
33: end for
34: 针对每个生产者缓冲区定位末次访问者, 调整优先级以提前释放对应缓冲区版本
35: 放宽写入缓冲区集合不相交的生产者边界约束
36:  $\pi \leftarrow \text{REGAWARETOPOSORT}(C, E_C, \text{priority}, \text{reads})$  ▷ 同优先级优先调度末次访问者, 降低活跃区间峰值
   值
37:   hints  $\leftarrow$  按  $\pi$  推导缓冲区等待/释放位置
38: end if
   步骤四: 循环体重写
39: 按  $\pi$  重排  $L$  循环体中的消费者语句; 将 hints 注入为缓冲区级屏障位置提示
40: if 检测到 TMA 规约加模式 then
41:   注入三角色线程束特化标记
42: end if
43: return 重排后的循环  $L'$ 

```

上重叠。

Heddle 实现了两项协同优化。异步等待将循环体中最后一个 `warpgroup_wait<0>` 替换为 `warpgroup_wait<1>`，同时在循环结束后插入一条 `warpgroup_wait<0>`，保护尾部读取累加器的正确性。提前释放将反压屏障的到达操作从等待之后移到提交之后、等待之前，利用 WGMMA 发射阶段已完成共享内存读取这一硬件特性，提前释放对应缓冲区版本，使生产者的 TMA 加载提前数十个时钟周期开始。

两项优化的收益来源互补。异步等待缩短了 T_{consumer} ，节省量等于最后一个 WGMMA 的累加延迟与下一迭代首部可重叠工作量的较小值。提前释放缩短了 T_{producer} ，节省量等于累加延迟与 TMA 加载延迟的较小值。两者同时作用于 $\max(T_{\text{producer}}, T_{\text{consumer}})$ 的两侧，互补地缩短稳态迭代周期。

正确性依赖三点。对于多 WGMMA 算子，下一个 WGMMA 的 `warpgroup_wait<0>` 在读取累加器之前执行，此时前一迭代的异步 WGMMA 已完成。对于单 WGMMA 算子，硬件流水线自动序列化同一 WarpGroup 的连续 WGMMA 提交，后一迭代不会在前一迭代完成前发射。共享内存释放由 WGMMA 的发射语义保证，`commit_batch` 返回时共享内存操作数已被加载到张量核心的内部缓冲区。

图 5-3 对比了基线与 Heddle 调度的时序。图中的横向长度表示对稳态流水线的阻塞时间，不表示等待指令本身的编码长度。基线的 `wait<0>` 必须等 PV WGMMA 累加完成后返回，因此画成较长的阻塞段；Heddle 的 `wait<1>` 允许最后一个 WGMMA 保持未完成状态，只作为非阻塞等待标记出现。PV 的累加过程在图中单独表示，并与迭代 $k+1$ 的屏障等待、初始化和后续 QK 发射准备重叠。下方标注的可重叠窗口对应 Heddle 相比基线提前进入下一迭代首部工作的时间。

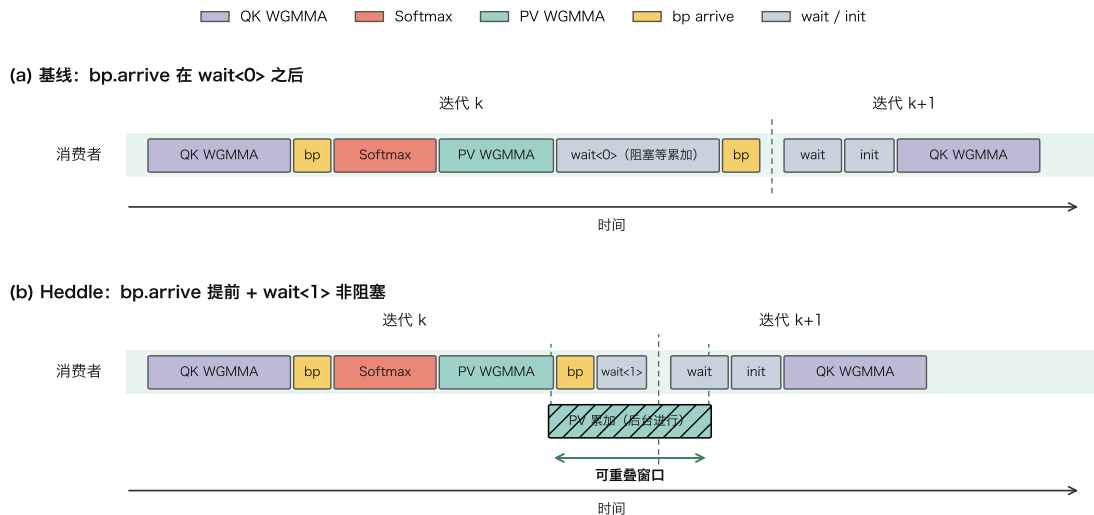


图 5-3 基线 `wait<0>` 与 Heddle `wait<1>` 的时序对比

Figure 5-3 Async WGMMA Scheduling Timeline, Baseline `wait<0>` vs Heddle `wait<1>`

这两项优化适用于单 WGMMA 的 GEMM，也适用于多 WGMMA 的 FlashAttention。第 6 章将其作为独立的 PTX 级变换统计。

无论来自启发式路径还是 CP-SAT 路径, HeddleConsumerSchedule 都输出同一种后端接口, 包括重排后的消费者语句序列、缓冲区等待/释放位置提示, 以及可选的双消费者或三角色 WS 标记。FGWS 重写器据此替换消费者部分, 并在相应位置插入屏障操作。

HeddleScheduler 统一记录调度时间与资源统计。多 Warp 映射变量 $\text{warp}[v, w]$ 在求解器中已实现; 单消费者 WarpGroup 场景下, 所有消费者语句运行在同一 WarpGroup 上, 该变量取固定值。双消费者和 TRWS 的角色划分由结构检测与代价模型确定, 求解器在给定候选结构下检查启动时刻和资源上限。 $\text{warp}[v, w]$ 变量使多角色候选能够沿用同一套资源约束表达。

5.3.3 底层求解器 HeddleScheduler

FGWS 确定生产者加载和缓冲区屏障后, 消费者侧仍有一个核心问题, 即哪些消费者语句先执行, 哪些语句可以错开到后面, 错开之后是否会超过 Tensor Core、寄存器或共享内存上限。HeddleScheduler 处理这一内核内时间表问题。它以已经流水化的单个内核和消费者依赖图为输入, 输出每条消费者语句的启动时刻、资源峰值统计, 以及可转换为消费者顺序和屏障位置的调度结果。

HeddleScheduler 的直接目标是找到一张资源可行的消费者时间表。时间表首先要满足数据依赖, 例如 Softmax 必须等 QK 结果就绪; 其次要满足功能单元容量, 例如同一个相对启动位置不能发射超过硬件能力的 WGMMA; 最后还要满足资源峰值, 例如中间值活跃区间叠加后不能超过寄存器和共享内存预算。

5.3.3.1 问题建模

本文将消费者子序列抽象为带属性依赖图 $G = (V, E)$ 。节点 $v \in V$ 表示一条消费者操作, 例如 WGMMA、Softmax 或类型转换; 边 $e = (u, v)$ 表示 v 必须等待 u 产生的值。调度器使用离散的相对启动位置作为时间单位, 该位置用于表达操作先后关系和功能单元占用, 不等同于精确硬件周期。

每条依赖边携带两个参数。 d_e 是边 e 的最小启动间隔, 即 v 相对 u 至少要晚多少个调度位置启动。普通数据依赖下, d_e 通常取生产者 u 的建模延迟; 若跨 WarpGroup 传值或阻塞同步带来额外等待, 则在该边上加入对应代价。 δ_e 是迭代距离, $\delta_e = 0$ 表示同一迭代内的依赖, $\delta_e = 1$ 表示下一次迭代中的消费者读取当前迭代产生的值。每个节点还携带资源预留信息, 包括使用的功能单元、持续时间和产生值的字节数。

图 5-4 以 FlashAttention 反向 dK/dV 为例展示建模过程。消费者语句先转换为依赖图和资源预留表; 快速传播可以得到依赖合法的 ASAP 排布, 但独立 WGMMA 可能落在同一 Tensor Core 发射位置; CP-SAT 路径在同一依赖图上加入功能单元和活跃区间约束, 重新分配启动时刻, 得到资源可行的排布。

模型的核心变量是启动时刻 T_v 和中间值活跃标记 $\text{live}[x, \tau]$ 。 T_v 决定操作 v 在调度窗口中的相对位置, 取值范围为 $[0, L)$; $\text{live}[x, \tau]$ 表示中间值 x 在位置 τ

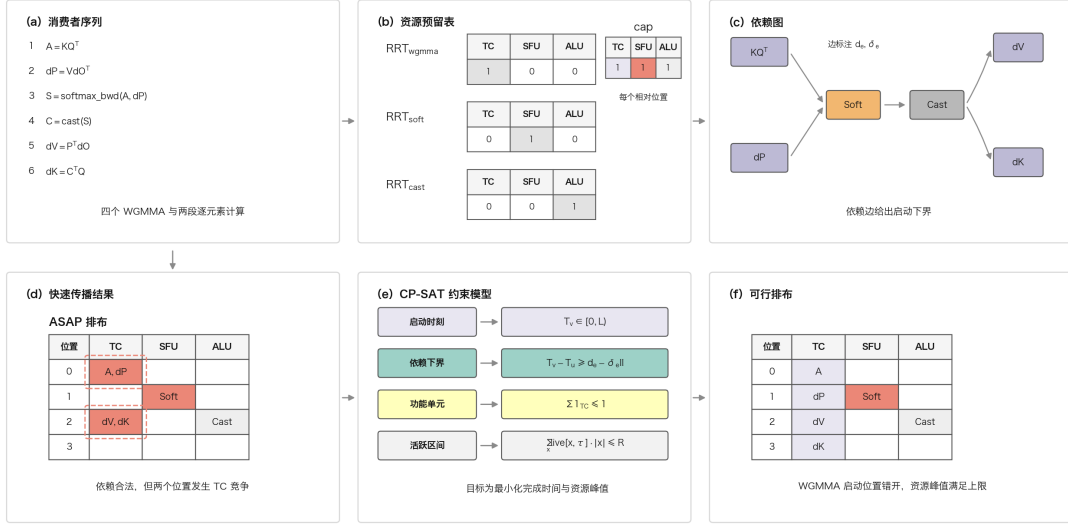


图 5-4 HeddleScheduler 约束建模示例

Figure 5-4 HeddleScheduler Constraint Modeling Example

是否仍占用寄存器或共享内存，其中 τ 也是窗口内相对位置。当依赖跨出当前调度窗口时，模型用 `incoming_live` 记录窗口外流入值对活跃区间的贡献。一般化接口还保留可选的 `WarpGroup` 分配变量 `warp[v, w]`，用于检查双消费者和 TRWS 等多角色候选。CP-SAT 约束路径的求解后端采用 OR-Tools CP-SAT，求解超时时间 $T_{\max}$ 的单位为毫秒。

功能单元约束使用资源预留表描述。设资源类型集合为 $\mathcal{R}$ ，包括 Tensor Core、TMA、CUDA Core/ALU 和 SFU 等。 $\text{cap}[r]$ 表示资源 $r \in \mathcal{R}$ 在同一相对位置上允许的最大占用量。例如 $\text{cap}[\text{TensorCore}] = 1$ 表示同一相对位置至多发射一个 WGMMMA。对操作 v ， C_v 表示其资源预留表长度，单位是离散时隙； $R_v[c, r]$ 表示 v 从启动位置起第 c 个偏移位置对资源 r 的占用量。多数消费者操作只在一个偏移位置上占用一个资源；持续多步的操作可以用更长的预留表表示。

依赖约束沿用标准模调度形式。对边 $e = (u, v)$ ， δ_e 表示从第 i 次迭代的 u 到第 $i + \delta_e$ 次迭代的 v 的距离，依赖下界为

$$T_v - T_u \geq d_e - \delta_e \cdot II, \quad \forall e = (u, v) \in E. \quad (5-2)$$

当 $\delta_e = 0$ 时，该式退化为同一迭代内的普通依赖 $T_v \geq T_u + d_e$ 。当 $\delta_e > 0$ 时，依赖跨到后续迭代；模调度把后续迭代折叠回长度为 II 的窗口，因此右端需要减去 $\delta_e \cdot II$ 。这一写法使同一套约束既能表达单次迭代内的消费者重排，也能表达跨迭代流水线中的依赖关系。除依赖约束外，模型还检查功能单元占用和资源峰值。功能单元约束要求同一相对位置上的资源使用量不超过 $\text{cap}[r]$ ；资源峰值约束通过 `live[x,  $\tau$ ]` 统计中间值活跃区间，要求寄存器活跃字节不超过 `RegLimit`，共享内存活跃字节不超过 `CapSMEM`。

5.3.3.2 两阶段求解

HeddleScheduler 采用两阶段求解。第一阶段用 Bellman-Ford 快速计算 ASAP 时刻并检查功能单元占用；第二阶段由 CP-SAT 求解器在有限窗口内重新分配启动时刻，并同步检查寄存器峰值。

第一阶段，Bellman-Ford ASAP 与功能单元检查。 第4章把每条依赖边视为单位延迟。Hopper 算子中 TMA、WGMMA 和标量计算的延迟差异明显，因此这里使用边延迟 d_e 和迭代距离 δ_e ，传播式写为 $T_v \leftarrow \max(T_v, T_u + d_e - II \cdot \delta_e)$ 。搜索器从 $II=1$ 开始递增，在每个候选值上同时检查依赖时序和功能单元占用，其中功能单元占用约束为

$$\sum_{v \in V} \sum_{c=0}^{C_v-1} R_v[c, r] \cdot \mathbb{1}[(T_v + c) \bmod II = m] \leq \text{cap}[r], \quad \forall r, \forall m \in [0, II-1]. \quad (5-3)$$

式中 r 表示资源类型，例如 Tensor Core、SFU 或 CUDA Core； m 表示长度为 II 的重复窗口中的相对位置； $\text{cap}[r]$ 是资源 r 在同一相对位置上允许的最大占用量。若两个 WGMMA 被排到同一模相位且超过 Tensor Core 发射能力，该式判定该 II 不可用。传播复杂度为 $O(|V| \cdot |E|)$ ，在 10 到 20 个节点的消费者图上几乎瞬间完成。当第一阶段找到可行 II 且无功能单元冲突时，其 ASAP 时刻直接作为调度结果；当第一阶段检测到冲突时，进入第二阶段。

第二阶段，CP-SAT 约束求解。 第二阶段在第一阶段确定的 II 附近，对每个候选 (II, L) 建立有限窗口约束模型，为每个操作分配启动时刻 $T_v \in [0, L)$ 。消费者重排路径中的依赖均位于同一迭代内，此时 $\delta_e = 0$ ，依赖约束写为 $T_v \geq T_u + d_e$ ；进入模调度接口时则使用式 (5-2) 的一般形式。求解目标为最小化完成时间。算法 6 和算法 7 分别给出两阶段流程。

当求解成功时，HeddleScheduler 的原生输出如表 5-4 所示。其核心结果是窗口内的启动时间、一般化接口下的可选 Warp 映射以及资源峰值统计。

基于这些启动时间，HeddleConsumerSchedule 按 T_v 排序得到消费者排列，并结合读写集定位每个共享缓冲区的首次读取者和末次访问者。首次读取者之前插入前向等待屏障，末次访问者之后插入反压释放屏障。求解器输出的时间表最终被转换为后端使用的循环体顺序和屏障提示。

CP-SAT 路径把依赖、资源上限和 WarpGroup 分工放在同一模型中检查。求解器返回可行解时，输出时间表在所采用的建模假设下满足全部约束；随后后端编译与运行测量继续验证寄存器分配、指令选择和真实吞吐。

启发式路径和 CP-SAT 路径最终生成同一形式的提示，下游 FGWS 重写器按统一格式使用这些提示。

算法 6 第一阶段最小启动间隔搜索**Algorithm 6** Stage 1: Minimum Initiation Interval Search

输入： 依赖图 $G = (V, E)$ ； 各类功能单元容量 $\text{cap}[\cdot]$

输出： 最小可行启动间隔 II^* 及对应的 ASAP 时刻，或 INFEASIBLE

```

1: for  $II \leftarrow 1$  to  $\text{MAX\_II}$  do
2:   for all  $v \in V$  do
3:      $T_v \leftarrow 0$ 
4:   end for
5:   for  $iter \leftarrow 1$  to  $|V|$  do
6:     for all  $e = (u, v) \in E$  do
7:        $T_v \leftarrow \max(T_v, T_u + d_e - II \cdot \delta_e)$ 
8:     end for
9:   end for
10:  if 存在边不满足  $T_v - T_u \geq d_e - II \cdot \delta_e$  then
11:    continue
12:  else if 每个资源  $r$  在每个相对位置  $m$  上的占用均满足  $\text{cap}[r]$  then
13:    return  $II, \{T_v\}_{v \in V}$ 
14:  end if
15: end for
16: return INFEASIBLE

```

5.3.4 求解器能力验证

本节用两个注意力算子的消费者图说明两条路径的分工。两条路径使用同一依赖图和同一套资源描述。启发式路径先通过依赖传播和拓扑排序生成低开销调度顺序，并从该顺序中提取屏障位置；当该顺序超过功能单元上限或活跃数据量上限时，CP-SAT 路径把每条消费者语句的启动时刻作为变量，在依赖下

算法 7 第二阶段时间-空间-资源联合约束求解**Algorithm 7** Stage 2: Joint Time-Space-Resource Constraint Solving

输入： $G = (V, E)$ ； 候选区间 $[II_{\min}, II_{\max}]$ ； 窗口集合 $\mathcal{L}$ ； WarpGroup 集合 $\mathcal{W}$ ； 硬件预算 $(\text{RegLimit}, \text{Cap}_{\text{MEM}}, \text{cap}[\cdot])$

输出： 调度结果、WarpGroup 分配、资源峰值和屏障提示，或 UNSAT

```

1: for  $II \leftarrow II_{\min}$  to  $II_{\max}$  do
2:    $\text{apsp} \leftarrow \text{FLOYDWARSHALL}(G, II)$  ▷ 计算窗口内依赖下界
3:   for all  $L \in \mathcal{L}$  do
4:      $S \leftarrow \text{NEWCP SATMODEL}$ ;  $S.\text{timeout} \leftarrow T_{\max}$ 
5:     建立变量  $\text{op}[v, t]$ 、 $\text{warp}[v, w]$ 、 $\text{live}[x, \tau]$  与  $\text{incoming\_live}[x, \tau]$ 
6:      $T_v \leftarrow \sum_t t \cdot \text{op}[v, t]$ 
7:     加入唯一性、依赖下界、阻塞同步、功能单元容量、活跃区间、寄存器上限和共享内存上限约束
8:     目标为先最小化  $\max_v T_v$ ，再最小化  $\sum_v T_v$ 
9:     if  $S.\text{check}() = \text{SAT}$  then
10:       hints  $\leftarrow$  从  $\{T_v\}$  提取每个缓冲区的首次读取和末次访问位置
11:       return  $\{II, L, \text{schedule}, \text{warp\_assign}, \text{reg\_peak}, \text{hints}\}$ 
12:     end if
13:   end for
14: end for
15: return UNSAT

```

表 5-4 HeddleScheduler 生成的调度配置与编译决策输出示例

Table 5-4 Schedule Configuration and Compilation Decisions Generated by HeddleScheduler

类别	项	含义
调度结果	$\text{schedule}[v] = t^*$	操作 v 在窗口内的启动时刻
可选调度参数	ii, window	可行启动间隔与求解窗口参数
可选空间信息	$\text{warp_assign}[v] = w^*$	一般化接口下的 WarpGroup 映射
资源统计	reg_peak	各 WarpGroup 的寄存器峰值

界、功能单元互斥和资源峰值约束下重新分配启动次序。

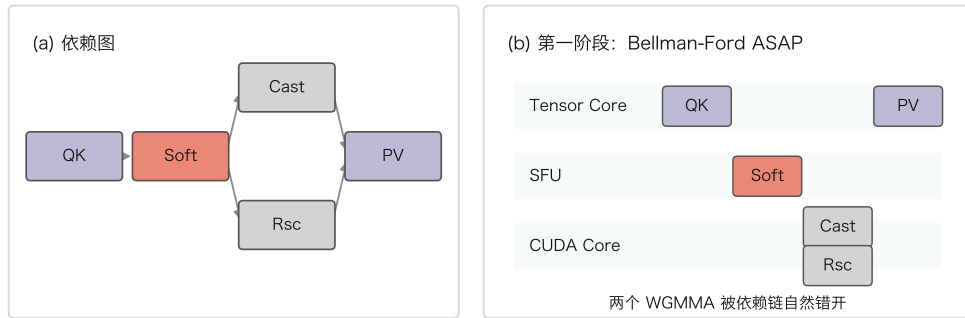


图 5-5 FlashAttention 前向中的快速路径调度示例

Figure 5-5 Fast-Path Scheduling Example for FlashAttention Forward

图 5-5 给出了 FlashAttention 前向的典型情形。消费者依赖链由 QK、Softmax、类型转换、重缩放和 PV 组成，两个 WGMMA 分别位于链首和链尾，中间的逐元素计算使两次 Tensor Core 发射之间形成间隔。依赖传播得到的顺序已经满足依赖关系和功能单元上限，后续步骤据此生成共享缓冲区的等待和释放提示。

图 5-6 展示了需要进入 CP-SAT 路径的情形。FlashAttention 反向 dK/dV 的消费者图包含 KQ^T 、 dP 、Softmax、类型转换、 dS 、 dV 和 dK 。其中 dP 由 VdO^T GEMM 产生， dS 是 Softmax 反向中的逐元素计算。图中从左到右表示调度器给消费者语句分配的相对启动次序，不表示硬件周期数；浅色块表示下一轮迭代中已经进入窗口的语句。依赖传播依据依赖下界给出最早启动次序，因此 KQ^T 与 dP 被放到相同启动位置，后续的 dV 与 dK 也被放到相同启动位置。虚线框内上下叠放的两个紫色块表示两个 WGMMA 同时竞争 Tensor Core。该排布依赖合法，但不满足同一启动位置至多一个 WGMMA 的功能单元约束。CP-SAT 路径保留同一依赖图，并加入功能单元互斥和资源峰值约束；求解后，同类 WGMMA 的启动位置被错开， dS 仍保留在 CUDA Core 路径上。最终调度同时满足依赖、功能单元和活跃字节约束，再被转换为消费者顺序和屏障位置提示。

这两个例子说明，Heddle 以快速路径处理常见的串行依赖模式；对于独立 WGMMA 较多的候选，CP-SAT 路径在资源约束下重新分配启动时刻。调度结果

作用于已经确定的内核内部；图级融合或分离方案仍由 AutoMixed 的候选评估、编译反馈和实测吞吐共同决定。

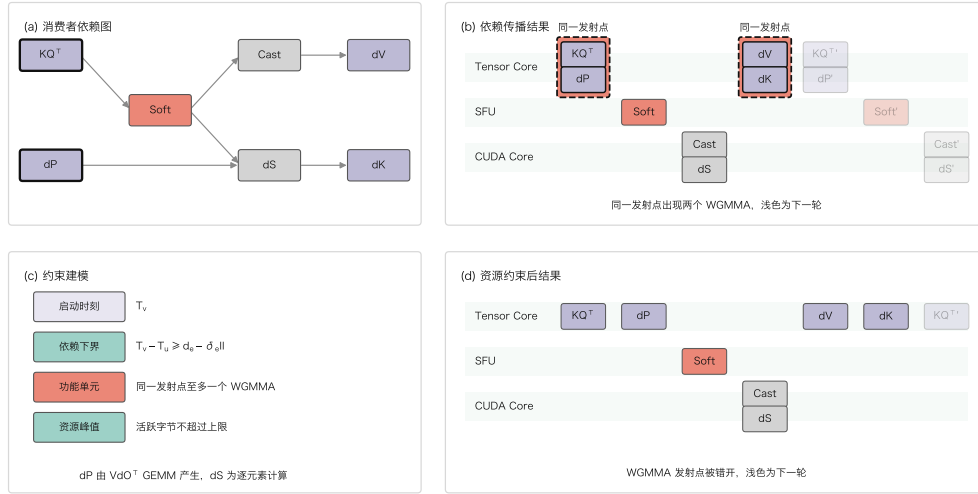


图 5-6 FlashAttention 反向 dK/dV 中的约束路径调度示例

Figure 5-6 Constrained Scheduling Example for FlashAttention Backward dK/dV

5.4 多角色协同生成

单消费者 WarpGroup 内的重排适合调整语句顺序和屏障位置。面对多段 WGMMMA 计算链时，寄存器状态会集中在同一个消费者 WarpGroup；面对阻塞式 TMA 写回时，消费者关键路径会被写回等待拉长。Heddle 因此在 IR 中识别两类可拆分结构：一类是由两个 WGMMMA 与中间标量计算组成的计算链，另一类是 TMA 规约加写回序列。前者生成双消费者结构，将 QK 与 Softmax 链和 PV 链分配到不同 WarpGroup；后者生成 TRWS 结构，将 dQ 写回从消费者关键路径中移出。

双消费者候选针对 WGMMMA 计算、非 Tensor Core 计算与后续 WGMMMA 计算串接形成的计算链。以 FlashAttention 前向为例，QK WGMMMA 与 Softmax 生成概率块，PV WGMMMA 在下一流水线片段中读取该概率块并累加输出。Heddle 根据定义-使用关系识别两段 WGMMMA 之间的中间值传递，并生成 QK 消费者与 PV 消费者的功能分工。该执行结构降低了单个 WarpGroup 持有的寄存器状态，也把跨迭代的概率块生产和消费映射到两个消费者 WarpGroup 上。

TRWS 候选针对 TMA 规约加等阻塞写回序列。以 FlashAttention 反向为例，dQ 的跨块累加会下降为 tma_store、到达通知和等待序列；若该序列留在消费者 WarpGroup 中，会阻塞后续 dK 与 dV 计算。Heddle 通过 DetectTmaReduceAdd 识别这类写回路径，并将其剥离为独立角色，使写回与主计算链并行推进。

CUTLASS 3.x 在乒乓调度 (PingPong) 模板中实现了两个消费者 WarpGroup 的交替推进。其核心同步原语是 OrderedSequenceBarrier，通过全局序列计数器

协调相邻迭代之间的 QK、PV 等计算阶段。Heddle 采用相近的并行化目标，但在 TIR 编译器变换层根据内核数据流生成角色分工和同步协议。具体差异如表 5-5 所示。

表 5-5 CUTLASS 乒乓调度与本文方案的设计维度对比

Table 5-5 Design Dimensions: CUTLASS Ping-Pong vs Proposed Approach

维度	CUTLASS 乒乓调度	本文方案
实现层次	C++ 模板库，手工编写	TIR 编译器 Pass 生成
同步原语	OrderedSequenceBarrier	命名屏障与乒乓双缓冲
数据传递	隐式流水线状态偏移	显式寄存器-共享内存-寄存器 中转
消费者分工	乒乓调度，对称消费者按序 交替推进	功能分工，QK 消费者负责 QK， PV 消费者负责 PV
跨 WarpGroup 检测	手工指定	use-def 分析与写回模式识别
适用范围	需为每个算子定制模板	两段 WGMMA 计算链；TMA 规 约加写回路径

两者的主要差异在于生成方式和分工方式。CUTLASS 的乒乓调度(PingPong)采用对称消费者设计，两个消费者 WarpGroup 按全局序号交替推进相邻迭代的主计算阶段。Heddle 在 IR 层识别结构模式后生成对应执行结构：双消费者采用功能分工，QK 消费者执行 QK GEMM 与 Softmax，PV 消费者执行 PV GEMM；TRWS 则把 TMA 规约加写回从消费者关键路径中剥离。前者降低每个 WarpGroup 同时持有的寄存器状态，后者减少阻塞写回对主计算链的影响。

生成最终循环注解前，Heddle 会统一比较可用的执行机制。异步 WGMMA 和提前释放主要改变等待位置与缓冲区释放时机，适用于含 WGMMA 的流水化循环；双消费者会引入共享内存中转和乒乓同步；TRWS 需要额外的写回线程束与握手屏障。Heddle 以稳态迭代时间为准则，在满足结构条件的候选中选择最终配置。

$$T_{\text{iter}} = \max(T_{\text{producer}}, T_{\text{consumer}}). \quad (5-4)$$

生产者时间主要由 TMA 搬运和反压等待决定，消费者时间主要由计算语句、屏障等待和跨 WarpGroup 中转决定。提前释放作用在生产者侧，使 TMA 在 WGMMA 发射后即可搬运下一版本；异步 WGMMA 作用在消费者侧，使最后一个 WGMMA 的累加与下一迭代首部工作重叠。双消费者将 QK 与 Softmax 链和 PV 计算链分配到两个 WarpGroup 上执行，代价是增加共享内存中转和乒乓同步。TRWS 将 dQ 写回移出消费者关键路径，使写回与后续计算并行。

候选评估的输入来自当前 Tile IR。WGMMA 延迟按分块 K 维估算，TMA 延迟由缓冲区字节数和标定带宽估算，屏障等待与跨 WarpGroup 中转采用 H100 上的微基准标定值。 T_{iter} 最小的配置被写入循环注解，交由后续重写器生成对应执行结构。

5.4.1 双消费者模式的 WGMMMA 流水线拆分

注意力前向中, QK WGMMMA 先计算注意力分数, Softmax 生成概率块, PV WGMMMA 再读取概率块和 V 分块并累加输出。同一语义迭代内的 QK→Softmax→PV 依赖必须保留, 因此双消费者并不把同一迭代的 PV 提前到对应 Softmax 之前。它利用的是稳态流水线中的相邻迭代错位: 当 QK 消费者在一个流水线片段内写出第 k 个概率块后, 下一片段可以继续生成第 $k+1$ 个概率块; PV 消费者在这一片段读取已经写好的第 k 个概率块并执行 PV WGMMMA。这样, 同一迭代的数据依赖仍按 QK→Softmax→PV 执行, 但 QK 与 Softmax 链和 PV 链不再在同一个消费者 WarpGroup 内串行推进。

这一关系与第 4 章的 FlashAttention 前向案例一致。AutoPiped 在单消费者 WarpGroup 内生成跨迭代错位的 order 和 stage 注解, 使同一稳态片段中同时出现下一块的 QK 与 Softmax、上一块的 PV。双消费者模式接续这一时间结构, 把两条链分配给不同消费者 WarpGroup, 并通过中转缓冲和乒乓屏障维护概率块的就绪与复用。AutoPiped 确定循环体可表达的跨迭代交错关系, Heddle 进一步判断该关系是否适合映射为两个消费者 WarpGroup 的空间分工。

Heddle 在 IR 中检测两个 WGMMMA 之间夹有非 Tensor Core 计算的结构, 作为双消费者候选。该结构要求前一个 WGMMMA 及其中间标量计算产生概率块, 后一个 WGMMMA 读取该概率块。本文实现的前向配置采用 $bM=64$, 使每个 WGMMMA 由 1 个 WarpGroup 执行, 三类角色共 384 线程。生产者 WarpGroup 负责 TMA 加载, QK 消费者负责 QK WGMMMA 与 Softmax, PV 消费者负责 PV WGMMMA。两类消费者之间通过共享内存中转概率块, 并使用乒乓屏障维护数据就绪和缓冲区复用关系。

是否启用双消费者由式 (5-4) 的代价模型决定。若 QK 与 Softmax 链已经主导稳态时间, 且 Tensor Core 发射资源接近饱和, 共享内存中转和乒乓同步会抵消拆分收益, 模型保留单消费者配置。若头维度较大或分块形状使单消费者寄存器压力较高, 双消费者可以拆分概率块片段和输出累加器状态; 当 PV 侧计算能够与下一轮 QK 和 Softmax 形成有效错位时, 模型选择双消费者配置。

共享内存中转使用的缓冲区由 FGWS 的读写集分析确定。QK 消费者写集与 PV 消费者读集的交集即为传输集合。编译器区分 f16 和 bf16 的 WGMMMA 中间结果以及 f32 的标量状态, 分别通过不同的写入和读取原语处理。QK 消费者将寄存器片段通过 128 位向量化写入共享内存缓冲区, PV 消费者从中读取后使用 wgmma_rs。为消除向量化写入引入的存储体冲突, 缓冲区在每行末尾添加跨步填充。经过这两项处理后, 双消费者候选的代价主要来自乒乓屏障和中转读写, 可由代价模型计入; 不同形状下的启用决策见第 6.3.6 节。

为实现 QK 与 PV 的跨迭代重叠, FGWS 设计了基于 4 个命名屏障的乒乓协议。引入两个缓冲版本 $s \in \{0, 1\}$, 每个版本使用一对屏障, 如表 5-6 所示。

QK 消费者在第 k 次迭代使用版本 $k \bmod 2$, 先等待 att_consumed, 再执行 QK 与 Softmax 并写入中转缓冲, 最后通知 att_ready; PV 消费者先等待 att_ready,

表 5-6 双消费者乒乓协议的命名屏障

Table 5-6 Named Barriers in the Dual-Consumer Ping-Pong Protocol

屏障	含义
att_ready[s]	QK 消费者完成 Softmax 并写入中转缓冲版本 s 后通知 PV 消费者数据已就绪
att_consumed[s]	PV 消费者读完中转缓冲版本 s 后通知 QK 消费者该版本可被重用

执行 PV GEMM 后通知 att_consumed。PV 消费者在循环前预先通知两个版本的 att_consumed，使 QK 消费者前两次迭代无需等待。进入稳态后，第 $k+1$ 次 QK 与 Softmax 和第 k 次 PV 使用不同中转版本并行执行。图 5-7 中，横向分段表示流水线片段，而不是单个语义迭代；上三行给出生产者、QK 消费者和 PV 消费者的并行执行关系，下两行展开中转槽 S0 和 S1 的生命周期。以 S0 为例，QK0 在片段 0 写入 S0，PV0 在片段 1 读取 S0，随后 QK2 在片段 2 复用同一槽位。

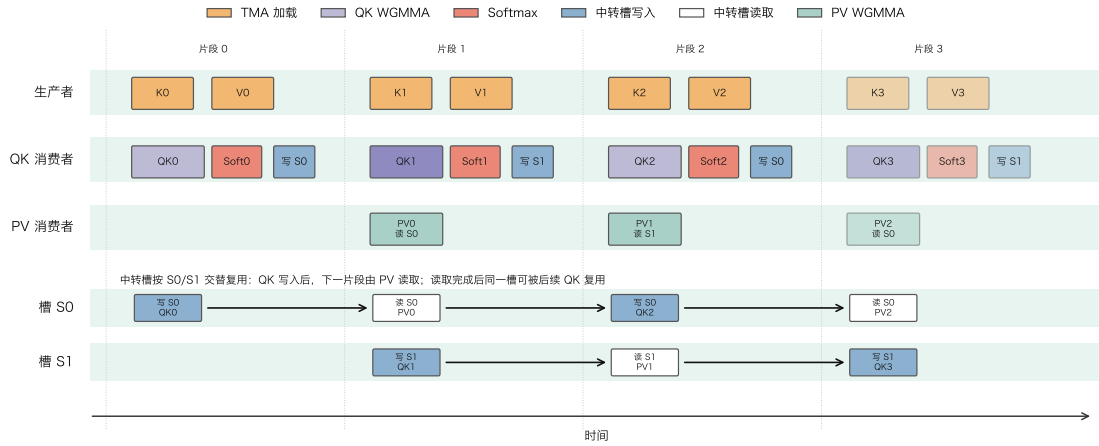


图 5-7 双消费者乒乓协议时序

Figure 5-7 Dual-Consumer Ping-Pong Protocol Timing

此外，双消费者模式下 QK 消费者不再负责最终输出累加，但原始 IR 中 O_{acc} 的缩放操作仍保留在 QK 消费者侧。FGWS 通过模式匹配检测并消除这类死代码。

5.4.2 三角色模式的阻塞写回剥离

FlashAttention 反向中 dQ 的跨块累加需要原子操作，经下降变换后生成 TMA 规约加序列 (tma_store $\rightarrow$ arrive $\rightarrow$ wait)。该序列占用 TMA 写通道，wait 会阻塞当前线程直到写回完成，若在消费者 WarpGroup 中串行执行将直接阻断后续计算。

TRWS 候选通过 DetectTmaReduceAdd 扫描消费者语句生成。检测到 TMA 规约加模式后，编译器将相关语句移入独立的 dQ 写回循环，并由代价模型判断是否启用三角色配置。启用后，原生产者侧被拆成 TMA 加载路径和 dQ 写回路径，二者分别使用独立线程束执行；其余线程束继续服务于主消费者路径。消费

者与 dQ 写回线程束之间通过一对命名屏障 dQ_full 和 dQ_empty 完成握手，前者通知数据已就绪，后者通知缓冲区已腾空。该结构使 dQ 的 TMA 写回与下一次迭代的 dK 与 dV 计算并行，减少写回等待对消费者关键路径的占用。

5.5 与 Twill 的技术路线对比

Twill^[49] 同样关注张量核心流水线中的依赖、资源与线程束特化。二者的主要差异在求解位置。Twill 面向较一般的 Tile 级程序调度，输入是一组张量核心操作、访存操作及其依赖关系，求解器直接决定操作的模调度时刻和 WS 映射。Heddle 位于本文三层编译流程的末端，在 AutoMixed 已确定融合边界、AutoPiped 已生成基础流水线注解、FGWS 已建立生产者侧屏障协议之后，处理单个内核内部的消费者启动时刻、消费者排列、屏障位置和资源峰值检查。

在约束表达上，Twill 通过展开多个流水线迭代表达跨迭代依赖。Heddle 将迭代折叠到有限调度窗口中，用启动时刻变量 T_v 、依赖延迟 d_e 和迭代距离 δ_e 表达依赖下界，并用 live 与 incoming_live 跟踪窗口内外的活跃值。循环携带依赖、功能单元占用和寄存器活跃区间因而可以在同一窗口内检查。

在空间维表达上，Twill 将操作映射到 WS 执行单元。Heddle 的多角色候选来自两类 IR 模式：双消费者沿 WGMMA 计算链切分 QK 与 Softmax 链和 PV 计算链，TRWS 沿 TMA 写回路径剥离 dQ 写回；候选是否启用由结构检测和代价模型共同决定，CP-SAT 路径负责检查给定候选下的启动时刻和资源峰值。

两者定位不同。Twill 更接近独立的 Tile 级约束调度器；Heddle 位于本文的编译链路末端，求解结果会转化为 TileLang 后端使用的消费者顺序、屏障位置提示和多角色注解。第 6 章据此采用同一 TileLang 后端上的消融实验，评估 FGWS、消费者重排、异步 WGMMA、双消费者和 TRWS 的增量收益。

5.6 本章小结

本章介绍了 Heddle 的内核内细粒度线程束特化调度。Heddle 承接 AutoMixed 生成的内核结构和 AutoPiped 生成的基础流水线注解，进一步处理缓冲区同步、消费者顺序、资源峰值和多 WarpGroup 协同。

执行层上，FGWS 通过异步拷贝模式提取、缓冲区级屏障布局 and 相位计数器，把阶段级生产者-消费者同步细化到共享缓冲区版本。在测试的三类算子上，阶段级 WS 的编译通过率为 33% 到 50%，FGWS 均提升至 100%，并把消费者排列顺序暴露为后续调度空间。

调度层上，HeddleConsumerSchedule 在消费者依赖图上生成语句顺序和屏障位置提示。启发式路径处理常见依赖结构，CP-SAT 路径处理独立 WGMMA 较多或资源峰值较高的候选。异步 WGMMA 和提前反压释放从生产者侧和消费者侧共同缩短稳态迭代周期；CP-SAT 资源约束路径进一步检查寄存器活跃区间和共享内存上限。

多角色扩展层上，Heddle 根据 WGMMA 计算链和 TMA 写回路径生成多 WarpGroup 分工。双消费者模式通过共享内存中转和乒乓双缓冲支持 QK 与 Softmax 链和 PV 计算链的跨 WarpGroup 协同；TRWS 将 dQ 的 TMA 规约加操作从消费者关键路径移出。至此，缓冲区级同步、消费者调度和多角色执行结构共同构成 Heddle 的内核内调度流程。

第 6 章 实验评估

本章在 NVIDIA H100 GPU 上评估 AutoMixed、AutoPiped 和 Heddle。实验分为三类：六类算子的端到端性能，各模块的消融验证，以及关键优化机制的贡献分析。

6.1 实验设置

6.1.1 硬件与软件环境

表 6-1 给出了实验主机的硬件配置，表 6-2 给出了软件栈与对比基准。

表 6-1 实验主机硬件配置
Table 6-1 Hardware Configuration

项目	配置
GPU	NVIDIA H100 80GB HBM3 (SXM5)
GPU 显存带宽	3.35 TB/s
GPU FP16 Tensor Core 峰值	989.4 TFLOPS
处理器	Intel Xeon Platinum 8480C (2×56 核)
主机内存	2 TB DDR5

6.1.2 测量方法与评价指标

除特别说明外，Heddle、TileLang 与 Triton 的自动调优在相同候选参数范围内进行，厂商库和 FlashAttention 系列使用各自默认调度路径。性能结果统计完成编译、调优和首次 JIT 之后的运行时间，不包含编译时间、搜索时间或缓存构建时间。实验脚本在每组配置开始前清除编译缓存并重新生成内核，该准备阶段不计入表中延迟。

本文区分内核级测量和端到端测量。GEMM、FlashAttention 前向以及后文给出 NCU 硬件指标的消融实验使用 NVIDIA Nsight Compute² 采集，延迟取内核级 `gpu_time_duration.sum`，Tensor Core 利用率等指标也来自同一次采集。每个配置先预热 3 次，再执行 1 次 NCU 采集；由于 NCU profile 开销较大，表中硬件指标用于解释瓶颈来源，主要加速比结论以 CUDA Event 或端到端重复计时为准。对 NCU 表中的关键配置另行重复采集 3 次，确认寄存器数、占用率和主要吞吐趋势一致，正文不再绘制误差条。Conv2D、Dequant GEMM、Grouped GEMM 等只报告端到端吞吐的算子采用 CUDA Event 计时，每项预热 20 次、测

¹Triton 的 GEMM、Dequant GEMM 和 FlashAttention 前向使用 Triton JIT 内核；Grouped GEMM 顺序调用 Triton GEMM；Conv2D 由 torch.compile 的 Triton Inductor 生成；不对比 FlashAttention 反向。

²简称 NCU，NVIDIA 的 GPU 内核级性能分析工具。

表 6-2 软件栈与对比基准
Table 6-2 Software Stack and Baselines

类别	版本	说明
平台软件栈		
CUDA Toolkit	13.0	含 nvcc、ptxas 编译器
PyTorch	2.10	深度学习框架
Python	3.12	运行环境
厂商手写库		
cuBLAS	v13.0	NVIDIA 官方 BLAS 库, GEMM 的工业级基准
cuDNN	v9.0	NVIDIA 官方深度学习原语库, 用作 Conv2D 基线; 标准注意力通过 PyTorch SDPA 调用其 FP16 Attention 后端, 图中标为 cuDNN
FlashAttention-2	v2.7	Dao 等人的开源注意力实现
FlashAttention-3	v3.0	为 Hopper 架构优化的 SASS 级手写内核
编译框架		
Triton	v3.5	OpenAI 开源 GPU 编程语言 ¹
TileLang	v0.1.6.post2	Tile-AI 开源 Tile 级编程语言

量 100 次并取中位数。模型级推理实验同样采用 CUDA Event 端到端重复计时, 其中 LLaMA-2 7B 五变体实验预热 5 次、测量 20 次取中位数, Qwen-2.5 7B 补充实验预热 20 次、测量 100 次取中位数。

本章以吞吐量和加速比作为主要评价指标。吞吐量以 TFLOPS 表示, 按理论浮点运算量除以该表采用的延迟计算: NCU 实验使用 NCU 报告的内核延迟, CUDA Event 实验使用 100 次测量的中位延迟。加速比定义为

$$\text{Speedup} = \frac{t_{\text{base}}}{t_{\text{ours}}}, \quad (6-1)$$

其中 t_{base} 为基线实现的延迟, t_{ours} 为完整系统生成内核的延迟。算子级对比中, 基线分别取对应的厂商库或编译框架; 消融实验中, 基线为逐步移除关键优化后的配置。除特别说明外, 所有实验的数据类型为 FP16。

编译框架对比以 TileLang 为主要参照。本章表格中的“Heddle”列表示 AutoMixed、AutoPiped 和 Heddle 三模块均启用的完整系统; 讨论第三个模块本身时, 正文显式称为 Heddle 模块或 Heddle 调度。完整系统与 TileLang 共享 Tile IR 和后端代码生成流程, 二者差异集中在本文新增的调度与资源决策; Triton 作为独立编译器基线, 用于补充观察不同编程系统下的性能水平。

为说明算子级实验与主流大模型负载之间的对应关系, 本文从典型 decoder-only 大模型中抽取线性层、Attention 和 FFN 的代表性形状。GEMM 对应 QKV/O 投影和 FFN up/down 投影, FlashAttention 对应 prefill 阶段的多头注意力计算, Grouped GEMM 则对应 MoE 模型中多个 expert FFN 的批量执行模式。除用于控制变量分

析的规则方阵外，本文还在 Qwen、LLaMA 和 Gemma 等 7B–9B 规模模型的**真实矩阵形状上进行补充验证**。表 6-3 给出了这些形状的来源。

表 6-3 主流模型代表性算子形状

Table 6-3 Representative Operator Shapes from Mainstream Models

模型	QKV 投影	O 投影	FFN up+gate	FFN down
Qwen-2.5 7B	3584 → 4608	3584 → 3584	3584 → 37888	18944 → 3584
LLaMA-2 7B	4096 → 12288	4096 → 4096	4096 → 22016	11008 → 4096
LLaMA-3 8B	4096 → 6144	4096 → 4096	4096 → 28672	14336 → 4096
Gemma-2 9B	3584 → 7680	3584 → 3584	3584 → 28672	14336 → 3584

表中形状为线性层输入维度到输出维度；prefill GEMM 实验使用序列长度作为 M 维度。

6.2 性能对比

本节比较六类核心深度学习算子的单算子性能，比较对象包括 TileLang、Triton 以及各算子的常用外部基线。表 6-4 汇总所测形状上的结果，后文按算子类型逐项分析。

表 6-4 算子级性能总结

Table 6-4 Operator-Level Performance Summary

算子	相对 TileLang	相对 Triton	相对外部基线
GEMM	1.10–1.45x; 1.29x	0.81–2.36x; 1.30x	0.79–0.96x [†] ; 0.87x
Grouped GEMM	1.11–1.22x; 1.18x	0.93–2.09x; 1.46x	0.61–1.09x [¶] ; 0.79x
Conv2D	1.48–1.83x; 1.67x	2.44–5.44x; 4.13x	0.94–2.21x [‡] ; 1.53x
Dequant GEMM	1.64–2.03x; 1.83x	5.57–5.94x; 5.76x	0.78–0.82x ; 0.80x
FlashAttention 前向	1.01–1.04x; 1.03x	2.39x [#]	0.76–0.93x [§] ; 0.85x
FlashAttention 反向	1.33–1.45x; 1.38x	—	1.45–1.83x [§] ; 1.69x

[†] vs cuBLAS; [‡] vs cuDNN; [§] vs FlashAttention-3; [¶] vs torch.mm (顺序 cuBLAS); ^{||} vs Marlin。

单元格格式为“范围; 均值”。[#] Triton FlashAttention 前向仅测 $B=4, T=4096$; 反向未纳入 Triton 对比。

6.2.1 通用矩阵乘法

GEMM 结构规则、依赖简单，适合作为观察流水线调度收益的基础场景。

Heddle 在所有规模上均超越 TileLang，加速比随矩阵规模增大从 1.10 倍增长至 1.45 倍，最大规模上达到 cuBLAS 的 96%。这一趋势与流水线收益的规模特征一致：矩阵规模越大，跨迭代重叠隐藏访存延迟的空间越充分。

与 cuBLAS 的相对差距随规模变化。 $M=2048$ 时 Heddle 达到 cuBLAS 的 79%， $M=8192$ 时达到 96%。中等规模上的差距主要来自模板族和搜索空间覆盖度，例如 split-K、CTA 重排和持久化分块等策略尚未纳入当前实现。

表 6-5 GEMM 端到端性能对比

Table 6-5 GEMM End-to-End Performance Comparison

矩阵规模 $M = N = K$	cuBLAS	Triton	TileLang	Heddle	加速比
1024	235.1	84	180.5	197.9	1.10x
2048	691.0	609	419.6	546.4	1.30x
4096	753.9	671	499.1	660.9	1.32x
8192	695.0	580	460.6	668.6	1.45x

单位 TFLOPS；此处取方阵 $M=N=K$ 。加速比为 Heddle 相对于 TileLang。

在表 6-3 所列真实非方阵形状上，Heddle GEMM 达到 cuBLAS 的 0.68 到 1.00 倍。其中 FFN down 类投影（ K 维度较大、 N 较小）的比值最高，为 0.87 到 1.00；FFN up+gate 类投影（ N 极大）的比值较低，为 0.68 到 0.81。该结果说明方阵实验展示的是调度收益上限，而真实模型中的非方阵线性层还会受到矩阵长宽比和 cuBLAS 专用分块策略的影响。

6.2.2 分组矩阵乘法

Grouped GEMM 将多个不同大小的矩阵乘法打包到单次内核启动中。实验从三个维度评估：不同子矩阵规模、不同内积维度 K 与 N ，以及不同分组数量。

表 6-6 Grouped GEMM 不同子矩阵规模下的性能对比

Table 6-6 Grouped GEMM Performance across Submatrix Sizes

批次大小	torch.bmm	Triton	torch.mm	TileLang	Heddle	加速比
[64, 128, 256, 512]	341	208	398	367	435	1.19x
[128, 256, 512, 1024]	354	317	578	407	481	1.18x
[256, 512, 1024, 2048]	339	434	697	413	490	1.19x
[512, 1024, 2048, 4096]	303	507	716	423	471	1.11x

单位 TFLOPS， $K=N=4096$ ，4 组；torch.mm 为顺序 cuBLAS，torch.bmm 为填充后 batched cuBLAS，Triton 为顺序 JIT GEMM。加速比相对 TileLang。

表 6-6 给出了四种子矩阵规模下的结果。Heddle 相比 TileLang 加速 1.11 到 1.19 倍。小批次配置上 Heddle 高于顺序 cuBLAS 和 Triton，单内核打包减少了多次启动开销。大批次配置上顺序 cuBLAS 反超 Heddle，因为单次矩阵乘已接近峰值吞吐。torch.bmm 因填充冗余始终最低。

表 6-7 从两个维度补充考察 Grouped GEMM。首先固定 4 组子矩阵并增大 K 和 N ，Heddle 相对 TileLang 的加速比由 1.16 倍提升至 1.21 倍。较大的内积维度增加了主循环迭代数，TMA 加载与 WGMMMA 计算的重叠更充分。其次，在总 M 规模基本不变的条件下改变分组数量，Heddle 的加速比保持在 1.17 到 1.20 倍。Triton 顺序调用在 8 组配置下降至 377 TFLOPS，主要受多次小矩阵调用的启动开销影响；Heddle 在单个内核中调度所有分组，不同分组数下吞吐保持在 483 到 508 TFLOPS。

6.2.3 隐式卷积

隐式卷积实验关注 im2col 数据重排与矩阵计算的重叠效果。实验在 ResNet 风格的卷积配置下评估 Heddle 的隐式 GEMM 实现。

Heddle 相比 TileLang 实现了 1.48 到 1.83 倍的加速，是六类算子中增益最大的类别之一。以 ResNet 14×14 配置为例，TileLang 仅达 6.0 TFLOPS，而 Heddle 提升至 11.0 TFLOPS，加速 1.83 倍。该配置下特征图较小、通道数较大 ($C=256$)，im2col 重排的访存压力成为瓶颈，AutoPiped 构建的多阶段流水线有效隐藏了数据重排延迟。

与 cuDNN 相比，Heddle 在大特征图配置上分别达到 cuDNN 的 2.21 倍和 1.51 倍；在小特征图高通道数配置上为 cuDNN 的 0.94 倍。大特征图上的收益来自 AutoPiped 构建的多阶段流水线对 im2col 重排延迟的隐藏。

6.2.4 低比特反量化矩阵乘

低比特反量化矩阵乘的循环体同时包含 WGMMA、位解包和反量化操作，适合观察标量计算与张量核心计算的重叠效果。实验评估 W4A16 反量化矩阵乘 (Dequant GEMM)，该算子在大语言模型权重量化推理中广泛使用。

表 6-7 Grouped GEMM 在不同 K 与 N 维度和分组数量下的性能

Table 6-7 Grouped GEMM Performance across K and N Dimensions and Group Counts

配置	torch.mm	Triton	TileLang	Heddle	加速比
不同 K 与 N 维度					
$K=N=2048$	442	299	402	465	1.16x
$K=N=4096$	697	428	414	489	1.18x
$K=N=8192$	700	526	398	481	1.21x
不同分组数量					
1 组 [4096]	673	525	414	484	1.17x
2 组 [2048, 2048]	715	516	431	508	1.18x
4 组 [1024, 1024, 1024, 1024]	680	502	426	501	1.18x
8 组 [512, ..., 512]	660	377	404	483	1.20x

单位 TFLOPS。加速比为 Heddle 相对于 TileLang。

表 6-8 Conv2D 端到端性能对比

Table 6-8 Conv2D End-to-End Performance Comparison

配置	Triton	cuDNN	TileLang	Heddle	加速比
ResNet 56×56 ($C_{in}=64$, $C_{out}=64$)	5.4	13.3	19.8	29.4	1.48x
ResNet 28×28 ($C_{in}=128$, $C_{out}=128$)	4.5	13.3	11.4	19.3	1.69x
ResNet 14×14 ($C_{in}=256$, $C_{out}=256$)	4.5	11.7	6.0	11.0	1.83x
Default 64×64 ($C_{in}=128$, $C_{out}=128$)	23.1	65.9	58.9	99.6	1.69x

单位 TFLOPS， $N=1$ ，卷积核 3×3，FP16。Triton 由 torch.compile (Inductor/Triton) 生成；加速比相对 TileLang。

表 6-9 W4A16 Dequant GEMM 端到端性能对比

Table 6-9 W4A16 Dequant GEMM End-to-End Performance Comparison

M=N=K	Triton	AWQ	TileLang	Heddle	Marlin	加速比
2048	47	67.5	159.5	261.9	329	1.64x
4096	50	92.8	157.6	288.3	372	1.83x
8192	50	105.5	145.6	295.9	361	2.03x

单位 TFLOPS。Triton 为循环内 4-bit 解包的 JIT 内核；Marlin 和 AWQ 为 vLLM 内置 W4A16 内核，group_size=128。加速比相对 TileLang。

Heddle 相比 TileLang 实现了 1.64 到 2.03 倍的加速，加速比随矩阵规模增大单调增长。高加速比源于循环体中位解包与反量化操作的延迟较长，AutoPiped 基于生产者-消费者依赖生成的多阶段流水线使位操作与 WGMMMA 计算在时间上重叠执行。

表中同时列出了两类专用 W4A16 推理内核。Marlin 在权重打包格式、访存合并模式和 warp 级数据分发上做了专用优化，Heddle 达到 Marlin 的 0.78 到 0.82 倍。AWQ 内核吞吐低于 Heddle 和 Marlin。Heddle 使用通用 dequant GEMM 模板，位解包在循环体内以标量方式执行；专用权重布局和解包指令选择仍是量化内核继续优化的重点。

6.2.5 FlashAttention

FlashAttention 同时包含 TMA 加载、WGMMMA、Softmax 和跨迭代状态更新，是检验三层编译决策的主要复杂算子。实验在不同序列长度 $T \in \{1024, \dots, 8192\}$ 下评估前向和反向性能。

Heddle 与 TileLang 各自使用调优后的最佳配置时，前向传播加速 1.01 到 1.04 倍，在长序列上达到 cuDNN 的 89% 到 92%。短序列上达到 cuDNN 的 80%，主要因为循环迭代次数较少，异步 WGMMMA 的重叠窗口利用不足。固定分块形状、线程数与流水线级数后，仅开启异步 WGMMMA 等待深度调整和提前释放可带来 1.06 到 1.09 倍的增量。

在真实模型注意力形状上，Qwen-2.5 7B、LLaMA-3 8B 和 LLaMA-2 7B 均采用 $D=128$ 的头维度。在 $B=1, T=4096$ 的 prefill 配置下，Heddle 相比 SDPA 加速 1.63 到 1.69 倍，相比 Triton 加速 1.73 到 1.79 倍；不同模型间 Heddle 的绝对吞吐差异不超过 2%，说明该调度质量不依赖具体头数配置。

NCU 分析显示，当前前向内核的计算吞吐已达 60.5%，Tensor Core 发射资源接近饱和；后续收益主要取决于更底层的发射控制和 WarpGroup 迭代交错能力。

反向传播的计算依赖关系更复杂，包含 4 个 WGMMMA、Softmax 反向以及 dQ 的原子归约。AutoMixed 的资源分析选择了分离式设计，将 dKdV 与 dQ 拆为两个独立内核，降低了单个内核的寄存器压力和共享内存占用。

Heddle 在四个测试配置下超越 TileLang 基线 1.33 到 1.45 倍，并在这些配置

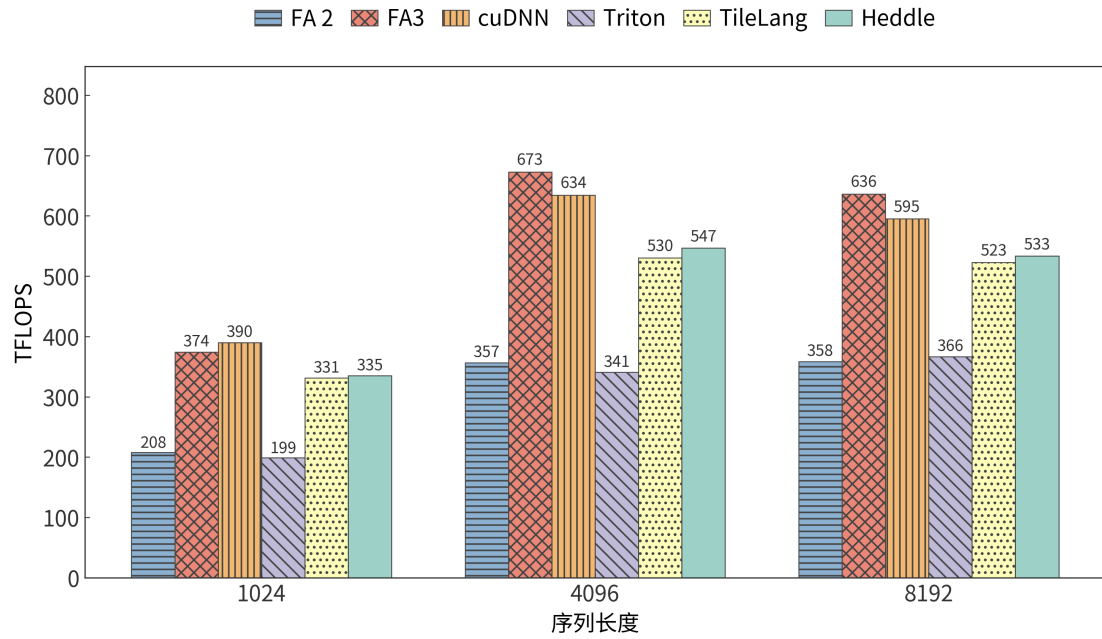
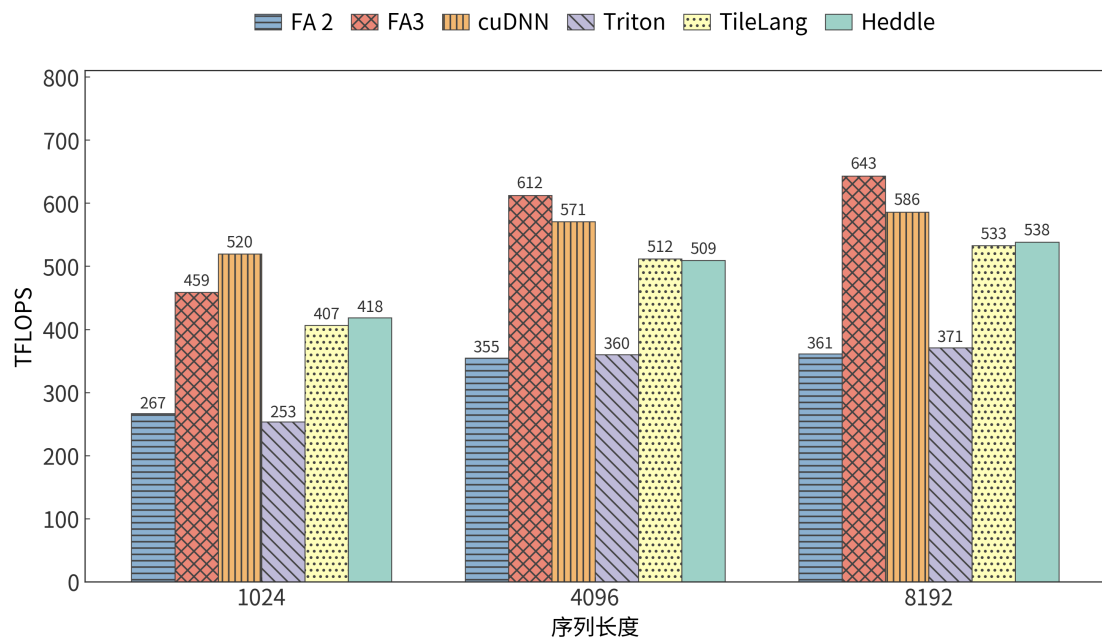
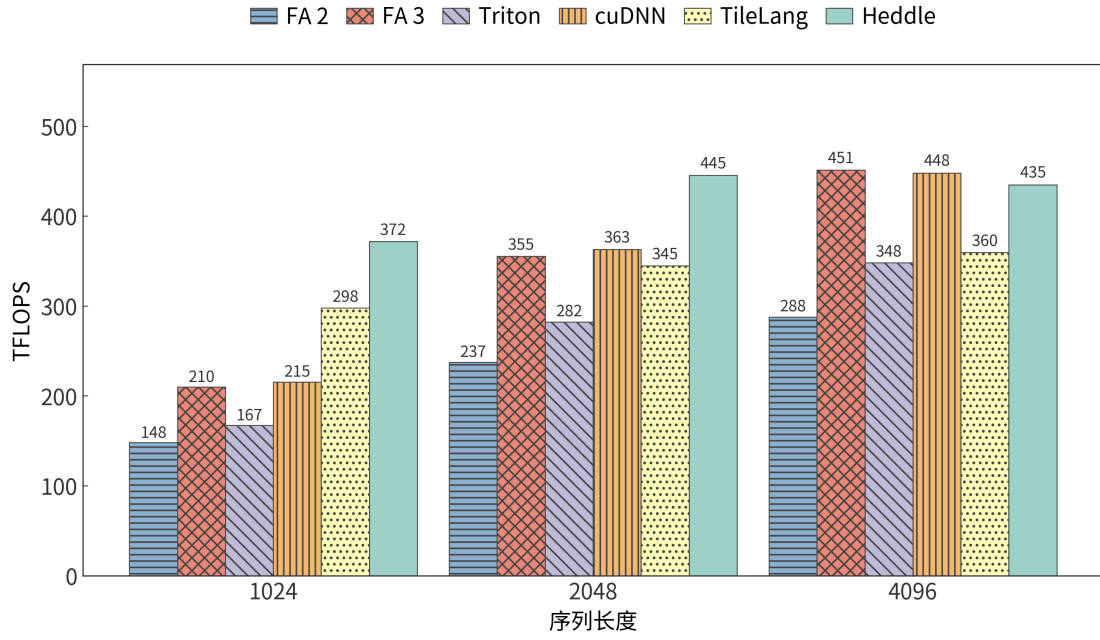
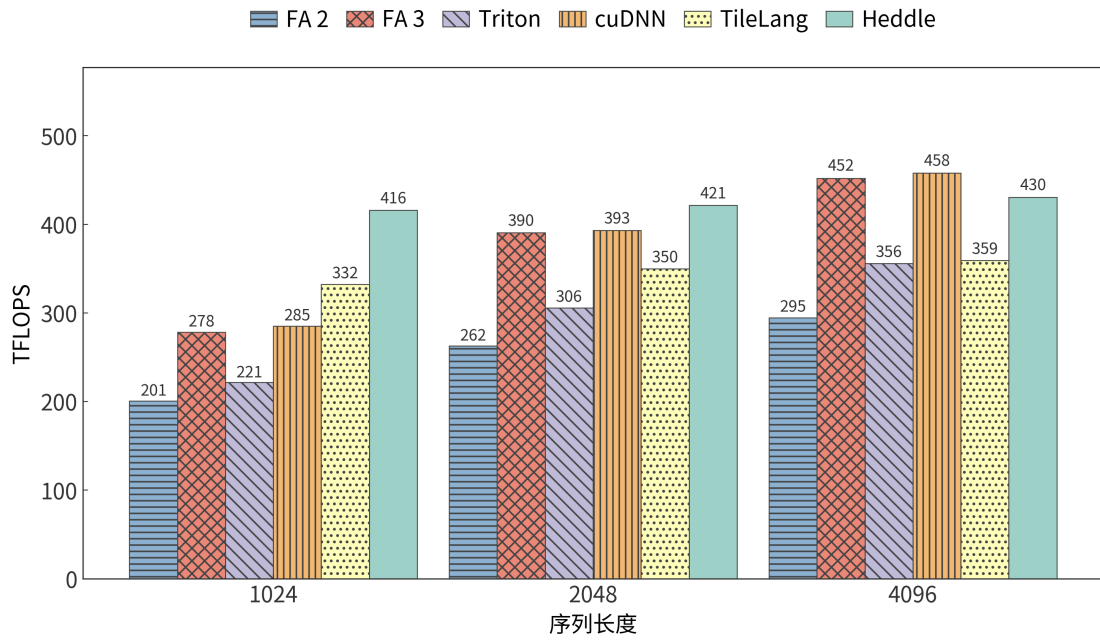
(a) $B=1$ (b) $B=4$

图 6-1 FlashAttention 前向性能对比

Figure 6-1 FlashAttention Forward Performance



(a) $B=1$



(b) $B=4$

图 6-2 FlashAttention 反向性能对比

Figure 6-2 FlashAttention Backward Performance

下高于 FlashAttention-3。短序列上加速比最高，长序列上加速比降至 1.45 倍，因为 FlashAttention-3 的 SASS 级优化在计算量充足时更充分发挥。

性能增益主要来自两部分。AutoMixed 选择分离式设计，将融合内核的 9 个同时存活张量拆分为两个内核各 7 到 8 个，共享内存占用降低至原来的 0.77 到 0.86 倍，为后续流水线调度留出资源空间。在此基础上，TRWS 将 dQ 写回从消费者关键路径中移出，使 dQ 写回与 dKdV 计算并行执行，并在反向传播上带来 1.22 倍的额外吞吐提升。

6.2.6 注意力变体通用性与优化步骤分解

注意力变体实验关注非标准注意力逻辑的支持能力。实验在 5 个变体上进行端到端评估和逐步优化分解。FlashAttention-2 和 FlashAttention-3 仅支持其中的标准注意力；其余变体涉及 logit softcapping、掩码或打分逻辑修改，表 6-10 中标为 N/A。

表 6-10 注意力变体端到端性能与库支持情况

Table 6-10 Attention Variant End-to-End Performance and Library Support

变体	PyTorch	torch.compile	Heddle	FA2	FA3
标准注意力	34	328	416	331 (1.26x)	608 (0.68x)
Gemma2	27	103	321	N/A [†]	N/A [†]
H2O	32	94	412	N/A [‡]	N/A [‡]
RoCo	26	90	414	N/A [‡]	N/A [‡]
SnapKV	32	94	414	N/A [‡]	N/A [‡]

[†] FA2/FA3 不支持 Gemma2 logit softcapping；[‡] FA2/FA3 不支持对应掩码或打分修改逻辑。

单位 TFLOPS， $B=1$ ， $H=32$ ， $T=4096$ ， $D=128$ 。括号为 Heddle 相对对应库的吞吐比；N/A 表示库接口不支持该变体。

本文系统相比 PyTorch 逐操作基线实现了 11.8 到 15.6 倍的加速，平均加速 13.2 倍；在标准注意力上相比 FlashAttention-2 加速 1.26 倍。表中的 torch.compile 列表示由 PyTorch Inductor/Triton 生成的代码。除标准注意力外，变体逻辑会改变打分或掩码，难以直接调用标准 SDPA 后端。AutoMixed 将注意力主循环与变体逻辑放入同一图级分区，减少中间张量物化；Heddle 模块在该内核内生成流水线和线程束特化调度，使各变体的吞吐保持在 321 到 416 TFLOPS。

表 6-11 的逐步优化结果显示，图级分区是主要加速来源。AutoMixed 将注意力主循环及其变体逻辑划入同一候选内核，减少注意力得分等中间张量的物化开销，可将 PyTorch 逐操作实现提升至 6.1 到 8.3 倍；分块搜索进一步将加速比提升至 11.6 到 15.6 倍。Heddle 双消费者在多数变体上的直接吞吐变化不超过 1%。在 FlashAttention 前向这类 QK 与 Softmax 链占主导、Tensor Core 发射资源接近饱和的配置中，双消费者主要拆分寄存器状态并提供跨 WarpGroup 执行形态，对稳态周期的直接压缩空间有限；这一结果与第 6.3.6 节的 NCU 分析一致。

各变体间加速比的差异主要取决于 PyTorch 逐操作基线的开销。RoCo 包含

表 6-11 注意力变体逐步优化加速比
Table 6-11 Attention Variant Step-wise Speedup

变体	基线	+torch.compile	+ 图级分区	+ 分块搜索	+Heddle 双消费者
标准注意力	1.0x	9.7x	6.1x	11.6x	11.8x
Gemma2	1.0x	3.7x	7.4x	13.2x	13.1x
H2O	1.0x	3.1x	6.7x	12.7x	12.6x
RoCo	1.0x	3.7x	8.3x	15.6x	15.6x
SnapKV	1.0x	3.1x	6.7x	12.6x	12.7x

加速比相对 PyTorch 逐操作基线； $B=4, H=32, T=4096, D=128$ 。torch.compile 为 PyTorch Inductor/Triton 生成代码。

额外的注意力打分修改逻辑，图级分区后获得最高的 15.6 倍加速；标准注意力的逐操作路径相对规整，加速比为 11.8 倍。

6.2.7 模型级推理性能验证

模型级实验进一步检查算子级收益能否传递到完整推理流程。本节模型级实验关注 prefill 阶段，即给定输入序列后一次性执行 Transformer 层计算、构建上下文状态的阶段；实验测量随序列长度变化的 prefill 延迟，不包含逐 token 生成的 decode 阶段，下文统一称为模型级推理延迟。实验涉及两个 7B 级 decoder-only 模型：LLaMA-2 7B 为 32 层，隐藏维度 $H=4096$ ，32 个注意力头，头维度 $D=128$ ，FFN 中间维度 $I=11008$ ；Qwen-2.5 7B 为 28 层，隐藏维度 $H=3584$ ，28 个 Query 头和 4 个 KV 头，头维度 $D=128$ ，FFN 中间维度 $I=18944$ 。两组实验的观察维度不同：LLaMA-2 7B 实验固定模型结构，比较 5 个注意力变体在不同序列长度下的端到端收益；Qwen-2.5 7B 实验固定标准注意力，补充主流模型真实形状下随序列长度变化的端到端结果。

PyTorch 基线使用 cuBLAS 执行全部线性层，Attention 部分回退为逐操作的 torch.matmul 与 softmax 组合实现，因 SDPA 不支持变体逻辑。Heddle 端将第 3.8 节的带宽代价模型应用于整个 Transformer 层。在图级分区生成的 Attention 内核基础上，AutoMixed 根据逐元素带宽模型识别并应用三项逐元素融合，分别是残差加与 RMSNorm 融合、Gate 与 Up 投影合并、SiLU 激活与逐元素乘法融合。

表 6-12 给出了 AutoMixed 带宽代价模型在不同序列长度下的预测。每层消除的 GMEM 流量与序列长度线性增长， $T=4096$ 时每层消除约 332 MB，32 层累计消除 10.4 GB。以 H100 峰值 HBM 带宽估算，带宽节省的时间下界从 $T=512$ 的 0.4 ms 增长至 $T=4096$ 的 3.3 ms。

表 6-13 和图 6-3 给出了 LLaMA-2 7B 上五类变体在四个序列长度下的延迟与加速比。所有变体在全部配置下均获得正收益，加速比随序列长度增大而持续增长。

加速比的增长源于两个互补的机制。图级分区生成的 Attention 内核将 PyTorch

表 6-12 AutoMixed 逐元素融合的带宽节省随序列长度的变化

Table 6-12 Bandwidth Savings of AutoMixed Elementwise Fusion vs. Sequence Length

序列长度	每层节省	全模型节省	带宽时间下界
512	42 MB	1.3 GB	0.4 ms
1024	83 MB	2.6 GB	0.8 ms
2048	166 MB	5.2 GB	1.7 ms
4096	332 MB	10.4 GB	3.3 ms

$B=1$, FP16, 32 层。带宽时间下界按 H100 峰值 HBM 带宽 3.35 TB/s 估算。各模式的逐项分析见表 3-3。

表 6-13 LLaMA-2 7B 模型级推理延迟与加速比

Table 6-13 LLaMA-2 7B End-to-End Inference Latency and Speedup

变体	序列长度 $T=512$			序列长度 $T=1024$			序列长度 $T=2048$			序列长度 $T=4096$		
	PyTorch	Heddle	加速比	PyTorch	Heddle	加速比	PyTorch	Heddle	加速比	PyTorch	Heddle	加速比
Attention	21.0	13.0	1.61x	44.4	23.7	1.87x	115.2	49.9	2.31x	359.7	105.6	3.41x
Gemma2	21.7	13.2	1.64x	48.6	23.9	2.03x	131.4	50.5	2.60x	420.7	106.3	3.96x
H2O	21.4	13.2	1.62x	45.6	24.0	1.90x	118.5	50.6	2.34x	371.3	106.7	3.48x
RoCo	22.3	13.6	1.64x	48.0	24.4	1.97x	127.3	51.3	2.48x	405.5	107.8	3.76x
SnapKV	21.5	13.4	1.61x	45.7	24.1	1.90x	118.6	51.3	2.31x	371.4	106.7	3.48x

单位 ms, 批大小 $B=1$, FP16。PyTorch 列为逐操作基线延迟。

的逐操作实现压缩为单一 CUDA 内核，消除了 $O(T^2)$ 规模的注意力得分矩阵的中间读写，收益随 T 的增大而二次增长。 $T=512$ 时 Attention 在总延迟中占比有限，加速比为 1.61 到 1.64 倍； $T=4096$ 时 Attention 占据主导，加速比扩大至 3.41 到 3.96 倍，平均 3.62 倍。逐元素融合消除了 FFN 路径上的冗余 GMEM 流量，其收益与 T 线性增长。两者叠加使长序列的加速比高于短序列。

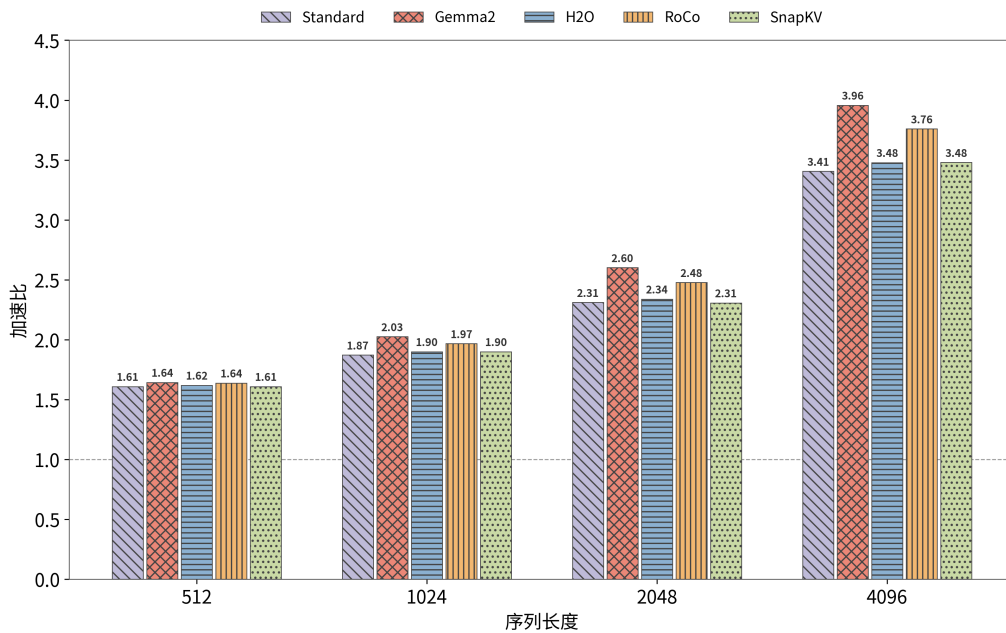


图 6-3 LLaMA-2 7B 模型级推理加速比

Figure 6-3 LLaMA-2 7B End-to-End Inference Speedup

图级分区生成的 Attention 内核对 $O(T^2)$ 中间张量的消除是模型级加速的主要来源，逐元素融合提供了与序列长度线性增长的补充收益。以表 6-12 的带宽代价模型预测为参照， $T=4096$ 时逐元素融合的带宽时间下界为 3.3 ms，仅占实测总加速量的很小比例。

五类变体之间的加速比差异取决于 PyTorch 基线的实现效率。Gemma2 取得最高加速比，其 logit softcapping 在 PyTorch 中引入了额外的 tanh 计算与 GMEM 读写，使基线延迟最高；图级分区后的 Attention 内核把 softcapping 放入主循环后消除了这部分开销。标准注意力的加速比最低，因为标准注意力的逐操作路径本身效率相对较高，中间张量的形状规则、访存模式连续。各变体的 Heddle 延迟差异不超过 2%，表明分区后的 Attention 内核在引入变体逻辑后未产生显著的性能退化。

为进一步验证不同模型结构下的端到端收益，表 6-14 报告了 Qwen-2.5 7B 完整 Transformer 的标准注意力模型级推理延迟。Heddle+Fusion 配置将 Attention 替换为 Heddle 编译的 FlashAttention 内核，并启用逐元素融合；PyTorch 基线使用 eager 模式，Attention 部分回退为逐操作的 torch.matmul+softmax 组合实现。

表 6-14 Qwen-2.5 7B 模型级推理延迟 (ms)

Table 6-14 Qwen-2.5 7B End-to-End Inference Latency (ms)

序列长度	PyTorch	torch.compile	Heddle+Fusion	vs PyTorch	vs compile
512	18.97	13.00	12.17	1.56×	1.07×
1024	39.83	26.44	23.49	1.70×	1.13×
2048	97.33	57.28	47.69	2.04×	1.20×
4096	292.25	139.99	103.68	2.82×	1.35×

$B=1$, FP16, H100 SXM 80 GB。torch.compile 使用 PyTorch Inductor/Triton 后端，mode="reduce-overhead"。

Qwen-2.5 7B 的加速比同样随序列长度单调增长。相比 PyTorch eager 基线， $T=512$ 时加速 1.56 倍， $T=4096$ 时达到 2.82 倍；相比 torch.compile 基线， $T=512$ 时加速 1.07 倍， $T=4096$ 时达到 1.35 倍。该收益来自两类互补优化：Heddle 编译的 FlashAttention 内核将逐操作 Attention 压缩到单一主循环中，避免注意力得分矩阵等 $O(T^2)$ 中间张量的全局内存物化；逐元素融合进一步减少残差、归一化和 FFN 激活路径上的冗余读写。随着序列长度增加，Attention 计算和中间张量流量在端到端延迟中的占比上升，因此上述两类优化的整体收益同步放大。

6.3 消融实验与机制验证

前一节给出完整系统的性能结果，本节进一步拆分收益来源。实验分别考察 FGWS 基础设施、AutoPiped 与 AutoMixed 的配置搜索、联合调度的资源检查，以及 Heddle 中双消费者和 TRWS 的执行模型扩展。

6.3.1 全栈功能消融

全栈消融从两个方向观察不同优化的贡献。GEMM 实验逐步叠加优化，展示完整性能如何形成；FlashAttention 实验逐一移除关键机制，观察性能和编译通过率的变化。

表 6-15 GEMM 全栈优化叠加式消融

Table 6-15 Stacking Ablation of Full-Stack Optimizations on GEMM

优化步骤	配置说明	延迟 (ms)	TFLOPS	加速比
无流水线 WS 基线	128x128x32, t=128, s=0	8.454	130.1	1.00x
叠加流水线	128x128x64, t=128, s=2	2.849	386.0	2.97x
叠加 AutoPiped	128x128x64, t=128, s=3	2.474	444.4	3.42x
叠加 AutoMixed	128x256x64, t=256, s=3	1.738	632.7	4.86x
叠加 Heddle 调度	128x256x64, t=256, s=3	1.560	704.8	5.42x
cuBLAS	—	1.709	643.3	4.94x

GEMM $M=N=K=8192$; $bM \times bN \times bK$, t , s 分别为分块形状、线程数和流水线级数。基线为 TileLang 默认 WS 配置，流水线级数 $s=0$ 表示不启用软流水线；后续各行依次叠加对应优化。Heddle 调度为异步 WGMMMA 与提前释放。

表 6-16 FlashAttention 前向全栈机制移除式消融

Table 6-16 Removal Ablation of Full-Stack Mechanisms on FlashAttention Forward

配置	吞吐 (TFLOPS)	编译通过率	影响
完整 Heddle	551	100%	—
移除异步调度	504	100%	异步 WGMMMA 与提前释放贡献约 1.09 倍
移除 FGWS	编译失败	50%	全局屏障使一半配置资源超限
移除 AutoMixed	208	100%	失去分块、布局与分区搜索
移除联合求解	551	75%	寄存器溢出配置无法修正

FlashAttention 前向, $B=4, H=32, T=4096, D=128$; 本表不包含 TRWS。

GEMM 的流水线配置、分块搜索与异步调度叠加消融（表 6-15）中，流水线带来 2.97 倍的基础加速，AutoPiped 进一步提升至 3.42 倍，AutoMixed 将总加速比推至 4.86 倍。在分块与线程配置不变的前提下，Heddle 调度仅通过异步 WGMMMA 和提前释放两项编译变换即加速 1.11 倍，总加速比达到 5.42 倍。

FlashAttention 前向的调度、分区与资源检查移除消融（表 6-16）中，依次移除异步调度、FGWS、AutoMixed 分区搜索和联合求解。移除异步调度后性能从 551 降至 504 TFLOPS，对应约 1.09 倍的增量。移除 FGWS 后，全局屏障使一半配置超出资源上限。移除 AutoMixed 后性能降至 208 TFLOPS，说明分块、布局与分区搜索是该实验中的主要增量来源。移除联合求解后最优配置性能不变，但可编译配置比例从 100% 降至 75%，表明该机制主要提升配置空间的覆盖率。

6.3.2 AutoPiped 的流水线级数验证

本节对应第 4 章的流水线参数生成方法。实验通过流水线级数扫描，观察 AutoPiped 推断出的配置是否能够带来有效重叠。

流水线级数扫描对比了三种操作数布局：共享内存-共享内存布局、寄存器-共享内存布局以及带中间共享内存缓冲的寄存器布局。实验固定分块大小，扫描 2 到 5 级流水线，加速比以同一布局下的 2 级流水线为基准。

表 6-17 不同模式随流水线级数的扩展性

Table 6-17 Pipeline Stage Scalability across Modes

流水线级数	SS		RS		RS_SMEM	
	延迟 (ms)	加速比	延迟 (ms)	加速比	延迟 (ms)	加速比
2	0.5261	1.00x	0.9424	1.00x	0.5472	1.00x
3	0.3781	1.39x	0.9496	0.99x	0.4700	1.16x
4	0.3153	1.67x	0.9440	1.00x	0.4546	1.20x
5	0.3155	1.67x	0.9494	0.99x	0.4697	1.17x

$M=N=K=4096$ ；加速比以同一布局下的 2 级流水线为基准。SS 为共享内存-共享内存布局，RS 为寄存器-共享内存布局，RS_SMEM 为带中间共享内存缓冲的寄存器布局。

寄存器-共享内存布局的延迟几乎不随流水线级数变化。该布局下操作数直接从寄存器读取，缺少全局内存到共享内存的异步拷贝阶段，增加流水线级数带来的收益有限。共享内存-共享内存布局随级数增加性能持续提升，在 4 级流水线时达到最优，表明 TMA 异步加载得到了充分利用。中间共享内存缓冲是 AutoMixed 在检测到寄存器操作数后引入的补充方案。系统先通过 TMA 将数据异步加载到共享内存，再从共享内存读入寄存器参与 WGMMA 计算，为寄存器操作数补充可流水化的数据搬运路径，将性能恢复至相近水平。

分组策略对比中，AutoPiped 的默认启发式把源码中相邻的同类操作归入一组，再按组间依赖和缓冲区读写确定执行顺序。在本文测试的 FlashAttention 前向上，默认分组与手动分组的延迟差异在 5% 以内。CP-SAT 路径在默认分组基础上检查寄存器和共享内存上限，用于消费者依赖更复杂的候选。

6.3.3 AutoMixed 的资源感知决策验证

本节对应第 3 章的资源感知图级决策。实验先考察不同注意力头维度下的操作数布局选择，测试对象为 FlashAttention 前向的 $P \times V$ GEMM，注意力头维度取 64、128 和 256，流水线级数取 1 到 4。头维度直接影响寄存器-共享内存布局中左操作数 P 的寄存器占用，因此是决定布局可行性的关键变量。

在注意力头维度为 64 时，两类操作数布局均可编译且无寄存器溢出，其中寄存器-共享内存布局始终快约 1.10 倍。NCU 分析表明，该布局下 PV GEMM 的左操作数 P 驻留于寄存器，减少了共享内存-共享内存布局中将 P 写回共享内存再读取的额外流量，Tensor Core 利用率从 25.1% 提升至 28.0%。

在注意力头维度为 128 时，两类布局仅在 1 级流水线时可行，流水线级数不小于 2 时均因共享内存超出 H100 单 SM 的 228 KB 上限而编译失败。共享内存-共享内存布局的共享内存占用高于寄存器-共享内存布局，因此更早触发共享内存超限。注意力头维度为 256 时所有配置均超限。

表 6-18 两类操作数布局对比
Table 6-18 Operand Layout Comparison

注意力头维度	流水线级数	RS 延迟 (ms)	RS TFLOPS	SS 延迟 (ms)	SS TFLOPS
64	1	0.526	261	0.576	239
64	2	0.519	265	0.574	239
64	4	0.519	265	0.577	238
128	1	0.746	369	0.803	342
128	2	SMEM 超限	—	SMEM 超限	—
256	1	SMEM 超限	—	SMEM 超限	—

$B=4, H=32, T=2048$ ，分块 128×128 ，线程数 256。RS 为寄存器-共享内存布局，SS 为共享内存-共享内存布局。

两类操作数布局的选择取决于注意力头维度、流水线级数与分块形状。在较小的头维度，寄存器-共享内存布局可带来 1.07 到 1.10 倍的性能收益；而在高头维度或深流水线时，AutoMixed 需要切换布局或缩小分块以满足资源上限。

隐式寄存器溢出实验显示，仅观察寄存器数量不足以判断配置质量。表 6-19 在 FlashAttention 前向只改变线程数，ptxas 报告的溢出行为和性能差异明显。

线程数为 128 时，ptxas 报告寄存器数为 240，但同时出现本地堆栈和数百字节的溢出写入，性能仅为 174 TFLOPS。线程数为 256 时，每线程寄存器降至 168，溢出写入消失，性能提升至 370 TFLOPS。二者仅线程数不同，性能相差 2.1 倍。

128 线程下增加流水线级数会进一步拉长延迟：2 级流水线时延迟为 2.61 ms，3 级升至 3.23 ms，4 级直接编译失败。溢出字节数在各级数下均超过 800 B，说

表 6-19 线程数对寄存器溢出与性能的影响

Table 6-19 Thread Count Impact on Register Spills and Performance

线程数	级数	寄存器数	堆栈 (B)	溢出 (B)	延迟 (ms)	TFLOPS
128	1	240	608	1196	1.580	174.0
128	2	240	376	900	2.605	211.0
128	3	240	352	872	3.230	170.2
128	4	240	376	900	编译失败	
256	1	168	0	0	0.742	370.4
256	2	168	0	0	0.746	368.4

FlashAttention 前向，分块大小 128×128 ，注意力头维度为 128。128 线程行采用 $B=1, H=64$ 配置；256 线程行采用寄存器-共享内存布局。

明寄存器压力始终存在，更深的流水线并未缓解反而因活跃区间增加导致性能恶化。AutoMixed 通过解析 ptxas 编译反馈捕获这类静态寄存器计数之外的后端溢出。

为区分分块搜索和下游执行模型的贡献，实验在 FGWS 细粒度屏障的基础上逐步叠加优化层。L0 为启用 FGWS 后的默认线程束特化配置。

表 6-20 FlashAttention 前向配置层叠消融

Table 6-20 FlashAttention Forward Configuration Stacking Ablation

优化层	$T=1024$	$T=2048$	$T=4096$	$T=8192$	增量
L0 FGWS 基线，查询分块 128	179	196	208	213	基线
L1 +AutoPiped 流水线推断	179	198	207	213	$\leq 1\%^*$
L2 +AutoMixed 分块搜索	327	375	394	401	1.83–1.93x
L3 +Heddle 双消费者	325	379	390	401	$\leq 1\%$
FlashAttention-2 (基线)	325	322	331	338	—
FlashAttention-3 (基线)	598	648	608	637	—

FlashAttention 前向， $B=4, H=32, D=128$ ；增量相对上一优化层。* L1 固定分块和流水线级数，仅加入 AutoPiped 推断出的组间顺序。

L1 固定分块和流水线级数，只加入 AutoPiped 生成的组间顺序，因此边际收益较小。若同时调整阶段和组间顺序，生产者预取距离也会变化，结果不再是本表的单因素增量。L2 的 AutoMixed 分块搜索带来最大增量，资源模型和调优反馈将查询分块从 128 降至 64、键值分块从 64 增至 128，合计加速 1.83 到 1.93 倍。L3 的双消费者用于验证跨 WarpGroup 数据传递和代价模型，对该前向配置的直接吞吐影响不超过 1%。L0 到 L3 的总加速比为 1.88 倍。

6.3.4 联合调度的配置通过率、求解效率与硬件指标

本节验证第 5 章的联合调度路径。更深的流水线级数能提升稳态吞吐，也会延长中间值活跃区间并增加寄存器压力。实验在 FlashAttention 前向上扫描同一组分块、线程数和流水线级数组合，比较不同调度策略的编译结果与最低延迟。表中的“溢出失败”包含两种情况：ptxas 在寄存器占用率约束下无法完成分配而中止编译，以及编译成功但溢出字节数过高导致性能不可接受而被 AutoMixed 标记为不可行。

扩大调度自由度需要配套资源约束检查。TileLang 手动线程束特化在 48 个配置中有 6 个因寄存器溢出失败；AutoPiped 进一步调整流水线分组与执行顺序后，最低延迟降至 13.0 ms，但失败配置增加到 12 个。Heddle 将寄存器容量与活跃区间约束纳入联合调度，在同样的 48 个配置上消除了溢出失败，并保留最低延迟 12.5 ms 的配置。

表 6-22 汇总了 HeddleScheduler 在各类算子上的实测求解时间。自适应超时阈值按 $T_{\text{timeout}} = \min(500 + 100 \times |V|, 15000)$ 计算，其中 $|V|$ 为消费者节点数。

表 6-21 FlashAttention 前向不同调度策略的可行性与最佳延迟

Table 6-21 Feasibility and Best Latency of FlashAttention Forward Scheduling Strategies

策略	扫描配置数	溢出失败配置	最低延迟 (ms)
无流水线基线	16	0	25.0
TileLang 手动线程束特化	48	6	15.0
AutoPiped 推断调度	48	12	13.0
Heddle 联合调度	48	0	12.5

扫描分块大小、流水线级数 1-3 和线程数 128/256；溢出失败指 ptxas 报告寄存器溢出并中止编译。

表 6-22 约束求解时间统计

Table 6-22 Constraint Solve Time Statistics

算子	节点数	求解时间 (ms)	超时阈值 (ms)	结果
GEMM	3	3.1	800	可行
Grouped GEMM	5	5.7	1000	可行
Dequant GEMM	6	8.4	1100	可行
Conv2D	8	18.7	1300	可行
FlashAttention 前向	12	66.3	1700	可行
FlashAttention 反向 2-分离方案	16	130.7	2100	可行
FlashAttention 反向完全融合方案	22	400	2700	资源超限

六类实际算子的节点数在 3 到 16 之间，全部在超时阈值内完成求解，中位求解时间 12 ms，最大 131 ms。FlashAttention 反向的完全融合方案包含 22 个节点，当前 CP-SAT 实现可在超时阈值内返回资源超限判定，说明约束模型能够处理比主实验路径更复杂的消费者依赖图。

表 6-23 对比了求解器预测值与 GPU 编译反馈，包括启动间隔 II 、调度总时长和寄存器峰值。

表 6-23 约束模型预测结果

Table 6-23 Constraint Model Predictions Results

算子	节点数	预测启动间隔 II	预测调度总时长	预测寄存器
GEMM	3	27	82	168
Grouped GEMM	3	27	82	168
FlashAttention 前向	12	87	135	168
FlashAttention 反向 3-分离 dK	5	35	82	96
FlashAttention 反向 2-分离 dKdV	7	35	109	128
FlashAttention 反向融合 dQdKdV	22	—	—	192

GEMM 为 $M=N=K=4096$ ；FA 为 $B=4, H=32, T=4096, D=128$ 。寄存器单位为寄存器/线程。

所有可行配置的预测寄存器峰值均与 ptxas 实测一致。约束模型在编译前依据 WGMMMA 累加器公式估计寄存器峰值，AutoMixed 再结合 ptxas 编译反馈捕获隐式溢出，二者共同覆盖静态估算和后端分配两类风险。FlashAttention 反向

的完全融合方案预测寄存器为每线程 192 个，超出当前配置下每线程 128 个寄存器的 SM 占用率上限，求解器据此判定资源超限。值得注意的是，启动间隔 II 与调度总时长是内核内部调度量，分区方案评估仍需结合资源上限、编译反馈和实测结果。

6.3.4.1 编译耗时分析

表 6-24 报告了各优化模块在典型算子上的单内核端到端编译耗时³。四列分别对应不启用任何调度 pass 的基线、累加 AutoPiped、累加 AutoMixed 以及完整 Heddle。

表 6-24 单内核端到端编译耗时 (s)

Table 6-24 Single-Kernel End-to-End Compilation Time (s)

算子	TileLang	+ AutoPiped	+ AutoMixed	+ Heddle
GEMM ($M=N=K=4096$)	2.68	2.70	2.74	2.71
GEMM ($M=N=K=8192$)	2.73	2.72	2.69	2.70
FA FWD ($T=4096$)	3.14	3.12	3.05	3.60
FA FWD ($T=8192$)	3.13	3.13	3.05	3.58
Conv2D	3.21	3.20	3.18	3.19

GEMM 配置 $128 \times 128 \times 64$ ，3 级流水线，128 线程；FA FWD 配置 $B=4, H=32, D=128$ ，分块 128×64 ，2 级流水线，128 线程；Conv2D 为 $C=256, H=W=64, K=3$ 。每项取 3 次独立编译的中位数。

单内核编译耗时为 2.7 到 3.6 s，最耗时的部分是 TVM IR 降级与 ptxas 后端编译，这部分与算子的问题规模无关。相对 TileLang 基线，AutoPiped 的语义谓词推断和 AutoMixed 的资源感知变换在编译流水线中的额外开销均不超过 0.1 s。Heddle 仅在 FlashAttention 等消费者依赖图较复杂的算子上增加约 0.5 s，包含 CP-SAT 求解和双消费者变换，在 GEMM 和 Conv2D 等单一消费者算子上无可测增量。

AutoMixed 全流程的主要开销来自候选配置遍历，而非单次编译中的 pass 开销。以 FlashAttention 前向为例，枚举产生 40 个候选配置，枚举与资源剪枝耗时不足 0.1 ms，随后每个候选需经过一次完整编译以获取 ptxas 寄存器反馈，总计约 $40 \times 3 \text{ s} \approx 120 \text{ s}$ 。该开销与编译器自动调优流程同阶，且编译结果可缓存复用，仅在首次编译或配置变更时产生。

表 6-25 给出了 Nsight Compute 采集的关键硬件指标。

FlashAttention 反向的 dKdV 内核计算吞吐约为 76.5%，属于计算受限型算子。启用 TRWS 后，dKdV 单内核的 NCU 指标与未启用时几乎相同，但整体 TFLOPS 从 355 提升至 436。收益主要来自 dQ 写回与 dKdV 计算的并行重叠，dKdV 内核本身的利用率变化不大。

表 6-25 NCU 硬件指标对比
Table 6-25 NCU Hardware Metrics

配置	寄存器/线程	占用率	计算吞吐	访存吞吐	TFLOPS
s=1 TileLang	44	50.6%	76.5%	17.2%	354
s=1 Heddle	44	50.6%	76.2%	17.2%	355
s=1 Heddle + TRWS [†]	44	50.6%	76.3%	17.2%	436

FlashAttention 反向, $B=4, H=32, T=4096, D=128$ 。NCU 指标为单内核采集, TFLOPS 为端到端测量; 关键配置重复采集 3 次确认指标趋势一致; [†] 采集 dKdV 内核, 端到端吞吐包含 TRWS 写回重叠收益。

6.3.5 FlashAttention 反向的多角色优化

本节评估 TRWS 对 FlashAttention 反向写回路径的作用。dQ 梯度需要跨块累加, 原始实现使用 atomic_add 完成, 会阻塞消费者计算流水线; 实验逐步加入 TMA 规约加和 TRWS, 观察写回剥离带来的收益。

两角色基线配合普通 atomic_add 达到 356.2 TFLOPS。将 atomic_add 替换为 TMA 规约加后性能提升至 395.0 TFLOPS, 加速 1.11 倍; 继续启用 TRWS 后, dQ 写回被剥离到生产者内的独立线程束, 与下一次迭代的 dKdV 计算并行, TFLOPS 提升至 435.4, 累计加速 1.22 倍。在这一固定配置下, TRWS 版本达到 FlashAttention-2 的 1.56 倍和 FlashAttention-3 的 87.5%。

表 6-26 FlashAttention 反向累进优化效果
Table 6-26 FlashAttention Backward Progressive Optimization

配置	TFLOPS	加速比	vs FA2	vs FA3
两角色与普通 atomic_add	356.2	—	1.28x	71.5%
两角色与 TMA 规约加	395.0	+10.9%	1.42x	79.3%
TRWS 与 TMA 规约加	435.4	+22.2%	1.56x	87.5%
FA2	278.9	—	—	56.0%
FA3	497.8	—	1.78x	—

单位 TFLOPS, $B=4, H=32, T=4096, D=128$; 加速比分别相对两角色与普通 atomic_add、FA2 和 FA3。

6.3.6 双消费者调度消融与代价模型验证

双消费者实验关注两件事: 跨 WarpGroup 中转本身是否足够高效, 以及代价模型能否选择合适的启用条件。表 6-27 记录了从朴素中转到完整协议的逐步消融。共享内存跨步填充贡献最大, 在键值分块为 128 的配置下将性能从 0.64x 提升至 1.01x, 表明存储体冲突是跨 WarpGroup 数据传递的主要瓶颈。键值分块为 64 时, PV WGMMA 规模较小, 同步成本占比更高, 代价模型因此选择不启用双消费者。

键值分块为 128 时, 双消费者的直接吞吐增量约为 1%。此时 QK+Softmax 关键路径长于 PV 侧, PV 重叠对稳态周期的压缩空间有限; 同时乒乓协议引入固定同步成本, Tensor Core 发射资源也已接近饱和。双消费者在该前向配置中

表 6-27 双消费者逐步优化消融

Table 6-27 Dual-Consumer Step-by-Step Ablation

配置	bN=64	加速比	bN=128	加速比
单 WarpGroup 基线	335	1.00x	398	1.00x
标量拷贝，无重叠	192	0.57x	152	0.38x
+ 乒乓双缓冲	226	0.68x	173	0.43x
+ 128 位向量化	310	0.93x	256	0.64x
+ 共享内存跨步填充	316	0.95x	401	1.01x

单位 TFLOPS，批大小 $B=4$ 、头数 $H=32$ 、序列长度头维度 $T=4096$ 、头维度 $D=128$ ；加速比相对单 WarpGroup 基线。

主要体现为寄存器状态拆分和跨 WarpGroup 中转能力。

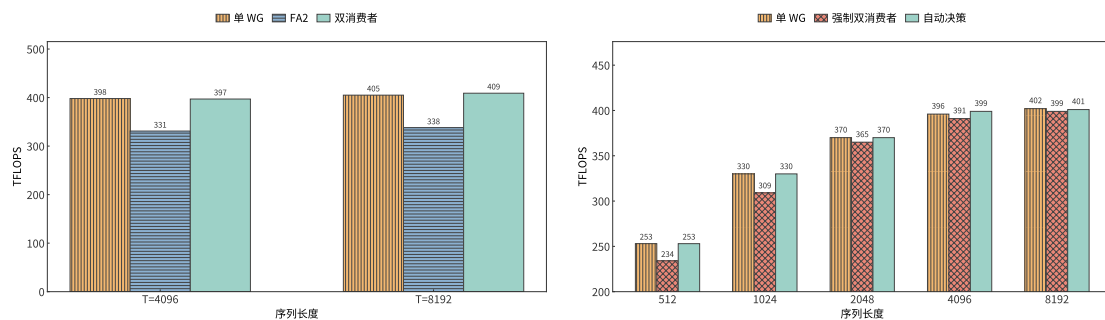
表 6-28 和图 6-4 进一步检查代价模型的启用决策。短序列下，双消费者的同步与中转成本占比较高，代价模型选择单 WarpGroup；长序列下同步开销被均摊，模型启用双消费者并获得稳定收益。在全部测试配置上，模型决策与最优策略的偏差不超过 3%。注意力头维度为 256 时，双消费者通过拆分寄存器状态覆盖了单 WarpGroup 配置未通过编译的形状。

表 6-28 双消费者代价模型验证

Table 6-28 Dual-Consumer Cost Model Validation

序列长度	单 WarpGroup	强制双消费者	模型决策	决策结果
512	253	234 (-7%)	253	选择单 WarpGroup
1024	330	309 (-6%)	330	选择单 WarpGroup
4096	396	391	399	启用双消费者
8192	402	399	401	自适应

单位 TFLOPS；批大小 $B=4$ 、头数 $H=32$ 、头维度 $D=128$ 、bN=128。



(a) 不同序列长度下单消费者与双消费者的吞吐对比

(b) 代价模型决策与强制启用/禁用的对比

图 6-4 双消费者性能与代价模型决策验证

Figure 6-4 Dual-Consumer Performance and Cost Model Validation

6.4 本章小结

本章在 NVIDIA H100 上对 AutoMixed、AutoPiped 和 Heddle 三个编译模块进行了系统评估。端到端实验表明，完整系统在六类核心算子的所测配置上均超越 TileLang 手动配置。相对外部厂商或专用手写库，性能关系随算子而变化：GEMM 最高达到 cuBLAS 的 96% 但平均为 0.87 倍，Dequant GEMM 达到 Marlin 的 0.78–0.82 倍，Conv2D 在部分所测配置上超越 cuDNN，FlashAttention 前向相比 FlashAttention-2 加速 1.44 到 1.61 倍，FlashAttention 反向在图 6-2 的配置上相比 TileLang 加速 1.33 到 1.45 倍并高于 FlashAttention-3。

按模块分解，AutoMixed 的主要作用是提高可达到的内核结构上限。FlashAttention 前向配置叠加实验中，AutoMixed 将查询分块、键值分块和线程参数调整到更高吞吐区域，带来 1.83 到 1.93 倍增量；FlashAttention 反向中，AutoMixed 选择 2-分离方案，在消除溢出的同时保留 dK 与 dV 的共享输入结构。模型级推理实验中，AutoMixed 的逐元素融合每层消除约 332 MB 的 GMEM 冗余流量，使 LLaMA-2 7B 的 5 个注意力变体在 $T=4096$ 时加速 3.41 到 3.96 倍，平均加速 3.62 倍；在 Qwen-2.5 7B 标准注意力模型级推理实验上，相比 PyTorch 和 torch.compile 分别加速 2.82 倍和 1.35 倍。

AutoPiped 的主要作用是把循环体中的生产者–消费者关系转化为可执行的流水线注解。GEMM 叠加消融中，AutoPiped 将流水线基线的加速比从 2.97 倍提升至 3.42 倍；FlashAttention 前向固定分块案例中，三缓冲配置相对 TileLang 默认配置将延迟降低 19.9%。Heddle 的主要作用是在线程束特化执行层提高可行性和稳态重叠质量。FGWS 保障了缓冲区级同步，联合求解将 FlashAttention 前向配置扫描的编译通过率从 75% 提升至 100%；TRWS 在 FlashAttention 反向上带来 1.22 倍的吞吐提升，双消费者代价模型决策与最优策略偏差不超过 3%。

第7章 总结与展望

7.1 工作总结

本文围绕异步 GPU 架构上的流水线内核编译问题,设计并实现了 AutoMixed、AutoPiped 和 Heddle 三个编译模块,并在 NVIDIA Hopper 架构上进行验证。三个模块分别处理图级内核结构、循环级流水线配置和线程束特化执行三个层次的决策。

AutoMixed 负责在图级确定内核结构。它在 Tile IR 生成前选择算子融合边界、WGMMA 操作数布局 and 候选分块形状,把寄存器压力、共享内存占用和 GMEM 带宽收益提前纳入搜索。FlashAttention 反向传播实验中,AutoMixed 选择 dKdV 与 dQ 的 2-分离方案,避免融合内核中的寄存器溢出,并保留 dK 与 dV 的共享输入结构。模型级推理实验中,AutoMixed 还识别了残差加与 RMSNorm、Gate 与 Up 投影、SiLU 与逐元素乘法三类带宽受限融合模式,减少层间中间张量的 GMEM 往返。

AutoPiped 负责在循环级生成异步流水线配置。它从 Tile IR 循环体出发,分析语句访存模式、共享内存读写关系和生产者-消费者依赖,生成 TileLang 后端所需的 group、order 和 stage 三类注解。FlashAttention 前向案例表明,三缓冲配置能够拉开 TMA 预取、QK 与 PV WGMMA、Softmax 与 Cast 之间的距离,减少稳态流水线中的等待。GEMM 叠加消融中,AutoPiped 将流水线基线的加速比从 2.97 倍提升至 3.42 倍。

Heddle 负责在线程束特化执行层实现上述流水线配置。FGWS 将同步粒度从全局生产者-消费者边界细化到单个共享缓冲区,使 QK 等消费者不必等待无关的 V 加载完成。消费者调度器根据依赖图生成语句顺序和屏障位置,并在复杂候选上检查 Tensor Core 发射冲突、寄存器峰值和共享内存上限。异步 WGMMA 与提前释放进一步缩短生产者和消费者之间的等待;TRWS 则把 FlashAttention 反向中的 dQ 写回移出消费者关键路径,使写回与 dK/dV 计算重叠。

实验覆盖六类算子和 5 个注意力变体。单算子结果显示,完整系统在所测配置上均高于 TileLang 等开源编译器基线;相对厂商或专用手写库时,GEMM 最高达到 cuBLAS 的 96% 但平均为 0.87 倍,Dequant GEMM 平均达到 Marlin 的 0.80 倍,Conv2D 在部分配置上超过 cuDNN,FlashAttention 反向在所测配置上高于 FlashAttention-3。FlashAttention 前向相比 FlashAttention-2 加速 1.44 到 1.61 倍,FlashAttention 反向在所测配置上相比 TileLang 加速 1.33 到 1.45 倍。端到端实验中,注意力变体相比 PyTorch 逐操作基线平均加速 13.2 倍;在 Llama2-7B 模型级推理中,序列长度 4096 时平均加速 3.62 倍。

7.2 未来工作

后续工作主要包括以下方向。

一是跨平台验证与适配。当前实验集中在 NVIDIA Hopper 平台。AutoPiped 的语义谓词推断和 AutoMixed 的图级资源决策不依赖 Hopper 特有硬件参数，在架构层面具有可移植性；Heddle 的约束模型需要适配目标架构的功能单元延迟和寄存器文件容量。Blackwell 引入张量内存（TMEM）和新的 Tensor Core 数据通路，流水线调度需要同时考虑新的片上存储层级、同步原语和指令延迟。后续工作将在 Blackwell 以及其他支持异步流水线的平台上进行验证，Heddle 的约束模型可以扩展资源类型和容量约束，并重新标定架构相关参数。

二是扩展 WarpGroup 迭代交错。当前双消费者采用 QK 与 PV 的功能分工，重点验证跨 WarpGroup 中转和乒乓同步协议。FlashAttention-3 采用不同思路，将两个 WarpGroup 分配到不同的 Q 行子集，并通过迭代偏移重叠 QK 与 PV。后续可以把这类行分裂与迭代偏移纳入 Heddle 的多角色候选空间，使代价模型在功能分工和行分裂之间做选择。

三是增强跨层级调度信息传递。当前调度决策主要以 TIR 注解和屏障提示传递给后端，部分指令级顺序仍可能在后续降级中被重新安排。后续可以把关键屏障位置、WGMMA 发射顺序和等待深度作为更强的后端约束，使高层调度结果在机器码层级得到更稳定的保留。

四是扩展到多卡流水线。在多 GPU 训练和推理中，NVLink/NCCL 通信可以与本地计算重叠，例如梯度归约与下一层前向计算并行。Heddle 的依赖图和资源约束可以把通信链路建模为新的功能单元，用同一套模型描述片上计算、片上访存和跨卡通信之间的重叠关系。

参考文献

- [1] Shah J, Bikshandi G, Zhang Y, et al. Flashattention-3: Fast and accurate attention with asynchrony and low-precision [J/OL]. Advances in Neural Information Processing Systems, 2024, 37: 68658-68685. DOI: [10.48550/arXiv.2407.08608](https://arxiv.org/abs/2407.08608).
- [2] Touvron H, Lavril T, Izacard G, et al. Llama: Open and efficient foundation language models [J/OL]. arXiv preprint arXiv:2302.13971, 2023. DOI: [10.48550/arXiv.2302.13971](https://arxiv.org/abs/2302.13971).
- [3] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need [J/OL]. Advances in neural information processing systems, 2017, 30. DOI: [10.48550/arXiv.1706.03762](https://arxiv.org/abs/1706.03762).
- [4] Choquette J. Nvidia hopper h100 gpu: Scaling performance [J/OL]. IEEE Micro, 2023, 43 (3): 9-17. DOI: [10.1109/MM.2023.3256796](https://doi.org/10.1109/MM.2023.3256796).
- [5] NVIDIA. cuBLAS: CUDA basic linear algebra subroutines [EB/OL]. 2025. <https://developer.nvidia.com/cuBLAS>.
- [6] NVIDIA. cuDNN: CUDA deep neural network library [EB/OL]. 2025. <https://developer.nvidia.com/cudnn>.
- [7] Dao T, Fu D, Ermon S, et al. Flashattention: Fast and memory-efficient exact attention with io-awareness [J/OL]. Advances in neural information processing systems, 2022, 35: 16344-16359. DOI: [10.48550/arXiv.2205.14135](https://arxiv.org/abs/2205.14135).
- [8] Dao T. FlashAttention-2: Faster attention with better parallelism and work partitioning [C/OL]//The Twelfth International Conference on Learning Representations (ICLR). 2024. DOI: [10.48550/arXiv.2307.08691](https://arxiv.org/abs/2307.08691).
- [9] Chen T, Moreau T, Jiang Z, et al. TVM: An automated end-to-end optimizing compiler for deep learning [C/OL]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). 2018: 578-594. DOI: [10.48550/arXiv.1802.04799](https://arxiv.org/abs/1802.04799).
- [10] Chen T, Zheng L, Yan E, et al. Learning to optimize tensor programs [C/OL]//Advances in Neural Information Processing Systems. 2018. <https://arxiv.org/abs/1805.08166>. DOI: [10.48550/arXiv.1805.08166](https://arxiv.org/abs/1805.08166).
- [11] Zheng L, Jia C, Sun M, et al. Ansor: generating high-performance tensor programs for deep learning [C/OL]//OSDI'20: Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation. USA: USENIX Association, 2020. DOI: [10.48550/arXiv.2006.06762](https://arxiv.org/abs/2006.06762).
- [12] Weng J, Jain A, Wang J, et al. Unit: Unifying tensorized instruction compilation [C/OL]//2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). 2021: 77-89. DOI: [10.1109/CGO51591.2021.9370330](https://doi.org/10.1109/CGO51591.2021.9370330).
- [13] Zheng S, Chen R, Wei A, et al. Amos: Enabling automatic mapping for tensor computations on spatial accelerators with hardware abstraction [C/OL]//Proceedings of the 49th Annual International Symposium on Computer Architecture. 2022: 874-887. DOI: [10.1145/3470496.3527440](https://doi.org/10.1145/3470496.3527440).
- [14] Feng S, Hou B, Jin H, et al. Tensorir: An abstraction for automatic tensorized program optimization [C/OL]//Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 2023. <https://arxiv.org/abs/2207.04296>. DOI: [10.1145/3575693.3576933](https://doi.org/10.1145/3575693.3576933).

- [15] Zhu H, Wu R, Diao Y, et al. Roller: Fast and efficient tensor compilation for deep learning [C/OL]//USENIX Symposium on Operating Systems Design and Implementation. 2022. <https://api.semanticscholar.org/CorpusID:266160403>. DOI: 10.48550/arXiv.2208.00226.
- [16] Shi Y, Yang Z, Xue J, et al. Welder: Scheduling deep learning memory access via tile-graph [C/OL]//17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23). 2023: 701-718. DOI: 10.48550/arXiv.2308.04985.
- [17] Zhong R, Jin Y, Zhang C, et al. Flashtensor: Optimizing tensor programs by leveraging fine-grained tensor property [C/OL]//Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming. 2025: 183-196. DOI: 10.1145/3710848.3710864.
- [18] Tillet P, Kung H T, Cox D. Triton: an intermediate language and compiler for tiled neural network computations [C/OL]//Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages. 2019: 10-19. DOI: 10.1145/3315508.3329973.
- [19] Chen H, Fan B, et al. Tawa: Automatic warp specialization for modern GPUs with asynchronous references [C]//Proceedings of the 24th ACM/IEEE International Symposium on Code Generation and Optimization (CGO '26). 2026.
- [20] Cheng Y, Wang L, Shi Y, et al. Pipethreader: software-defined pipelining for efficient dnn execution [C]//OSDI '25: Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation. USA: USENIX Association, 2025.
- [21] Chen H, Novikov A, Vū N, et al. Magellan: Autonomous discovery of novel compiler optimization heuristics with alpheavolve [EB/OL]. 2026. <https://doi.org/10.48550/arXiv.2601.21096>.
- [22] NVIDIA. NVIDIA Volta architecture whitepaper [EB/OL]. 2017. <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>.
- [23] Guan Y, Fang Y, Zhou K, et al. KPerfIR: Towards an open and compiler-centric ecosystem for GPU kernel performance tooling on modern AI workloads [C]//19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25). 2025.
- [24] Bauer M, Cook H, Khailany B. CudaDMA: Optimizing GPU memory bandwidth via warp specialization [C/OL]//Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11). New York, NY, USA: Association for Computing Machinery, 2011. DOI: 10.1145/2063384.2063400.
- [25] Bauer M, Treichler S, Aiken A. Singe: Leveraging warp specialization for high performance on GPUs [C/OL]//Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '14). New York, NY, USA: Association for Computing Machinery, 2014: 119-130. DOI: 10.1145/2555243.2555258.
- [26] NVIDIA. CUTLASS: CUDA templates for linear algebra subroutines [EB/OL]. 2025. <https://github.com/NVIDIA/cutlass>.
- [27] Dong J, Feng B, Guessous D, et al. Flexattention: A programming model for generating optimized attention kernels [C/OL]//Proceedings of Machine Learning and Systems: volume 7. 2025. DOI: 10.48550/arXiv.2412.05496.
- [28] Chen F, Cheng Y, Wang L, et al. Attentionengine: A versatile framework for efficient attention mechanisms on diverse hardware platforms [J/OL]. arXiv preprint arXiv:2502.15349, 2025. DOI: 10.48550/arXiv.2502.15349.

- [29] Kwon W, Li Z, Zhuang S, et al. Efficient memory management for large language model serving with pagedattention [C/OL]//Proceedings of the 29th symposium on operating systems principles. 2023: 611-626. DOI: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).
- [30] Prabhu R, Nayak A, Mohan J, et al. vattention: Dynamic memory management for serving llms without pagedattention [C/OL]//Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1. 2025: 1133-1150. DOI: [10.1145/3669940.3707256](https://doi.org/10.1145/3669940.3707256).
- [31] Ye Z, Chen L, Lai R, et al. FlashInfer: Efficient and customizable attention engine for LLM inference serving [C/OL]//Proceedings of Machine Learning and Systems: volume 7. 2025. DOI: [10.48550/arXiv.2501.01005](https://doi.org/10.48550/arXiv.2501.01005).
- [32] Chen H, Xie W, Zhang B, et al. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models [C/OL]//SOSP '25: Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles. New York, NY, USA: Association for Computing Machinery, 2025: 1014-1029. DOI: [10.1145/3731569.3764843](https://doi.org/10.1145/3731569.3764843).
- [33] Xing J, Wang L, Zhang S, et al. Bolt: Bridging the gap between auto-tuners and hardware-native performance [J/OL]. Proceedings of Machine Learning and Systems, 2022, 4: 204-216. DOI: [10.48550/arXiv.2206.13643](https://doi.org/10.48550/arXiv.2206.13643).
- [34] Wang H, Zhai J, Gao M, et al. PET: Optimizing tensor programs with partially equivalent transformations and automated corrections [C/OL]//15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21). 2021: 37-54. DOI: [10.48550/arXiv.2107.10726](https://doi.org/10.48550/arXiv.2107.10726).
- [35] Jia Z, Padon O, Thomas J, et al. Taso: optimizing deep learning computation with automatic generation of graph substitutions [C/OL]//Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 47-62. DOI: [10.1145/3341301.3359630](https://doi.org/10.1145/3341301.3359630).
- [36] Niu W, Guan J, Wang Y, et al. Dnnfusion: accelerating deep neural networks execution with advanced operator fusion [C/OL]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021: 883-898. DOI: [10.1145/3453483.3454083](https://doi.org/10.1145/3453483.3454083).
- [37] Jung W, Dao T T, Lee J. Deepcuts: a deep learning optimization framework for versatile gpu workloads [C/OL]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021: 190-205. DOI: [10.1145/3453483.3454038](https://doi.org/10.1145/3453483.3454038).
- [38] Zhu L, Yao J, Guan H. Spacefusion: Advanced deep learning operator fusion via space-mapping graph [C/OL]//Proceedings of the Twentieth European Conference on Computer Systems. 2025: 787-802. DOI: [10.1145/3689031.3696087](https://doi.org/10.1145/3689031.3696087).
- [39] Zheng Z, Yang X, Zhao P, et al. Astitch: enabling a new multi-dimensional optimization space for memory-intensive ml training and inference on modern simt architectures [C/OL]//Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2022: 359-373. DOI: [10.1145/3503222.3507723](https://doi.org/10.1145/3503222.3507723).
- [40] Zheng S, Chen S, Song P, et al. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion [C/OL]//2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA). IEEE, 2023: 1113-1126. DOI: [10.1109/HPCA56546.2023.10071018](https://doi.org/10.1109/HPCA56546.2023.10071018).
- [41] Zhang Z, Yang D, Zhou X, et al. Mcfuser: High-performance and rapid fusion of memory-bound compute-intensive operators [C/OL]//SC24: International Conference for High Perfor-

- mance Computing, Networking, Storage and Analysis. IEEE, 2024: 1-15. DOI: [10.1109/SC41406.2024.00040](https://doi.org/10.1109/SC41406.2024.00040).
- [42] Zhao J, Gao X, Xia R, et al. Apollo: Automatic partition-based operator fusion through layer by layer optimization [J/OL]. Proceedings of Machine Learning and Systems, 2022, 4: 1-19. DOI: [10.48550/arXiv.2210.09435](https://doi.org/10.48550/arXiv.2210.09435).
- [43] Lam M. Software pipelining: An effective scheduling technique for VLIW machines [C/OL]// Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation (PLDI '88). New York, NY, USA: Association for Computing Machinery, 1988: 318-328. DOI: [10.1145/960116.54022](https://doi.org/10.1145/960116.54022).
- [44] Rau B R. Iterative modulo scheduling: An algorithm for software pipelining loops [C/OL]// Proceedings of the 27th Annual International Symposium on Microarchitecture (MICRO 27). New York, NY, USA: Association for Computing Machinery, 1994: 63-74. DOI: [10.1145/192724.192731](https://doi.org/10.1145/192724.192731).
- [45] Govindarajan R, Altman E, Gao G. Minimizing register requirements under resource-constrained rate-optimal software pipelining [C/OL]//Proceedings of MICRO-27: The 27th Annual IEEE/ACM International Symposium on Microarchitecture. 1994: 85-94. DOI: [10.1109/MICRO.1994.717418](https://doi.org/10.1109/MICRO.1994.717418).
- [46] NVIDIA. Hopper architecture whitepaper. [EB/OL]. 2022. <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>.
- [47] NVIDIA. Parallel thread execution ISA version 8.5 [EB/OL]. 2025. <https://docs.nvidia.com/cuda/parallel-thread-execution/>.
- [48] Wang L, Cheng Y, Shi Y, et al. Tilelang: A composable tiled programming model for ai systems [EB/OL]. 2025. <https://arxiv.org/abs/2504.17577>. DOI: [10.48550/arXiv.2504.17577](https://doi.org/10.48550/arXiv.2504.17577).
- [49] Soi R, Yadav R, Kjolstad F, et al. Optimal software pipelining and warp specialization for tensor core GPUs [C/OL]//Proceedings of the Symposium on Operating Systems Design and Implementation (OSDI). 2026. DOI: [10.48550/arXiv.2512.18134](https://doi.org/10.48550/arXiv.2512.18134).
- [50] PyTorch Team. Helion: A high-level dsl for performant and portable ml kernels [Z]. 2025.
- [51] Tillet P. Triton: Past, present, and future [Z]. 2025.

致 谢

光阴荏苒，三年的计算所求学生涯即将落下帷幕。回首这段充实而宝贵的时光，有太多需要感恩的人。

首先，我要向我的导师肖俊敏研究员致以最诚挚的谢意。感谢肖老师在研究生期间对我的悉心指导，正是您的知遇之恩，让我有幸进入计算所学习，从而收获了一笔宝贵的精神财富。从选题方向的把控到技术路线的推敲，您总能在关键节点上给出准确的判断，让我在每次讨论后都能拨云见日。您严谨求实的科研态度和渊博深厚的专业知识，让我受益匪浅。

其次，我要特别感谢中心主任谭光明研究员。感谢您为我们营造了浓厚的科研氛围与优越的工作环境，使我们能够心无旁骛地专注学术。您在百忙之中仍不时抽出时间与学生们交流，关心大家的内心想法，并不断鼓励我们“科研要为国分忧”，这份家国情怀令我深感敬佩。同时，感谢贾伟乐老师、邵恩老师、王银山老师、姚二林老师和袁良老师在不同方面给予的专业指导，让我在编译优化和体系结构领域有了更开阔的视野。

感谢实验室里一起学习、工作、生活的同学们。感谢陈思言师兄和李鸣一师兄，从最初的代码规范到后来的系统设计，是你们手把手带我入门；感谢王耀宇、王泽华、王康宇、庞云飞几位师兄在不同阶段给予我的热心帮助与宝贵建议。感谢我的同门罗源、王会兵、代瀚坤，大家并肩作战的日子虽然辛苦，却是我记忆中闪光的快乐时光。特别感谢毕泽峰，我们研究方向相近，许多灵感都是在反复的思维碰撞中逐渐成型，不少 bug 也是在互相讨论想法，互相审核代码时得以解决。

感谢我的父母、朋友和亲人们。感谢你们在背后默默的付出与支持，你们是我不断前进的动力，也是我疲惫时可以停靠的温暖港湾。感谢你们带给我的无私关爱与感动，我会加倍努力，绝不辜负你们的殷切期望。

最后，感谢各位专家评委在百忙之中对本论文进行的悉心批评与指正！

2026 年 6 月

作者简历及攻读学位期间发表的学术论文与其他相关学术成果

作者简历：

陈茜，女，汉族，中共党员，山东省济南市人。

2019 年 9 月——2023 年 6 月，在湖南大学信息科学与工程学院获得学士学位。

2023 年 9 月——2026 年 6 月，在中国科学院大学计算技术研究所攻读硕士学位。

已发表（或正式接受）的学术论文：

- (1) Jiashu Yao, Junmin Xiao, Baokang Xie, Shilong Xu, **Xi Chen**, Yunfei Pang, Mingyi Li, Hui Ma, Yun Song, Guangming Tan. Hiperti: High Performance System for Cross-Platform Code Generation of Transformer Model Inference Based on MLIR. *CCF Transactions on High Performance Computing*, 7(6): 589–622, 2025.
- (2) Mingyi Li, Junmin Xiao, Siyan Chen, Hui Ma, **Xi Chen**, Peihua Bao, Liang Yuan, Guangming Tan. Stencil-Lifting: Hierarchical Recursive Lifting System for Extracting Summary of Stencil Kernel in Legacy Codes. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2): 3037–3064, 2025.

参与的研究项目：

- (1) “卷积神经网络训练与推理的并行优化研究”，国家自然科学基金面上项目，2022 年 01 月至 2025 年 12 月
- (2) “面向 E 级计算的性能可移植的建模及优化技术”，国家自然科学基金重点项目，2021 年 01 月至 2025 年 12 月
- (3) “多尺度自适应环境大气动力学模式与再分析数据集” 第一课题 “三维动态自适应环境大气模式 Fluidity-AEM 动力框架”，国家重点研发计划 “重大共性关键技术” 重点专项，2024 年 01 月至 2026 年 12 月

获奖情况：

- (1) 2023-2024 学年中国科学院大学三好学生
- (2) 2023-2024 学年中国科学院大学优秀学生干部
- (3) 2023-2024 学年中国科学院大学优秀共青团员
- (4) 2024-2025 学年中国科学院大学三好学生
- (5) 2024-2025 学年中国科学院大学优秀学生干部

